NetApp上的BeeGFS与E系列存储

BeeGFS on NetApp with E-Series Storage

NetApp
March 21, 2024

This PDF was generated from https://docs.netapp.com/zh-cn/beegfs/index.html on March 21, 2024.
Always check docs.netapp.com for the latest.

目录

NetApp上的BeeGFS与E系列存储 . . . 1
入门 . . . 2
此站点包含的内容 . . . 2
术语和概念 . . . 2
使用经验证的架构 . . . 4
概述和要求 . . . 4
查看解决方案 设计 . . . 11
部署解决方案 . . . 22
使用自定义架构 . . . 66
概述和要求 . . . 66
初始设置 . . . 67
定义BeeGFS文件系统 . . . 70
部署BeeGFS文件系统 . . . 94
管理BeeGFS集群 . . . 105
概述、关键概念和术语 . . . 105
何时使用Ansible与pcs工具 . . . 106
检查集群的状态 . . . 106
重新配置和更新 . . . 108
维护和维护 . . . 111
故障排除 . . . 117
法律声明 . . . 124
版权 . . . 124
商标 . . . 124
专利 . . . 124
隐私政策 . . . 124
开放源代码 . . . 124

NetApp上的BeeGFS与E系列存储

入门

此站点包含的内容

本站点介绍了如何使用NetApp验证架构(NetApp Verified Architectures、NVA)和自定义架构在NetApp上部署和管理BeeGFS。NVA设计经过全面测试、可为客户提供参考配置和规模估算指导、从而最大程度地降低部署风险并加快上市速度。NetApp还支持在NetApp硬件上运行的自定义BeeGFS架构、使客户和合作伙伴可以灵活地设计文件系统、以满足广泛的要求。这两种方法都利用Ansible进行部署、从而提供了一种类似于设备的方法、可以在灵活的硬件范围内以任意规模管理BeeGFS。

术语和概念

以下术语和概念适用于NetApp解决方案 上的BeeGFS。

请参见 "管理BeeGFS集群" 第节、了解有关与BeeGFS高可用性(HA)集群交互的特定术语和概念的更多详细信息。

期限	**Description**
AI	人工智能。
可逆清单	包含用于描述所需BeeGFS HA集群的YAML文件的目录结构。
BMC	基板管理控制器。有时称为服务处理器。
块节点	存储系统。
客户端	HPC集群中运行需要利用文件系统的应用程序的节点。有时也称为计算或GPU节点。
深度	深度学习。
文件节点	BeeGFS文件服务器。
HA	高可用性。
HIC	主机接口卡。
HPC	高性能计算。

期限	**Description**
HPC模式的工作负载	HPC模式的工作负载通常具有多个计算节点或GPU、这些节点或GPU都需要并行访问同一数据集、以便执行分布式计算或培训作业。这些数据集通常由大型文件组成、这些文件应在多个物理存储节点之间进行条带化、以消除传统硬件瓶颈、避免同时访问单个文件。
毫升	机器学习。
NLP	自然语言处理。
NLU	自然语言理解。
NVA	NetApp经验证的架构(NVA)计划为特定工作负载和使用情形提供参考配置和规模估算指导。这些解决方案经过全面测试、旨在最大程度地降低部署风险并加快上市速度。
存储网络/客户端网络	客户端用来与BeeGFS文件系统通信的网络。这通常与HPC集群节点之间用于并行消息传递接口(MPI)和其他应用程序通信的网络相同。

使用经验证的架构

概述和要求

解决方案概述

NetApp解决方案 上的BeeGFS将BeeGFS并行文件系统与NetApp EF600存储系统相结合、打造了一个可靠、可扩展且经济高效的基础架构、可满足苛刻的工作负载要求。

这种设计利用了最新企业级服务器和存储硬件以及网络速度所提供的性能密度、 要求文件节点采用双AMD EPYC 7003 "米兰"处理器并支持使用200 GB (HDR) InfiniBand直接连接的PCIe 4.0、以块节点使用NVMe/IB协议提供端到端NVMe和NVMeOF。

NVA计划

NetApp解决方案 上的BeeGFS是NetApp验证架构(NVA)计划的一部分、该计划为客户提供针对特定工作负载和使用情形的参考配置和规模估算指导。NVA解决方案经过全面测试和设计、可最大限度地降低部署风险并加快上市速度。

用例

以下使用情形适用于NetApp解决方案 上的BeeGFS：

- 人工智能(AI)、包括机器学习(ML)、深度学习(DL)、大规模自然语言处理(NLP)和自然语言理解(NLF)。有关详细信息，请参见 "BeeGFS for AI：事实与虚构"。
- 高性能计算(HPC)、包括通过MPI (消息传递接口)和其他分布式计算技术加速的应用程序。有关详细信息，请参见 "BeeGFS为何超越HPC"。
- 应用程序工作负载的特征为：
  - 读取或写入大于1 GB的文件
  - 多个客户端(10、100和1000)读取或写入同一文件
- 多TB或多PB数据集。
- 需要一个存储命名空间的环境、可以优化大小文件的组合。

优势

在NetApp上使用BeeGFS的主要优势包括：

- 提供经验证的硬件设计、可实现硬件和软件组件的全面集成、以确保可预测的性能和可靠性。
- 使用Ansible进行部署和管理、以实现规模化的精简性和一致性。
- 使用E系列性能分析器和BeeGFS插件提供监控和可观察性。有关详细信息，请参见 "介绍用于监控NetApp E系列解决方案的框架"。
- 高可用性采用共享磁盘架构、可提供数据持久性和可用性。
- 支持使用容器和Kubernetes进行现代工作负载管理和流程编排。有关详细信息，请参见 "Kubernetes认识BeeGFS：一个适应未来需求的投资故事"。

HA架构

NetApp上的BeeGFS通过创建与NetApp硬件完全集成的解决方案 来扩展BeeGFS企业版的功能、从而实现共享磁盘高可用性(HA)架构。

> 虽然BeeGFS社区版可以免费使用、但企业版要求从NetApp等合作伙伴购买专业支持订阅合同。企业版支持使用多项附加功能、包括故障恢复能力、配额强制实施和存储池。

下图对无共享和共享磁盘HA架构进行了比较。

[]

有关详细信息，请参见 "宣布NetApp支持的BeeGFS高可用性"。

Ansible

NetApp上的BeeGFS是使用Ansible自动化交付和部署的、Ansible自动化托管在GitHub和Ansible GALAXY上(BeeGFS集合可从获取 "Ansible Galax河" 和 "NetApp的E系列GitHub"）。虽然Ansible主要是使用用于组装BeeGFS组件的硬件进行测试的、但您可以将其配置为使用受支持的Linux版本在几乎任何基于x86的服务器上运行。

有关详细信息，请参见 "使用E系列存储部署BeeGFS"。

设计代数

NetApp解决方案 上的BeeGFS目前正处于第二代设计阶段。

第一代和第二代均采用一种基础架构、其中整合了BeeGFS文件系统和NVMe EF600存储系统。但是、第二代基于第一代产品构建、其中包括以下附加优势：

- 将性能和容量提高一倍、同时仅添加2U机架空间
- 高可用性(High Availability、HA)基于共享磁盘的两层硬件设计
- NVIDIA DGX A100 SuperPOD和NVIDIA BasePOD架构的外部认证

第二代设计

基于NetApp的第二代BeeGFS经过优化、可满足高性能计算(HPC)和HPC模式机器学习(ML)、深度学习(DL)以及类似人工智能(AI)技术等苛刻工作负载的性能要求。通过整合共享磁盘高可用性(HA)架构、NetApp解决方案 上的BeeGFS还可以满足企业和其他组织在寻找可扩展的存储以适应其工作负载和使用情形时无法承受停机或数据丢失的需求。此解决方案 不仅已通过NetApp的验证、还通过了外部认证、作为NVIDIA DGX SuperPOD和DGX BasePOD的存储选项。

第一代设计

第一代基于NetApp的BeeGFS专为使用NetApp EF600 NVMe存储系统、BeeGFS并行文件系统、NVIDIA DGX®A100系统和NVIDIA®Mellanox®Quantum®QM8700 200 Gbps IB交换机的机器学习(ML)和人工智能(AI)工作负载而设计。此设计还为存储和计算集群互连网络结构提供了200 Gbps InfiniBand (IB)、可为高性能工作负载提供完全基于IB的架构。

有关第一代产品的详细信息、请参见 "采用NVIDIA DGX A100系统和BeeGFS的NetApp EF系列AI"。

架构概述

NetApp解决方案 上的BeeGFS包括用于确定支持经验证的工作负载所需的特定设备、布线和配置的架构设计注意事项。

构建块架构

根据存储要求、可以使用不同的方式部署和扩展BeeGFS文件系统。例如、以大量小文件为主要特征的使用情形将受益于额外的元数据性能和容量、而大文件较少的使用情形则可能有利于提高实际文件内容的存储容量和性能。这些多种注意事项会影响并行文件系统部署的不同维度、从而增加了文件系统设计和部署的复杂性。

为了应对这些挑战、NetApp设计了一个标准的构建块架构、用于横向扩展其中的每个方面。通常、BeeGFS组件部署在以下三种配置文件之一中：

- 一个基础组件、包括BeeGFS管理、元数据和存储服务
- BeeGFS元数据加上存储组件
- 仅供BeeGFS存储使用的组件

这三个选项之间的唯一硬件更改是使用较小的驱动器来获取BeeGFS元数据。否则、所有配置更改将通过软件应用。借助Ansible作为部署引擎、为特定组件设置所需的配置文件可以使配置任务变得简单明了。

有关更多详细信息、请参见 [经验证的硬件设计]。

文件系统服务

BeeGFS文件系统包括以下主要服务：

- *管理服务。*注册并监控所有其他服务。
- *存储服务。*存储称为数据区块文件的分布式用户文件内容。
- *元数据服务。*跟踪文件系统布局、目录、文件属性等。
- *客户端服务。*挂载文件系统以访问存储的数据。

下图显示了NetApp E系列系统中使用的BeeGFS解决方案 组件和关系。

[]

作为一个并行文件系统、BeeGFS会将其文件条带化到多个服务器节点上、以最大限度地提高读/写性能和可扩展性。服务器节点协同工作、可提供一个文件系统、其他服务器节点(通常称为_clients_）可以同时挂载和访问该文件系统。这些客户端可以查看和使用分布式文件系统、类似于NTFS、XFS或ext4等本地文件系统。

这四项主要服务在广泛支持的Linux分发版上运行、并通过任何支持TCP/IP或RDMA的网络进行通信、包括InfiniBand (IB)、OMNI-Path (OPA)和基于融合以太网的RDMA (RoCE)。BeeGFS服务器服务(管理、存储和元数据)是用户空间守护进程、而客户端是原生 内核模块(无修补)。所有组件均可在不重新启动的情况下安装或更新、您可以在同一节点上运行任何服务组合。

已验证节点

NetApp解决方案 上的BeeGFS包括以下经验证的节点：NetApp EF600存储系统(块节点)和联想ThinkSystem SR665服务器(文件节点)。

块节点：**EF600**存储系统

NetApp EF600全闪存阵列可提供一致、近乎实时的数据访问、同时支持任意数量的工作负载。为了快速、持续地向AI和HPC应用程序馈送数据、EF600存储系统在一个机箱中提供高达200万个缓存读取IOPS、低于100微秒的响应时间以及42 GBps顺序读取带宽。

文件节点：**Lenovo ThinkSystem SR665**服务器

SR665是一款采用PCIe 4.0的双插槽2U服务器。如果配置为满足此解决方案 的要求、则它可以提供充足的性能、以便在与直连E系列节点提供的吞吐量和IOPS相当的配置中运行BeeGFS文件服务。

有关联想SR665的详细信息、请参见 "联想的网站"。

经验证的硬件设计

该解决方案的组件(如下图所示)使用两个双插槽PCIe 4.0服务器作为BeeGFS文件层、并使用两个EF600存储系统作为块层。

[]

> 由于每个组件都包含两个BeeGFS文件节点、因此要在故障转移集群中建立仲裁、至少需要两个组件。虽然您可以配置双节点集群、但此配置具有一些限制、可能会阻止成功进行故障转移。如果您需要双节点集群、则可以将第三个设备作为Tiebreaker (但是、此站点不会介绍此设计)。

每个组件都通过两层硬件设计提供高可用性、这种设计可分隔文件层和块层的故障域。每个层都可以独立进行故障转移、从而提高故障恢复能力并降低级联故障的风险。将HDR InfiniBand与NVMeOF结合使用可在文件和块节点之间提供高吞吐量和最低延迟、并提供完全冗余和足够的链路超额预订、以避免分散设计成为瓶颈、即使系统部分降级也是如此。

NetApp解决方案 上的BeeGFS可跨部署中的所有组件运行。部署的第一个组件必须运行BeeGFS管理、元数据和存储服务(称为基础组件)。所有后续组件均通过软件配置为运行BeeGFS元数据和存储服务、或者仅运行存储服务。由于每个组件都具有不同的配置配置文件、因此可以使用相同的底层硬件平台和组件设计扩展文件系统元数据或存储容量和性能。

一个独立的Linux HA集群最多可组合五个组件、从而确保每个集群资源管理器(Pacemaker)有合理数量的资源、并降低保持集群成员同步所需的消息传送开销(Corosync)。建议每个集群至少有两个组件、以便有足够的成员建立仲裁。将这些独立BeeGFS HA集群中的一个或多个组合在一起、以创建一个BeeGFS文件系统(如下图所示)、此文件系统可作为一个存储命名空间供客户端访问。

[]

尽管每个机架的组件数量最终取决于给定站点的电源和散热要求、 解决方案 经过专门设计、可在一个42U机架中部署多达五个组件、同时仍可为用于存储/数据网络的两个1U InfiniBand交换机提供空间。每个构建块需要八个IB端口(每个交换机四个用于实现冗余)、因此五个构建块会在一个40端口HDR InfiniBand交换机(如NVIDIA QM8700)上保留一半的端口、以便实施ft-tree或类似的非阻塞拓扑。此配置可确保存储或计算/GPU机架的数量可以纵向扩展、而不会出现网络瓶颈。或者、可以根据存储网络结构供应商的建议使用超额预订的存储网络结构。

下图显示了一个80节点的ft-tree拓扑。

[]

通过使用Ansible作为部署引擎在NetApp上部署BeeGFS、管理员可以使用现代基础架构作为代码实践来维护整

个环境。这样可以极大地简化原本复杂的系统、使管理员可以在一个位置定义和调整所有配置、然后确保无论环境扩展多大、都能始终如一地应用。可从获取BeeGFS集合 "Ansible Galax河" 和 "NetApp的E系列GitHub"。

技术要求

要在NetApp解决方案 上实施BeeGFS、请确保您的环境满足技术要求。

硬件要求

下表列出了在NetApp解决方案 上实施BeeGFS的单个第二代组件设计所需的硬件组件。

在任何特定解决方案 实施中使用的硬件组件可能会因客户要求而异。

计数	硬件组件	要求
2.	BeeGFS文件节点。	每个文件节点都应满足或超出以下配置、以实现预期性能。 处理器： • 2个AMD EPYC 7343 16C 3.2 GHz。 • 配置为两个NUMA分区。 内存： • 256 GB。 • 16个16 GB TruDDR4 3200MHz (2Rx8 1.2V) RDIMM-A (首选较小的DIMM、而不是较大的DIMM)。 • 填充以最大程度地提高内存带宽。 • PCIe扩展：四个PCE第4代x16插槽：* • 每个NUMA分区有两个插槽。 • 每个插槽都应为Mellanox MCX653106A-HDAT适配器提供足够的电源/散热。 其他： • 在RAID 1中为操作系统配置两个1 TB 7.2K SATA驱动器(或类似驱动器)。 • 10GbE OCP 3.0适配器(或类似适配器)、用于带内操作系统管理。 • 采用Redfish API的1GbE BMC、用于带外服务器管理。 • 双热插拔电源和高性能风扇。 • 如果需要连接到存储InfiniBand交换机、则必须支持Mellanox光纤InfiniBand缆线。 联想**SR665**： • 自定义NetApp型号包括支持双端口Mellanox ConnectX-6适配器所需的XiclityL控制器固件版本。有关订购详细信息、请联系NetApp。

计数	硬件组件	要求
8.	Mellanox ConnectX-6 HCA (适用于文件节点)。	• MCX653106A-HDAT主机通道适配器(HDR IB 200GB、双端口QSFP56、PCIe4.0 x16)。
8.	100万根HDR InfiniBand缆线(用于文件/块节点直接连接)。	• MCP1650-H001E30 (1米Mellanox无源铜缆、IB HDR、高达200 Gbps、QSFP56、30AWM)。 如果需要、可以调整此长度、以考虑文件节点与块节点之间的距离较长。
8.	HDR InfiniBand缆线(用于文件节点/存储交换机连接)	需要长度适当的InfiniBand HDR缆线(QSFP56收发器)、才能将文件节点连接到存储叶交换机。可能的选项包括： • MCP1650-H002E26 (2米Mellanox无源铜缆、IB HDR、高达200 GB/秒、QSFP56、30AWM)。 • MFS1S00-H003E (3米Mellanox有源光缆、IB HDR、高达200 GB/秒、QSFP56)。
2.	E系列块节点	两个EF600控制器配置如下： • 内存：256 GB (每个控制器128 GB)。 • 适配器：2端口200 GB/HDR (NVMe/IB)。 • Drives：已配置为与所需容量匹配。

软件要求

为了获得可预测的性能和可靠性、NetApp解决方案 上的BeeGFS版本会使用实施解决方案 所需的特定版本软件组件进行测试。

软件部署要求

下表列出了在基于Ansible的BeeGFS部署中自动部署的软件要求。

软件	**version**
BeeGFS	7.2.6
核心同步	3.1.5-1
起搏器	2.1.0-8.
opensm	openssm-5.9.0 (从mlnx_OFED 5.4.1.0.3.0开始) 只有直接连接才需要启用虚拟化。

Ansible控制节点要求

NetApp解决方案 上的BeeGFS可从Ansible控制节点进行部署和管理。有关详细信息，请参见 "Ansible文档"。

下表中列出的软件要求特定于下面列出的NetApp BeeGFS Ansible集合版本。

软件	**version**
Ansible	2.11通过pip安装时：Ansible 4.7.0和Ansible核心< 2.12、>=2.11.6
Python	3.9.
其他Python软件包	加密-35.0.0、netaddr-0.8.0
BeeGFS Ansible资料集	3.0.0

文件节点要求

软件	**version**
RedHat Enterprise Linux	具有高可用性的RedHat 8.4 Server物理服务器(双插槽)。 文件节点需要有效的RedHat Enterprise Linux Server订阅和Red Hat Enterprise Linux高可用性附加软件。
Linux内核	4.18.0-305.25.1.el8_4.x86_64
InfiniBand / RDMA驱动程序	收件箱
ConnectX-6 HCA固件	固件：20.31.1014
PXE：3.6.0403	UEFI：14.24.0013

EF600块节点要求

软件	**version**
SANtricity 操作系统	11.70.2
NVSRAM	N6000-872834-D06.DLP
驱动器固件	适用于所用驱动器型号的最新版本。

其他要求

下表中列出的设备用于验证、但可以根据需要使用适当的替代设备。通常、NetApp建议运行最新版本的软件、以避免意外问题。

硬件组件	已安装软件
• 2个Mellanox MQM8700 200 GB InfiniBand交换机	• 固件3.9.2110

硬件组件	已安装软件
• 1个可变控制节点(虚拟化)：* • 处理器：Intel ® Xeon ® Gold 6146 CPU @ 3.20GHz • 内存：8 GB • 本地存储：24 GB	• CentOS Linux 8.4.2105 • 内核4.18.0-305.3.1.el8.x86_64 已安装的Ansible和Python版本与上表中的版本匹配。
• 10个BeeGFS客户端(CPU节点)：* • 处理器：1个3.0 GHz AMD EPYC 7302 16核CPU • 内存：128 GB • 网络：2个Mellanox MCX653106A-HDAT (每个适配器连接一个端口)。	• Ubuntu 20.04 • 内核：5.4.0.100-generic • InfiniBand驱动程序：Mellanox OFED 5.3-1.0.3.0
• 1个BeeGFS客户端(GPU节点)：* • 处理器：2个2.5 GHz AMD EPYC 7742 64核CPU • 内存：1 TB • 网络：2个Mellanox MCX653106A-HDAT (每个适配器连接一个端口)。 此系统基于NVIDIA HGX A100平台、并包括四个A100 GPU。	• Ubuntu 20.04 • 内核：5.4.0.100-generic • InfiniBand驱动程序：Mellanox OFED 5.3-1.0.3.0

查看解决方案 设计

设计概述

要支持NetApp解决方案 上的BeeGFS、需要使用特定的设备、布线和配置、该BeeGFS并行文件系统与NetApp EF600存储系统相结合。

了解更多信息。

- "硬件配置"
- "软件配置"
- "设计验证"
- "规模估算准则"
- "性能调整"

设计和性能各不相同的衍生架构：

- "高容量组件"

硬件配置

NetApp上BeeGFS的硬件配置包括文件节点和网络布线。

文件节点配置

文件节点具有两个CPU插槽、这些插槽配置为单独的NUMA分区、其中包括对相同数量的PCIe插槽和内存的本地访问。

InfiniBand适配器必须填充到相应的PCI提升板或插槽中、以便在可用的PCIe通道和内存通道上平衡工作负载。您可以通过将单个BeeGFS服务的工作完全隔离到特定NUMA节点来平衡工作负载。目标是从每个文件节点获得类似的性能、就像两个独立的单插槽服务器一样。

下图显示了文件节点NUMA配置。

[]

BeeGFS进程固定到特定的NUMA区域、以确保使用的接口位于同一区域。此配置可避免通过插槽间连接进行远程访问。插槽间连接有时称为QPI或GMI2链路；即使在现代处理器架构中、也可能会在使用诸如HDR InfiniBand等高速网络时成为瓶颈。

网络布线配置

在一个构建块中、每个文件节点使用总共四个冗余InfiniBand连接连接连接到两个块节点。此外、每个文件节点还具有四个到InfiniBand存储网络的冗余连接。

在下图中、请注意：

- 绿色中所示的所有文件节点端口均用于连接到存储网络结构；所有其他文件节点端口均直接连接到块节点。
- 特定NUMA区域中的两个InfiniBand端口连接到同一块节点的A和B控制器。
- NUMA节点0中的端口始终连接到第一个块节点。
- NUMA节点1中的端口连接到第二个块节点。

[]

> 对于具有冗余交换机的存储网络、浅绿色端口应连接到一个交换机、深绿色端口应连接到另一个交换机。

图中所示的布线配置允许每个BeeGFS服务：

- 无论运行BeeGFS服务的文件节点是什么、都可以在同一NUMA区域中运行。
- 无论故障发生在何处、都要有通向前端存储网络和后端块节点的二级最佳路径。
- 如果块节点中的文件节点或控制器需要维护、则最大限度地降低性能影响。

布线以利用带宽

要利用完整的PCIe双向带宽、请确保每个InfiniBand适配器上的一个端口连接到存储网络结构、另一个端口连接到块节点。理论上、一个HDR InfiniBand端口的最大速度为25 GBps (不考虑信号传输和其他开销)。PCIe 4.0 x16插槽的最大单向带宽为32 Gbps、如果实施的文件节点采用双端口InfiniBand适配器、理论上可处理50 GB的带宽、则可能会造成瓶颈。

下图显示了用于利用完整PCIe双向带宽的布线设计。

[]

对于每个BeeGFS服务、请使用同一适配器将用于客户端流量的首选端口连接到作为该服务卷的主所有者的块节点控制器的路径。有关详细信息，请参见 "软件配置"。

软件配置

NetApp上的BeeGFS软件配置包括BeeGFS网络组件、EF600块节点、BeeGFS文件节点、资源组和BeeGFS服务。

BeeGFS网络配置

BeeGFS网络配置由以下组件组成。

- *浮动IP*浮动IP是一种虚拟IP地址、可以动态路由到同一网络中的任何服务器。多个服务器可以拥有相同的浮动IP地址、但在任何给定时间、它只能在一个服务器上处于活动状态。

  每个BeeGFS服务器服务都有自己的IP地址、可以根据BeeGFS服务器服务的运行位置在文件节点之间移动。此浮动IP配置允许每个服务单独故障转移到另一个文件节点。客户端只需知道特定BeeGFS服务的IP地址即可；它不需要知道哪个文件节点当前正在运行该服务。

- * BeeGFS服务器多主机配置*为了提高解决方案 的密度、每个文件节点都有多个存储接口、其IP配置在同一IP子网中。

  为了确保此配置在Linux网络堆栈中正常运行、需要进行其他配置、因为默认情况下、如果一个接口的IP位于同一子网中、则可以在其他接口上对对对对请求做出响应。除了其他缺点之外、此默认行为还会使您无法正确建立或维护RDMA连接。

  基于Ansible的部署可处理反向路径(RP)和地址解析协议(ARP)行为的收紧、并确保浮动IP何时启动和停止；系统会动态创建相应的IP路由和规则、以使多宿主网络配置正常运行。

- * BeeGFS客户端多导轨配置*_Multi-raid_是指应用程序能够使用多个独立的网络"导轨"来提高性能。

  虽然BeeGFS可以使用RDMA进行连接、但BeeGFS使用IPoIB简化发现和建立RDMA连接的过程。要允许BeeGFS客户端使用多个InfiniBand接口、您可以为每个客户端配置位于不同子网中的IP地址、然后为每个子网中一半的BeeGFS服务器服务配置首选接口。

  在下图中、以淡绿色突出显示的接口位于一个IP子网中(例如、`100.127.0.0/16`)、而深绿色接口位于另一个子网中(例如、`100.128.0.0/16`)。

  下图显示了在多个BeeGFS客户端接口之间平衡流量的情况。

  []

  由于BeeGFS中的每个文件通常在多个存储服务之间进行条带化、因此多导轨配置可以使客户端实现比单个InfiniBand端口更大的吞吐量。例如、以下代码示例显示了一个通用文件条带化配置、该配置允许客户端在两个接口之间平衡流量：

```
root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
Entry type: file
EntryID: 11D-624759A9-65
Metadata node: meta_01_tgt_0101 [ID: 101]
Stripe pattern details:
+ Type: RAID0
+ Chunksize: 1M
+ Number of storage targets: desired: 4; actual: 4
+ Storage targets:
  + 101 @ stor_01_tgt_0101 [ID: 101]
  + 102 @ stor_01_tgt_0101 [ID: 101]
  + 201 @ stor_02_tgt_0201 [ID: 201]
  + 202 @ stor_02_tgt_0201 [ID: 201]
```

使用两个IPoIB子网是一个逻辑上的区别。如果需要、您可以使用一个物理InfiniBand子网(存储网络)。

> BeeGFS 7.3.0增加了多轨支持、允许在一个IPoIB子网中使用多个IB接口。NetApp解决方案上的BeeGFS设计是在BeeGFS 7.3.0全面推出之前开发的、因此展示了如何使用两个IP子网在BeeGFS客户端上使用两个IB接口。多IP子网方法的一个优势是无需在BeeGFS客户端节点上配置多主节点(有关详细信息、请参见 "BeeGFS RDMA支持"）。

EF600块节点配置

块节点由两个主动/主动RAID控制器组成、这些控制器可共享访问同一组驱动器。通常、每个控制器拥有系统上配置的一半卷、但可以根据需要接管另一个控制器。

文件节点上的多路径软件可确定每个卷的活动和优化路径、并在发生缆线、适配器或控制器故障时自动移至备用路径。

下图显示了EF600块节点中的控制器布局。

[]

为了便于使用共享磁盘HA解决方案 、卷会映射到两个文件节点、以便它们可以根据需要相互接管。下图显示了如何配置BeeGFS服务和首选卷所有权以实现最高性能的示例。每个BeeGFS服务左侧的接口指示客户端和其他服务用来与其联系的首选接口。

[]

在上一示例中、客户端和服务器服务更愿意使用接口i1b与存储服务1进行通信。存储服务1使用接口i1a作为与其第一个块节点的控制器A上的卷(storage_tgt_101、102)进行通信的首选路径。此配置可利用InfiniBand适配器可用的全双向PCIe带宽、并通过双端口HDR InfiniBand适配器获得比PCIe 4.0更高的性能。

BeeGFS文件节点配置

BeeGFS文件节点配置到高可用性(HA)集群中、以便于在多个文件节点之间对BeeGFS服务进行故障转移。

HA集群设计基于两个广泛使用的Linux HA项目：用于集群成员资格的corosync和用于集群资源管理

的Pacemaker。有关详细信息，请参见 "针对高可用性附加组件的Red Hat培训"。

NetApp编写并扩展了多个开放式集群框架(Open Cluster Framework、OCF)资源代理、使集群能够智能启动和监控BeeGFS资源。

BeeGFS HA集群

通常、在启动BeeGFS服务(无论是否具有HA)时、必须具备一些资源：

- 可访问服务的IP地址、通常由Network Manager配置。
- 用作BeeGFS数据存储目标的底层文件系统。

  这些参数通常在`/etc/fstab`中定义、并由systemd挂载。

- 一种系统服务、负责在其他资源准备就绪时启动BeeGFS进程。

  如果没有其他软件、这些资源只会在一个文件节点上启动。因此、如果文件节点脱机、则无法访问BeeGFS文件系统的一部分。

由于多个节点可以启动每个BeeGFS服务、因此Pacemaker必须确保每个服务和相关资源一次仅在一个节点上运行。例如、如果两个节点尝试启动同一BeeGFS服务、则在它们都尝试写入底层目标上的相同文件时、存在数据损坏的风险。为了避免这种情况、Pacemaker会依靠Corosync可靠地在所有节点之间保持整个集群的状态同步并建立仲裁。

如果集群发生故障、Pacemaker会做出反应、并在另一节点上重新启动BeeGFS资源。在某些情况下、Pacemaker可能无法与原始故障节点通信以确认资源已停止。要在其他位置重新启动BeeGFS资源之前验证节点是否已关闭、Pacemaker最好断开故障节点的电源。

有许多开源隔离代理可用于使Pacemaker使用配电单元(PDU)或使用服务器基板管理控制器(BMC)和Redfo* 等API来隔离节点。

当BeeGFS在HA集群中运行时、所有BeeGFS服务和底层资源都由资源组中的Pacemaker管理。每个BeeGFS服务及其所依赖的资源都会配置到一个资源组中、以确保资源按正确顺序启动和停止、并放置在同一节点上。

对于每个BeeGFS资源组、Pacemaker都会运行一个自定义BeeGFS监控资源、该资源负责检测故障情况、并在特定节点上无法再访问BeeGFS服务时智能地触发故障转移。

下图显示了由Pacemaker控制的BeeGFS服务和依赖项。

[]

> 为了在同一节点上启动多个相同类型的BeeGFS服务、我们将Pacemaker配置为使用多模式配置方法启动BeeGFS服务。有关详细信息，请参见 "有关多模式的BeeGFS文档"。

由于BeeGFS服务必须能够在多个节点上启动、因此每个服务(通常位于`/etc/beegfs`)的配置文件存储在用作该服务的BeeGFS目标的E系列卷之一上。这样、可能需要运行此服务的所有节点都可以访问此配置以及特定BeeGFS服务的数据。

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

设计验证

NetApp解决方案 上的BeeGFS的第二代设计已使用三个组件配置文件进行了验证。

配置配置文件包括以下内容：

- 一个基础组件、包括BeeGFS管理、元数据和存储服务。
- BeeGFS元数据加上存储构建块。
- 一个BeeGFS纯存储组件。

这些组件连接到两个Mellanox Quantum InfiniBand (MQM8700)交换机。此外、还将10个BeeGFS客户端连接到InfiniBand交换机、并用于运行综合基准实用程序。

下图显示了用于验证NetApp解决方案 上的BeeGFS的BeeGFS配置。

[]

BeeGFS文件条带化

并行文件系统的一个优势是能够在多个存储目标之间对各个文件进行条带化、这可能表示同一个或不同底层存储系统上的卷。

在BeeGFS中、您可以按目录和文件配置条带化、以控制每个文件使用的目标数量、并控制每个文件条带使用的机克大小(或块大小)。此配置允许文件系统支持不同类型的工作负载和I/O配置文件、而无需重新配置或重新启动服务。您可以使用`beegfs-ctl`命令行工具或对使用条带化API的应用程序应用条带设置。有关详细信息、请参见的BeeGFS文档 "条带化" 和 "条带化API"。

为了获得最佳性能、我们在整个测试过程中对条带模式进行了调整、并记录了每个测试使用的参数。

IOR带宽测试：多个客户端

IOR带宽测试使用OpenMPI运行合成I/O生成器工具IOR的并行作业(可从获取 "HPC GitHub")连接到一个或多个BeeGFS组件。除非另有说明：

- 所有测试都使用传输大小为1 MiB的直接I/O。
- BeeGFS文件条带化设置为1 MB的机克大小、每个文件一个目标。

以下参数用于IOR、并对区块数进行了调整、以使一个组件的聚合文件大小保持在5 TiB、而三个组件的聚合文件大小保持在40 TiB。

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

一个BeeGFS基础(管理、元数据和存储)组件

下图显示了一个BeeGFS基础(管理、元数据和存储)组件的IOR测试结果。

[]

BeeGFS元数据+存储组件

下图显示了一个BeeGFS元数据+存储组件的IOR测试结果。

[]

BeeGFS纯存储组件

下图显示了一个BeeGFS纯存储组件的IOR测试结果。

[]

三个BeeGFS组件

下图显示了三个BeeGFS组件的IOR测试结果。

[]

正如预期、基础组件与后续元数据+存储组件之间的性能差异可忽略不计。比较元数据+存储构建块和纯存储构建块后、由于用作存储目标的驱动器增加、读取性能略有提高。但是、写入性能没有显著差异。为了提高性能、您可以将多个组件添加到一起、以线性方式扩展性能。

IOR带宽测试：单个客户端

IOR带宽测试使用OpenMPI通过一个高性能GPU服务器运行多个IOR进程、以探索单个客户端可实现的性能。

此测试还会将客户端配置为使用Linux内核页面缓存(`tuneFileCacheType =`原生)时BeeGFS的重新读取行为和性能与默认的`缓冲区`设置进行比较。

原生 缓存模式会在客户端上使用Linux内核页面缓存、从而使重新读取操作可以来自本地内存、而不是通过网络重新传输。

下图显示了三个BeeGFS组件和一个客户端的IOR测试结果。

[]

> 在这些测试中、BeeGFS条带化设置为1 MB的机克大小、每个文件有8个目标。

尽管使用默认缓冲模式时写入和初始读取性能较高、但对于多次重新读取相同数据的工作负载、原生 缓存模式会显著提升性能。对于深度学习等工作负载来说、这种提高的重新读取性能非常重要、因为深度学习会在许多时间内多次重新读取同一数据集。

元数据性能测试

元数据性能测试使用MDTest工具(包含在IOR中)测量BeeGFS的元数据性能。这些测试利用OpenMPI在所有十个客户端节点上运行并行作业。

以下参数用于运行基准测试、其中进程总数从10扩展到320、步长为2倍、文件大小为4 k。

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

元数据性能首先使用一到两个元数据+存储组件进行衡量、以显示性能如何通过添加更多组件进行扩展。

一个**BeeGFS**元数据**+**存储组件

下图显示了一个BeeGFS元数据+存储组件的MDTest结果。

[]

两个**BeeGFS**元数据**+**存储组件

下图显示了包含两个BeeGFS元数据+存储组件的MDTest结果。

[]

功能验证

在验证此架构的过程中、NetApp执行了多项功能测试、其中包括：

- 通过禁用交换机端口使单个客户端InfiniBand端口出现故障。
- 通过禁用交换机端口使单个服务器InfiniBand端口出现故障。
- 使用BMC触发服务器立即关闭。
- 妥善地将节点置于备用状态并将服务故障转移到另一节点。
- 妥善地将节点重新联机、并将服务故障恢复到原始节点。
- 使用PDU关闭其中一个InfiniBand交换机。所有测试均在压力测试期间执行、并在BeeGFS客户端上设置了`ssysSessionChecksEnabled：false`参数。未发现I/O错误或中断。

> 存在一个已知的问题描述 (请参见 "ChangeLog")当BeeGFS客户端/服务器RDMA连接因主接口丢失(如`connInterfacesFile`中所定义)或BeeGFS服务器发生故障而意外中断时、活动客户端I/O可能会挂起多达十分钟、然后才能恢复。如果BeeGFS节点已妥善置于待机状态和待机状态并处于待机状态以进行计划内维护、或者正在使用TCP、则不会发生此问题描述。

NVIDIA DGX A100 SuperPOD和BasePOD验证

NetApp使用一个类似的BeeGFS文件系统验证了用于NVIDIA DGX A100 SuperPOD的存储解决方案 、该文件系统由三个组件组成、并应用了元数据和存储配置文件。资格认定工作涉及使用20台DGX A100 GPU服务器测试此NVA所述的解决方案 、这些服务器运行各种存储、机器学习和深度学习基准。经认证可在NVIDIA DGX A100 SuperPOD中使用的所有存储也会自动通过认证、可在NVIDIA BasePOD架构中使用。

有关详细信息，请参见 "采用NetApp技术的NVIDIA DGX SuperPOD" 和 "NVIDIA DGX基本POD"。

规模估算准则

BeeGFS解决方案 提供了基于验证测试的性能和容量规模估算建议。

构建块架构的目标是、通过添加多个构建块来满足特定BeeGFS系统的要求、创建一个易于调整大小的解决方案。您可以根据以下准则估算满足环境要求所需的BeeGFS组件的数量和类型。

请记住、这些估计是最佳性能。我们编写并利用了综合基准测试应用程序、以便以实际应用程序可能无法实现的方式优化底层文件系统的使用。

性能规模估算

下表提供了建议的性能规模估算。

配置配置文件	**1 MiB**读取	**1 MiB**写入
元数据+存储	62 GiBps	21 GiBps
仅存储	64 GiBps	21 GiBps

元数据容量估算是根据"经验法则"估算的、即500 GB的容量足以容纳BeeGFS中大约1.5亿个文件。(有关详细信息、请参见的BeeGFS文档 "系统要求")

使用访问控制列表等功能以及每个目录的目录和文件数量也会影响元数据空间的使用速度。存储容量估计值会考虑可用驱动器容量以及RAID 6和XFS开销。

元数据**+**存储组件的容量规模估算

下表提供了元数据和存储组件的建议容量大小调整。

驱动器大小**(2+2 RAID 1)**元数据卷组	元数据容量**(**文件数**)**	驱动器大小**(8+2 RAID 6)**存储卷组	存储容量**(**文件内容**)**
1.92 TB	1、938、572、200	1.92 TB	51.77 TB
3.84 TB	3、880、388、400	3.84 TB	103.55 TB
7.68 TB	8、125、278、000	7.68 TB	216.74 TB
15.3 TB	17、262、854、000	15.3 TB	460.60 TB

> 在估算元数据和存储组件的规模时、您可以使用较小的驱动器作为元数据卷组、而不是存储卷组、从而降低成本。

仅存储组件的容量规模估算

下表提供了仅存储组件的容量规模估算规则。

驱动器大小(**10+2 RAID 6**)存储卷组	存储容量(文件内容)
1.92 TB	59.89 TB
3.84 TB	119.80 TB
7.68 TB	251.89 TB
15.3 TB	538.55 TB

> 除非启用全局文件锁定、否则将管理服务包含在基本(第一个)组件中所带来的性能和容量开销极小。

性能调整

BeeGFS解决方案 提供了基于验证测试的性能调整建议。

虽然BeeGFS可提供合理的即装即用性能、但NetApp已开发出一组建议的调整参数来最大限度地提高性能。这些参数考虑了底层E系列块节点的功能以及在共享磁盘HA架构中运行BeeGFS所需的任何特殊要求。

文件节点的性能调整

您可以配置的可用调整参数包括：

1. *文件节点的UEFI/BIOS中的系统设置。*为最大限度地提高性能、建议在用作文件节点的服务器型号上配置系统设置。在设置文件节点时、您可以使用由基板管理控制器(BMC)提供的系统设置(UEFI/BIOS)或Redfish API来配置系统设置。

   系统设置因用作文件节点的服务器型号而异。必须根据所使用的服务器型号手动配置这些设置。要了解如何为经过验证的联想SR665文件节点配置系统设置、请参见 "调整文件节点系统设置以提高性能"。

2. *所需配置参数的默认设置。*所需配置参数会影响BeeGFS服务的配置方式以及Pacemaker对E系列卷(块设备)进行格式化和挂载的方式。这些必需的配置参数包括：
   - BeeGFS服务配置参数

     您可以根据需要覆盖配置参数的默认设置。有关可根据特定工作负载或用例调整的参数、请参见 "BeeGFS服务配置参数"。

   - 卷格式化和挂载参数设置为建议的默认值、只能针对高级使用情形进行调整。默认值将执行以下操作：
     - 根据目标类型(例如管理、元数据或存储)以及底层卷的RAID配置和分段大小优化初始卷格式化。
     - 调整Pacemaker挂载每个卷的方式、以确保所做的更改立即刷新到E系列块节点。这样可以防止在文件节点发生故障且正在进行活动写入时丢失数据。

       有关可根据特定工作负载或用例调整的参数、请参见 "卷格式化和挂载配置参数"。

3. * Linux操作系统中安装在文件节点上的系统设置。*在的步骤4中创建Ansible清单时、您可以覆盖默认Linux操作系统设置 "创建Ansible清单"。

默认设置用于验证NetApp解决方案 上的BeeGFS、但您可以对其进行更改、以便根据您的特定工作负载或使用情形进行调整。您可以更改的Linux操作系统设置示例包括：

- E系列块设备上的I/O队列。

  您可以在用作BeeGFS目标的E系列块设备上配置I/O队列、以便：

  - 根据设备类型(NVMe、HDD等)调整计划算法。
  - 增加未处理的请求数。
  - 调整请求大小。
  - 优化预读行为。

- 虚拟内存设置。

  您可以调整虚拟内存设置、以获得最佳的持续流性能。

- CPU设置。

  您可以调整CPU频率调速器和其他CPU配置、以获得最高性能。

- 读取请求大小。

  您可以增加Mellanox HCA的最大读取请求大小。

对块节点进行性能调整

根据应用于特定BeeGFS组件的配置文件、在块节点上配置的卷组会略有变化。例如、对于一个24驱动器EF600块节点：

- 对于单个基础组件、包括BeeGFS管理、元数据和存储服务：
  - 1个2+2 RAID 10卷组、用于BeeGFS管理和元数据服务
  - 2个8+2 RAID 6卷组、用于BeeGFS存储服务
- 对于BeeGFS元数据+存储构建块：
  - 1个2+2 RAID 10卷组、用于BeeGFS元数据服务
  - 2个8+2 RAID 6卷组、用于BeeGFS存储服务
- 仅限BeeGFS存储组件：
  - 2个10+2 RAID 6卷组、用于BeeGFS存储服务

> 与存储相比、BeeGFS对管理和元数据所需的存储空间要少得多、因此可以选择为RAID 10卷组使用较小的驱动器。较小的驱动器应填充到最外侧的驱动器插槽中。有关详细信息，请参见 "部署说明"。

所有这些设置均由基于Ansible的部署进行配置、此外、通常还建议使用其他几种设置来优化性能/行为、其中包括：

- 将全局缓存块大小调整为32 KiB、并将基于需求的缓存刷新调整为80%。

- 禁用自动加载平衡(确保控制器卷分配保持预期状态)。
- 启用读取缓存和禁用预读缓存。
- 通过镜像启用写入缓存并要求使用电池备份、以便在块节点控制器发生故障时保持缓存不变。
- 指定驱动器分配给卷组的顺序、在可用驱动器通道之间平衡I/O。

高容量组件

标准BeeGFS解决方案 设计充分考虑了高性能工作负载。寻求高容量使用情形的客户应观察此处所述的设计和性能特征的差异。

硬件和软件配置

高容量组件的硬件和软件配置是标准配置、但EF600控制器应更换为EF300控制器、并可选择在1到7个IOM扩展托盘之间连接、每个存储阵列使用60个驱动器。每个组件总共有2到14个扩展托盘。

部署高容量构建块设计的客户可能仅使用基础构建块模式配置、该配置包含每个节点的BeeGFS管理、元数据和存储服务。为了节省成本、高容量存储节点应在EF300控制器机箱中的NVMe驱动器上配置元数据卷、并为扩展托盘中的NL-SAS驱动器配置存储卷。

[]

规模估算准则

这些规模估算准则假定高容量组件在基础EF300机箱中配置了一个2+2 NVMe SSD卷组用于元数据、并在每个IOM扩展托盘中配置了6个8+2 NL-SAS卷组用于存储。

驱动器大小(容量HDD)	每个BB的容量(1个托盘)	每个BB的容量(2个托盘)	每个BB的容量(3个托盘)	每个BB的容量(4个托盘)
4 TB	439 TB	878 TB	1317 TB	1756 TB
8 TB	878 TB	1756 TB	2634 TB	3512 TB
10 TB	1097 TB	2195 TB	3292 TB	4390 TB
12 TB	1317 TB	2634 TB	3951 TB	5268 TB
16 TB	1756 TB	3512 TB	5268 TB	7024 TB
18 TB	1975-TB	3951 TB	5927 TB	7902 TB

部署解决方案

部署概述

您可以使用NetApp的第二代BeeGFS组件设计将NetApp上的BeeGFS部署到经验证的文件和块节点。

Ansible集合和角色

您可以使用Ansible在NetApp解决方案 上部署BeeGFS、Ansible是一种常用的IT自动化引擎、用于自动执行应用程序部署。Ansible使用一系列文件、统称为清单、用于为要部署的BeeGFS文件系统建立模型。

Ansible允许NetApp等公司使用Ansible Galax星 上的集合扩展内置功能(请参见 "NetApp E系列BeeGFS集合" ）。集合包括执行某些特定功能或任务(例如创建E系列卷)的模块、以及可以调用多个模块和其他角色的角色。这种自动化方法可减少部署BeeGFS文件系统和底层HA集群所需的时间。此外、它还简化了添加组件以扩展现有文件系统的过程。

有关其他详细信息，请参见 "了解Ansible清单"。

由于在NetApp解决方案 上部署BeeGFS涉及许多步骤、因此NetApp不支持手动部署解决方案。

BeeGFS组件的配置配置文件

部署过程包括以下配置文件：

- 一个基础组件、其中包括管理、元数据和存储服务。
- 包含元数据和存储服务的第二个组件。
- 仅包含存储服务的第三个组件。

这些配置文件展示了NetApp BeeGFS组件的完整建议配置文件。对于每个部署、元数据和存储组件或仅存储服务组件的数量可能因容量和性能要求而有所不同。

部署步骤概述

部署涉及以下高级任务：

硬件部署

1. 以物理方式组装每个组件。
2. 机架和缆线硬件。有关详细过程、请参见 "部署硬件"。

软件部署

1. "设置文件和块节点"。
    - 在文件节点上配置BMC IP
    - 安装受支持的操作系统并在文件节点上配置管理网络
    - 在块节点上配置管理IP
2. "设置Ansible控制节点"。
3. "调整系统性能设置"。
4. "创建Ansible清单"。
5. "为BeeGFS组件定义Ansible清单"。
6. "使用Ansible部署BeeGFS"。
7. "配置BeeGFS客户端"。

> 部署过程包括几个示例、其中需要将文本复制到文件。请密切注意对于任何应针对特定部署进行修改或可修改的内容、任何以"" 或"/"字符表示的实时注释。例如：`beegfs_ha_ntp_server_pools：` This is an example of a comment！ -"pool 0.pool.ntp.org iBurst maxsources 3"-"pool 1.pool.ntp.org iBurst maxsources 3"

部署建议各不相同的衍生架构：

- "高容量组件"

了解Ansible清单

开始部署之前、请确保了解如何使用Ansible使用第二代BeeGFS组件设计在NetApp解决方案 上配置和部署BeeGFS。

Ansible清单定义了文件和块节点的配置、并表示要部署的BeeGFS文件系统。清单包括描述所需BeeGFS文件系统的主机、组和变量。可从下载示例清单 "NetApp E系列BeeGFS GitHub"。

Ansible模块和角色

要应用Ansible清单所述的配置、请使用NetApp E系列Ansible集合中提供的各种Ansible模块和角色、尤其是BeeGFS HA 7.2角色(可从获取 "NetApp E系列BeeGFS GitHub")部署端到端解决方案。

NetApp E系列Ansible资料集中的每个角色都是在NetApp解决方案 上完整地端到端部署BeeGFS。这些角色使用NetApp E系列SANtricity 、主机和BeeGFS集合、通过这些集合、您可以使用HA (高可用性)配置BeeGFS文件系统。然后、您可以配置和映射存储、并确保集群存储已准备就绪、可以使用。

虽然角色附带了深入的文档、但部署过程介绍了如何使用角色使用第二代BeeGFS组件设计部署经验证的NetApp架构。

> 尽管部署步骤会尝试提供足够详细的信息、以便事先使用Ansible并不是前提条件、但您应该对Ansible及相关术语有所熟悉。

BeeGFS HA集群的清单布局

使用Ansible清单结构定义BeeGFS HA集群。

任何以前具有Ansible经验的用户都应了解BeeGFS HA角色可通过一种自定义方法来发现每个主机适用的变量(或事实)。要简化Ansible清单的构建、使其能够描述可在多个服务器上运行的资源、需要执行此操作。

Ansible清单通常由`host_vars`和`group_vars`中的文件以及一个`inventory.yml`文件组成、该文件可将主机分配给特定组(并可能将组分配给其他组)。

> 请勿使用本小节中的内容创建任何文件、本小节仅用作示例。

尽管此配置是根据配置配置文件预先确定的、但您应大致了解如何将所有内容作为Ansible清单进行布局、如下所示：

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
        meta_01:  # Group representing a metadata service with ID 01.
          hosts:
            file_node_01:  # This service is preferred on the first file
node.
            file_node_02:  # And can failover to the second file node.
        meta_02:  # Group representing a metadata service with ID 02.
          hosts:
            file_node_02:  # This service is preferred on the second file
node.
            file_node_01: # And can failover to the first file node.
```

对于每个服务、将在`group_vars`下创建一个附加文件、用于描述其配置：

```
# meta_01 - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: 8015
  connMetaPortUDP: 8015
  tuneBindToNumaZone: 0
floating_ips:
  - i1b: <IP>/<SUBNET_MASK>
  - i4b: <IP>/<SUBNET_MASK>
# Type of BeeGFS service the HA resource group will manage.
beegfs_service: metadata # Choices: management, metadata, storage.
# What block node should be used to create a volume for this service:
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        owning_controller: A
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.25
```

通过此布局、可以在一个位置为每个资源定义BeeGFS服务、网络和存储配置。在后台、BeeGFS角色会根据此清单结构为每个文件和块节点聚合所需的配置。有关详细信息、请参见以下博客文章： "NetApp借助Ansible加快了BeeGFS HA的部署"。

> 系统会根据组名称自动配置每个服务的BeeGFS数字和字符串节点ID。因此、除了组名称唯一的一般可逆要求之外、表示BeeGFS服务的组必须以该组所代表的BeeGFS服务类型唯一的数字结尾。例如、允许使用meta_01和stor_01、但不允许使用metadata_01和metada_01。

查看最佳实践

在NetApp解决方案 上部署BeeGFS时、请遵循最佳实践准则。

标准约定

在物理组装和创建Ansible清单文件时、请遵循以下标准约定(有关详细信息、请参见 "创建Ansible清单"）。

- 文件节点主机名按顺序编号(h01-HN)、机架顶部的数字较低、底部的数字较高。

  例如、命名约定`[location][row][rack]HN`的外观如下：`ictad22h01`。

- 每个块节点由两个存储控制器组成、每个控制器都有自己的主机名。

  存储阵列名称用于在Ansible清单中引用整个块存储系统。存储阵列名称应按顺序编号(A01 - A)、各个控制器的主机名均根据该命名约定派生。

例如、名为`ictad22a01`的块节点通常可以为每个控制器配置主机名、例如`ictad22a01-A`和`ictad22a01-b`、但在Ansible清单中称为`ictad22a01`。

- 同一构建块中的文件和块节点共享相同的编号方案、并在机架中彼此相邻、两个文件节点位于顶部、两个块节点位于其正下方。

  例如、在第一个构建块中、文件节点h01和h02都直接连接到块节点A01和A02。主机名依次为h01、h02、A01和A02。

- 组件会根据主机名按顺序进行安装、因此、编号较低的主机名位于机架顶部、编号较高的主机名位于底部。

  其目的是最大限度地缩短连接到机架交换机顶部的缆线长度、并定义标准部署实践以简化故障排除。对于因机架稳定性问题而不允许这样做的数据中心、当然也允许使用相反方法、即从下往上填充机架。

InfiniBand存储网络配置

每个文件节点上一半的InfiniBand端口用于直接连接到块节点。另一半连接到InfiniBand交换机、并用于BeeGFS客户端-服务器连接。在确定用于BeeGFS客户端和服务器的IPoIB子网的大小时、您必须考虑计算/GPU集群和BeeGFS文件系统的预期增长。如果您必须偏离建议的IP范围、请注意、一个构建块中的每个直接连接都有一个唯一的子网、并且与用于客户端-服务器连接的子网没有重叠。

直接连接

每个构建块中的文件和块节点始终使用下表中的IP进行直接连接。

> 此寻址方案遵循以下规则：第三个八位字节始终为奇数或偶数、具体取决于文件节点是奇数还是偶数。

文件节点	IB端口	IP 地址	块节点	IB端口	物理IP	虚拟IP
奇数(H1)	i1a	192.168.1.10	奇数(C1)	2a.	192.168.1.100	192.168.1.101
奇数(H1)	I2a.	192.168.3.10	奇数(C1)	2a.	192.168.3.100	192.168.3.101
奇数(H1)	i3a	192.168.5.10	偶数(C2)	2a.	192.168.5.100	192.168.5.101
奇数(H1)	i4a	192.168.7.10	偶数(C2)	2a.	192.168.7.100	192.168.7.101
偶数(H2)	i1a	192.168.2.10	奇数(C1)	2亿	192.168.2.100	192.168.2.101
偶数(H2)	I2a.	192.168.4.10	奇数(C1)	2亿	192.168.4.100	192.168.4.101
偶数(H2)	i3a	192.168.6.10	偶数(C2)	2亿	192.168.6.100	192.168.6.101
偶数(H2)	i4a	192.168.8.10	偶数(C2)	2亿	192.168.8.100	192.168.8.101

BeeGFS客户端-服务器IPoIB寻址方案(两个子网)

要使BeeGFS客户端能够使用两个InfiniBand端口、需要两个IPoIB子网、其中一半的BeeGFS服务器服务在每个子网上配置了首选IP、以确保客户端使用两个InfiniBand端口最大限度地提高文件系统的冗余和可能的吞吐量。

每个文件节点都运行多个BeeGFS服务器服务(管理、元数据或存储)。为了使每个服务能够独立故障转移到另一个文件节点、每个服务都配置了唯一的IP地址、这些地址可以在两个节点之间浮动(有时称为逻辑接口或LIF)。

此部署虽然不是必需的、但假定这些连接正在使用以下IPoIB子网范围、并定义了一个标准寻址方案、该方案将

应用以下规则：

- 根据文件节点InfiniBand端口是奇数还是奇数、第二个八位字节始终是奇数甚至偶数。
- BeeGFS集群IP始终为`xxx。127.100.yyy`或`xxx.128.100.yyy`。

> 除了用于带内操作系统管理的接口之外、Corosync还可以使用其他接口进行集群检测和同步。这样可以确保丢失一个接口不会导致整个集群关闭。

- BeeGFS管理服务始终为`xxx.yyy.101.0`或`xxx.yyy.102.0`。
- BeeGFS元数据服务始终位于`xxx.yyy.101.zzz`或`xxx.yyy.102.zzz`。
- BeeGFS存储服务始终位于`xxx.yyy.103.zzz`或`xxx.yyy.103.zzz`。
- 范围`100.xxx.1.1`到`100.xxx.99.255`的地址是为客户端预留的。

子网**A：100.127.0.0/16**

下表提供了子网A的范围：100.127.0.0/16。

目的	**InfiniBand**端口	**IP**地址或范围
BeeGFS集群IP	i1b	100.127.100.1 - 100.127.100.255
BeeGFS管理	i1b	100.127.101.0
BeeGFS元数据	i1b或i3b	100.127.101.1 - 100.127.101.255
BeeGFS存储	i1b或i3b	100.127.103.1 - 100.127.103.255
BeeGFS客户端	(因客户端而异)	100.127.1.1 - 100.127.99.255

子网**B：100.128.0.0/16**

下表提供了子网B的范围：100.128.0.0/16。

目的	**InfiniBand**端口	**IP**地址或范围
BeeGFS集群IP	i4b.	100.128.100.1 - 100.128.100.255
BeeGFS管理	i2b.	100.128.102.0
BeeGFS元数据	i2b或i4b	100.128.102.1 - 100.128.102.255
BeeGFS存储	i2b或i4b	100.128.104.1 - 100.128.104.255
BeeGFS客户端	(因客户端而异)	100.128.1.1 - 100.128.99.255

> 并非上述范围内的所有IP都用于此经过NetApp验证的架构。它们展示了如何预先分配IP地址、以便使用一致的IP寻址方案轻松扩展文件系统。在此方案中、BeeGFS文件节点和服务ID与已知IP范围中的第四个八位字节相对应。如果需要、文件系统可以扩展到255个节点或服务以上。

部署硬件

每个组件都包含两个经过验证的x86文件节点、这些文件节点使用HDR (200 GB) InfiniBand缆线直接连接到两个块节点。

> 由于每个组件都包含两个BeeGFS文件节点、因此要在故障转移集群中建立仲裁、至少需要两个组件。虽然可以配置双节点集群、但此配置存在一些限制、可能会在某些情况下阻止成功进行故障转移。如果需要双节点集群、也可以将第三个设备作为Tiebreaker、但此部署操作步骤 不会涉及这一点。

除非另有说明、否则集群中的每个组件的以下步骤都是相同的、无论该组件是用于运行BeeGFS元数据和存储服务还是仅用于运行存储服务。

步骤

1. 在InfiniBand模式下为每个BeeGFS文件节点配置四个PCIe 4.0 ConnectX-6双端口主机通道适配器(Host Channel Adapter、HCA)、并将其安装在PCIe插槽2、3、5和6中。
2. 为每个BeeGFS块节点配置一个双端口200 GB主机接口卡(HIC)、并在其两个存储控制器中的每个控制器中安装HIC。

   将组件装入机架、使两个BeeGFS文件节点位于BeeGFS块节点上方。下图显示了BeeGFS组件的正确硬件配置(后视图)。

   []

   > 用于生产用例的电源配置通常应使用冗余PSU。

3. 如果需要、请在每个BeeGFS块节点中安装驱动器。
   a. 如果要使用此组件运行BeeGFS元数据和存储服务、而将较小的驱动器用于元数据卷、请验证这些驱动器是否已填充到最外的驱动器插槽中、如下图所示。
   b. 对于所有组件配置、如果驱动器机箱未完全填充、请确保在插槽0–11和12–23中填充相同数量的驱动器、以获得最佳性能。

      []

4. 要为文件和块节点布线、请使用1米InfiniBand HDR 200 GB直连铜缆、以便与下图所示的拓扑结构匹配。

   []

   > 多个组件中的节点不会直接连接。每个组件都应视为一个独立单元、并且各个组件之间的所有通信都通过网络交换机进行。

5. 使用2米(或适当长度)的InfiniBand HDR 200 GB直连铜缆将每个文件节点上的其余InfiniBand端口连接到要用于存储网络的InfiniBand交换机。

   如果正在使用冗余InfiniBand交换机、请使用缆线将下图中以淡绿色突出显示的端口连接到不同的交换机。

   []

6. 根据需要、按照相同的布线准则组装其他组件。

   > 可部署在一个机架中的组件总数取决于每个站点的可用电源和散热。

部署软件

设置文件节点和块节点

虽然大多数软件配置任务都是使用NetApp提供的Ansible集合自动执行的、但您必须在每个服务器的基板管理控制器(Baseboard Management Controller、BMC)上配置网络、并在每个控制器上配置管理端口。

设置文件节点

1. 在每个服务器的基板管理控制器(BMC)上配置网络连接。

   要了解如何为经过验证的联想SR665文件节点配置网络、请参见 "联想ThinkSystem文档"。

   > 基板管理控制器(Baseboard Management Controller、BMC)有时也称为服务处理器、它是内置于各种服务器平台中的带外管理功能的通用名称、即使操作系统未安装或无法访问、也可以提供远程访问。供应商通常会使用自己的品牌推广此功能。例如、在联想SR665上、BMC称为_Lenovo Xicity1控制器(XCC)_。

2. 配置系统设置以实现最高性能。

   您可以使用UEFI设置(以前称为BIOS)或许多BMC提供的Redfish API配置系统设置。系统设置因用作文件节点的服务器型号而异。

   要了解如何为经过验证的联想SR665文件节点配置系统设置、请参见 "调整系统性能设置"。

3. 安装Red Hat 8.4并配置用于管理操作系统的主机名和网络端口、包括来自Ansible控制节点的SSH连接。

   此时、请勿在任何InfiniBand端口上配置IP。

   > 尽管严格要求不高、但后续章节假定主机名按顺序编号(例如H1-HN)、并提及应在奇数主机上完成的任务、而不是偶数主机上完成的任务。

4. 使用RedHat订阅管理器注册并订阅系统、以便可以从Red Hat官方存储库安装所需的软件包、并将更新限制为受支持的Red Hat版本：`ssubscription-manager release -set=8.4`。有关说明，请参见 "如何注册和订阅RHEL系统" 和 "如何限制更新"。

5. 启用包含高可用性所需软件包的Red Hat存储库。

   ```
   subscription-manager repo-override --repo=rhel-8-for-x86_64
   -highavailability-rpms --add=enabled:1
   ```

6. 将所有ConnectX-6 HCA固件更新为中建议的版本 "技术要求"。

   要完成此更新、可以下载并运行捆绑了建议固件的lxup工具版本。您可以从下载此工具 "lxup—更新和查询实用程序" （"用户指南"）。

设置块节点

通过在每个控制器上配置管理端口来设置EF600块节点。

1. 在每个EF600控制器上配置管理端口。

   有关配置端口的说明、请转至 "E系列文档中心"。

2. (可选)设置每个系统的存储阵列名称。

   通过设置名称、可以更轻松地在后续章节中引用每个系统。有关设置阵列名称的说明、请转到 "E系列文档中心"。

> 尽管严格要求不严格、但后续主题假定存储阵列名称按顺序编号(例如C1 - CN)、并提及在奇数系统甚至编号系统上应完成的步骤。

设置**Ansible**控制节点

要设置Ansible控制节点、您必须确定能够通过网络访问可用于配置解决方案 的所有文件和块节点的管理端口的虚拟机或物理机。

以下步骤已在CentOS 8.4上进行了测试。有关首选Linux分发版的特定步骤、请参见 "Ansible文档"。

1. 安装Python 3.9并确保安装了正确版本的`pip`。

```
sudo dnf install python3.9 -y
sudo dnf install python39-pip
sudo dnf install sshpass
```

2. 创建符号链接、确保每当调用`python3`或`python`时使用Python 3.9二进制文件。

```
sudo ln -sf /usr/bin/python3.9 /usr/bin/python3
sudo ln -sf /usr/bin/python3 /usr/bin/python
```

3. 安装NetApp BeeGFS集合所需的Python软件包。

```
python3 -m pip install ansible cryptography netaddr
```

> 要确保您安装的是受支持的Ansible版本以及所有必需的Python软件包、请参阅BeeGFS集合的自述文件。中还说明了支持的版本 "技术要求"。

4. 验证是否已安装正确版本的Ansible和Python。

```
ansible --version
ansible [core 2.11.6]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-
packages/ansible
  ansible collection location =
/root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.9.2 (default, Mar 10 2021, 17:29:56) [GCC 8.4.1
20200928 (Red Hat 8.4.1-1)]
  jinja version = 3.0.2
  libyaml = True
```

5. 将用于描述BeeGFS部署的Ansible清单存储在Git或BitBucket等源控制系统中、然后安装Git以与这些系统进行交互。

```
sudo dnf install git -y
```

6. 设置无密码SSH。这是允许Ansible从Ansible控制节点访问远程BeeGFS文件节点的最简单方法。
   a. 如果需要、在Ansible控制节点上、使用`ssh-keygen`生成一对公有 密钥
   b. 使用`ssh-copy-id <ip_or_hostname>`为每个文件节点设置无密码SSH

      Do * not* set up passworwless SSH to the block nodes.这既不受支持、也不是必需的。

7. 使用Ansible GALAXY安装中列出的BeeGFS集合版本 "技术要求"。

   此安装还包括其他Ansible依赖关系、例如NetApp SANtricity 软件和主机集合。

```
ansible-galaxy collection install netapp_eseries.beegfs:==3.0.1
```

创建**Ansible**清单

要定义文件和块节点的配置、请创建一个Ansible清单、用于表示要部署的BeeGFS文件系统。清单包括描述所需BeeGFS文件系统的主机、组和变量。

第**1**步：定义所有组件的配置

定义适用场景 所有组件的配置、无论您可以单独应用于这些组件的配置文件如何。

开始之前

- 使用BitBucket或Git等源控制系统存储包含Ansible清单和攻略手册文件的目录的内容。

- 创建一个`.gitignore`文件、用于指定Git应忽略的文件。这有助于避免在Git中存储大型文件。

步骤

1. 在Ansible控制节点上、确定要用于存储Ansible清单和攻略手册文件的目录。

   除非另有说明、否则在此步骤中创建的所有文件和目录以及后续步骤均会相对于此目录创建。

2. 创建以下子目录：

   `host_vars`

   `group_vars`

   软件包

第2步：为单个文件和块节点定义配置

定义适用场景 单个文件节点和单个组件节点的配置。

1. 在`host_vars/下、为名为<HOSTNAME>.yml`的每个BeeGFS文件节点创建一个文件、其中包含以下内容、请特别注意有关BeeGFS集群IP和以奇数与偶数结尾的主机名要填充的内容的注释。

   最初、文件节点接口名称与此处列出的名称匹配(例如ib0或ibs1f0)。这些自定义名称在中进行配置 [第4步：定义应应用于所有文件节点的配置]。

```
ansible_host: “<MANAGEMENT_IP>”
eseries_ipoib_interfaces:  # Used to configure BeeGFS cluster IP
addresses.
  - name: i1b
    address: 100.127.100. <NUMBER_FROM_HOSTNAME>/16
  - name: i4b
    address: 100.128.100. <NUMBER_FROM_HOSTNAME>/16
beegfs_ha_cluster_node_ips:
  - <MANAGEMENT_IP>
  - <i1b_BEEGFS_CLUSTER_IP>
  - <i4b_BEEGFS_CLUSTER_IP>
# NVMe over InfiniBand storage communication protocol information
# For odd numbered file nodes (i.e., h01, h03, ..):
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.1.10/24
    configure: true
  - name: i2a
    address: 192.168.3.10/24
    configure: true
  - name: i3a
    address: 192.168.5.10/24
    configure: true
  - name: i4a
    address: 192.168.7.10/24
    configure: true
# For even numbered file nodes (i.e., h02, h04, ..):
# NVMe over InfiniBand storage communication protocol information
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.2.10/24
    configure: true
  - name: i2a
    address: 192.168.4.10/24
    configure: true
  - name: i3a
    address: 192.168.6.10/24
    configure: true
  - name: i4a
    address: 192.168.8.10/24
    configure: true
```

如果您已部署BeeGFS集群、则必须先停止此集群、然后才能添加或更改静态配置的IP地址、包括用于NVMe/IB的集群IP和IP。这是必需的、以便这些更改能够正确生效、并且不会中断集群操作。

2. 在`host_vars/下、为名为<HOSTNAME>.yml`的每个BeeGFS块节点创建一个文件、并使用以下内容填充该文件。

   请特别注意有关以奇数与偶数结尾的存储阵列名称要填充的内容的注释。

   对于每个块节点、创建一个文件并为两个控制器(通常为A)之一指定`<management_IP>`。

```
eseries_system_name: <STORAGE_ARRAY_NAME>
eseries_system_api_url: https://<MANAGEMENT_IP>:8443/devmgr/v2/
eseries_initiator_protocol: nvme_ib
# For odd numbered block nodes (i.e., a01, a03, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101
    - 192.168.2.101
    - 192.168.1.100
    - 192.168.2.100
  controller_b:
    - 192.168.3.101
    - 192.168.4.101
    - 192.168.3.100
    - 192.168.4.100
# For even numbered block nodes (i.e., a02, a04, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.5.101
    - 192.168.6.101
    - 192.168.5.100
    - 192.168.6.100
  controller_b:
    - 192.168.7.101
    - 192.168.8.101
    - 192.168.7.100
    - 192.168.8.100
```

第3步：定义应应用于所有文件和块节点的配置

您可以使用与组对应的文件名在`group_vars`下定义一组主机的通用配置。这样可以防止在多个位置重复使用共享配置。

关于此任务

主机可以位于多个组中、运行时、Ansible会根据特定主机的可变优先级规则选择哪些变量适用于该主机。(有关这些规则的详细信息、请参见的Ansible文档 "使用变量")

主机到组分配在实际可Ansible清单文件中进行定义、该文件在该操作步骤 接近末尾时创建。

步骤

在Ansible中、可以在名为`All`的组中定义要应用于所有主机的任何配置。使用以下内容创建文件`group_vars/all.yml`：

```
ansible_python_interpreter: /usr/bin/python3
beegfs_ha_ntp_server_pools:  # Modify the NTP server addressess if
desired.
  - "pool 0.pool.ntp.org iburst maxsources 3"
  - "pool 1.pool.ntp.org iburst maxsources 3"
```

第4步：定义应应用于所有文件节点的配置

文件节点的共享配置在名为`ha_cluster`的组中定义。本节中的步骤将构建应包含在`group_vars/ha_cluster.yml`文件中的配置。

步骤

1. 在文件顶部、定义默认值、包括用作文件节点上的`sUdo`用户的密码。

```
### ha_cluster Ansible group inventory file.
# Place all default/common variables for BeeGFS HA cluster resources
below.
### Cluster node defaults
ansible_ssh_user: root
ansible_become_password: <PASSWORD>
eseries_ipoib_default_hook_templates:
  - 99-multihoming.j2 # This is required when configuring additional
static IPs (for example cluster IPs) when multiple IB ports are in the
same IPoIB subnet.
# If the following options are specified, then Ansible will
automatically reboot nodes when necessary for changes to take effect:
eseries_common_allow_host_reboot: true
eseries_common_reboot_test_command: "systemctl --state=active,exited |
grep eseries_nvme_ib.service"
```

> 尤其是在生产环境中、不要以纯文本格式存储密码。请改用Ansible Vault (请参见 "使用Ansible Vault加密内容")或运行攻略手册时的`-ask-pass-pass`选项。如果`Ansible_ssh_user`已是`root`、则可以选择省略`Ansible变为密码`。

2. (可选)配置高可用性(HA)集群的名称、并为集群内通信指定用户。

   如果要修改专用IP寻址方案、则还必须更新默认值`beegfs_ha_mgmtd_float_IP`。这必须与您稍后为BeeGFS管理资源组配置的内容匹配。

   使用`beegfs_ha_alert_email_list`指定一个或多个应接收集群事件警报的电子邮件。

```
### Cluster information
beegfs_ha_firewall_configure: True
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
# The following variables should be adjusted depending on the desired
configuration:
beegfs_ha_cluster_name: hacluster                  # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster              # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword             # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
beegfs_ha_mgmtd_floating_ip: 100.127.101.0         # BeeGFS management
service IP address.
# Email Alerts Configuration
beegfs_ha_enable_alerts: True
beegfs_ha_alert_email_list: ["email@example.com"]  # E-mail recipient
list for notifications when BeeGFS HA resources change or fail.  Often a
distribution list for the team responsible for managing the cluster.
beegfs_ha_alert_conf_ha_group_options:
      mydomain: “example.com”
# The mydomain parameter specifies the local internet domain name. This
is optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com).
# Adjusting the following parameters is optional:
beegfs_ha_alert_timestamp_format: "%Y-%m-%d %H:%M:%S.%N" #%H:%M:%S.%N
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

> 虽然`冗余、但当您将BeeGFS文件系统扩展到单个HA集群之外时、`beegfs_ha_mgmtd_floating_ip非常重要。部署后续HA集群时无需额外的BeeGFS管理服务、并指向第一个集群提供的管理服务。

3. 配置隔离代理。(有关详细信息、请参见 "在Red Hat High Availability集群中配置隔离") 以下输出显示了配置通用隔离代理的示例。选择以下选项之一。

   在此步骤中、请注意：

   - 默认情况下、隔离处于启用状态、但您需要配置隔离_agent_。
   - 在`PCMK_HOST_MAP`或`PCMK_HOST_LIST`中指定的`<HOSTNAME>`必须与Ansible清单中的主机

名相对应。

- 不支持在不使用隔离的情况下运行BeeGFS集群、尤其是在生产环境中。这在很大程度上是为了确保当BeeGFS服务(包括块设备等任何资源依赖关系)因问题描述 而发生故障转移时、不会存在多个节点并发访问导致文件系统损坏或其他不希望或意外行为的风险。如果必须禁用隔离、请参阅BeeGFS HA角色的入门指南中的一般说明、并在`ha_cluster_crm_config_options["stonith-enabled"]`中将`beegfs_ha_cluster.yml`设置为false。
- 可以使用多个节点级别的隔离设备、BeeGFS HA角色可以配置Red Hat HA软件包存储库中可用的任何隔离代理。如果可能、请使用通过不间断电源(UPS)或机架配电单元(rPDU)工作的隔离代理。 因为在某些故障情形下、某些隔离代理(如基板管理控制器(BMC)或服务器中内置的其他无人值守设备)可能无法响应隔离请求。

```
### Fencing configuration:
# OPTION 1: To enable fencing using APC Power Distribution Units
(PDUs):
beegfs_ha_fencing_agents:
 fence_apc:
   - ipaddr: <PDU_IP_ADDRESS>
     login: <PDU_USERNAME>
     passwd: <PDU_PASSWORD>
     pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>"
# OPTION 2: To enable fencing using the Redfish APIs provided by the
Lenovo XCC (and other BMCs):
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify “1”.
beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
# For details on configuring other fencing agents see
https://access.redhat.com/documentation/en-
us/red_hat_enterprise_linux/8/html/configuring_and_managing_high_avai
lability_clusters/assembly_configuring-fencing-configuring-and-
managing-high-availability-clusters.
```

4. 在Linux操作系统中启用建议的性能调整。

   虽然许多用户发现性能参数的默认设置通常运行良好、但您也可以选择更改特定工作负载的默认设置。因此、这些建议包含在BeeGFS角色中、但默认情况下不会启用、以确保用户了解应用于其文件系统的调整。

要启用性能调整、请指定：

```
### Performance Configuration:
beegfs_ha_enable_performance_tuning: True
```

5. (可选)您可以根据需要调整Linux操作系统中的性能调整参数。

   有关可调整的可用调整参数的完整列表、请参见中BeeGFS HA角色的"性能调整默认值"部分 "E系列BeeGFS GitHub站点"。可以覆盖此文件或单个节点的`host_vars`文件中集群中所有节点的默认值。

6. 要在块节点和文件节点之间实现完整的200 GB/HDR连接、请使用Mellanox Open Fabrics Enterprise Distribution (MLNx_OFED)中的Open Subnet Manager (OpenSM)软件包。(收件箱`opensm`软件包不支持所需的虚拟化功能。) 尽管支持使用Ansible进行部署、但您必须先将所需的软件包下载到用于运行BeeGFS角色的Ansible控制节点。

   a. 使用`cURL`或您所需的工具、将技术要求部分所列OpenSM版本的软件包从Mellanox的网站下载到`packages/`目录。例如：

   ```
   curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm
   https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
   1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm

   curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm
   https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
   1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm
   ```

   b. 在`group_vars/ha_cluster.yml`中填充以下参数(根据需要调整软件包)：

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
eseries_ib_opensm_options:
  virt_enabled: "2"
```

7. 配置`udev`规则、以确保逻辑InfiniBand端口标识符与底层PCIe设备的映射一致。

   `udev`规则对于用作BeeGFS文件节点的每个服务器平台的PCIe拓扑来说必须是唯一的。

   对于已验证的文件节点、请使用以下值：

```
### Ensure Consistent Logical IB Port Numbering
# OPTION 1: Lenovo SR665 PCIe address-to-logical IB port mapping:
eseries_ipoib_udev_rules:
  "0000:41:00.0": i1a
  "0000:41:00.1": i1b
  "0000:01:00.0": i2a
  "0000:01:00.1": i2b
  "0000:a1:00.0": i3a
  "0000:a1:00.1": i3b
  "0000:81:00.0": i4a
  "0000:81:00.1": i4b

# Note: At this time no other x86 servers have been qualified.
Configuration for future qualified file nodes will be added here.
```

8. (可选)更新元数据目标选择算法。

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  tuneTargetChooser: randomrobin
```

> 在验证测试中、通常会使用`Randomrobin`来确保测试文件在性能基准测试期间均匀分布在所有BeeGFS存储目标上(有关基准测试的详细信息、请参见BeeGFS站点 "对BeeGFS系统进行基准测试"）。在实际使用情况下、这可能会导致编号较低的目标达到发生原因 、从而比编号较高的目标更快地达到填充速度。已显示省略`randomrobin`以及仅使用默认值`randomized`值可在仍利用所有可用目标的情况下提供良好的性能。

第5步：定义通用块节点的配置

块节点的共享配置在名为`Eseries_storage_systems`的组中定义。本节中的步骤构建了应包含在`group_vars/eseries_storage_systems.yml`文件中的配置。

步骤

1. 将Ansible连接设置为local、提供系统密码、并指定是否应验证SSL证书。(通常、Ansible使用SSH连接到受管主机、但对于用作块节点的NetApp E系列存储系统、模块使用REST API进行通信。) 在文件顶部、添加以下内容：

```
### eseries_storage_systems Ansible group inventory file.
# Place all default/common variables for NetApp E-Series Storage Systems
here:
ansible_connection: local
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> 不建议以纯文本格式列出任何密码。使用`-ext-vars`运行Ansible时、请使用Ansible vault或提供`E系列系统密码`。

2. 为确保获得最佳性能、请在中安装为块节点列出的版本 "技术要求"。

   从下载相应的文件 "NetApp 支持站点"。您可以手动升级它们、也可以将其包含在Ansible控制节点的`packages/`目录中、然后在`Esery_storage_systems.yml`中填充以下参数以使用Ansible进行升级：

```
# Firmware, NVSRAM, and Drive Firmware (modify the filenames as needed):
eseries_firmware_firmware: "packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/N6000-872834-D06.dlp"
```

3. 从下载并安装可用于块节点中安装的驱动器的最新驱动器固件 "NetApp 支持站点"。您可以手动升级它们、也可以将其包含在Ansible控制节点的`packages/`目录中、然后在`Esery_storage_systems.yml`中填充以下参数以使用Ansible进行升级：

```
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
eseries_drive_firmware_upgrade_drives_online: true
```

> 将`esery_drive_firmware_upgrade_drives_online`设置为`false`可以加快升级速度、但在部署BeeGFS之前不应执行此操作。这是因为该设置要求在升级之前停止驱动器的所有I/O、以避免应用程序错误。尽管在配置卷之前执行联机驱动器固件升级仍很快、但我们建议您始终将此值设置为`true`以避免稍后出现问题。

4. 要优化性能、请对全局配置进行以下更改：

```
# Global Configuration Defaults
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required.
```

5. 要确保最佳卷配置和行为、请指定以下参数：

```
# Storage Provisioning Defaults
eseries_volume_size_unit: pct
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99:6,
99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

> 为`E系列_storage_pool_usable_drives`指定的值特定于NetApp EF600块节点、并控制驱动器分配给新卷组的顺序。此顺序可确保每个组的I/O在后端驱动器通道之间均匀分布。

为**BeeGFS**组件定义**Ansible**清单

定义了常规Ansible清单结构后、为BeeGFS文件系统中的每个组件定义配置。

这些部署说明演示了如何部署文件系统、该文件系统由包括管理、元数据和存储服务在内的基础组件、包含元数据和存储服务的第二个组件以及仅存储的第三个组件组成。

这些步骤旨在显示一系列典型配置文件、您可以使用这些配置文件来配置NetApp BeeGFS组件、以满足整个BeeGFS文件系统的要求。

> 在本节及后续章节中、根据需要进行调整、以构建表示要部署的BeeGFS文件系统的清单。特别是、请使用表示每个块或文件节点的Ansible主机名以及存储网络所需的IP寻址方案、以确保它可以扩展到BeeGFS文件节点和客户端的数量。

第**1**步：创建**Ansible**清单文件

步骤

1. 创建新的`inventory.yml`文件、然后插入以下参数、根据需要替换`Esery_storage_systems`下的主机、以表示部署中的块节点。这些名称应与`host_vars/<filename>.yml`使用的名称相对应。

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

在后续各节中、您将在`ha_cluster`下创建其他Ansible组、这些组表示您要在集群中运行的BeeGFS服务。

第2步：为管理、元数据和存储组件配置清单

集群或基础构建块中的第一个组件必须包括BeeGFS管理服务以及元数据和存储服务：

步骤

1. 在`inventory.yml`中、在`ha_cluster下填充以下参数：子项`：

```
      # ictad22h01/ictad22h02 HA Pair (mgmt/meta/storage building
block):
        mgmt:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_01:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_01:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_02:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_02:
          hosts:
```

```
            ictad22h01:
            ictad22h02:
        meta_03:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_03:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_04:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_04:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_05:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_05:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_06:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_06:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_07:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_07:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

```
        stor_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

2. 创建文件`group_vars/mgmt.yml`并包含以下内容：

```
# mgmt - BeeGFS HA Management Resource Group
# OPTIONAL: Override default BeeGFS management configuration:
# beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
#  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
floating_ips:
  - i1b: 100.127.101.0/16
  - i2b: 100.128.102.0/16
beegfs_service: management
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 1
            owning_controller: A
```

3. 在`group_vars/`下、使用以下模板为资源组`mETA_01`到`mETA_08`创建文件、然后参考下表填写每个服务的占位值：

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET> # Example: i1b:192.168.120.1/16
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 21.25 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

卷大小以整个存储池(也称为卷组)的百分比形式指定。NetApp强烈建议您在每个池中保留一些可用容量、以便为SSD过度配置留出空间(有关详细信息、请参见 "NetApp EF600阵列简介"）。存储池`beegfs_m1_m2_m3_m3_m6`也会将池容量的1%分配给管理服务。因此、对于存储池中的元数据卷`beegfs_m1_m2_m5_m6`、如果使用1.92 TB或3.84 TB驱动器、请将此值设置为`21.25`；对于7.65 TB驱动器、请将此值设置为`22.25`；对于15.3 TB驱动器、请将此值设置为`23.75`。

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_01.yml	8015	i1b ：100.127.1.1.1/16 i2b ：100.128.102.1/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	答
meta_02.yml	8025	i2b ：100.128.102.2/16 i1b ：100.127.101.2/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	B
meta_03.yml	8035	i3b ：100.127.101.3/16 i4b ：100.128.102.3/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	答

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_04.yml	8045	i4b：100.128.102.4/16 i3b：100.127.101.4/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	B
meta_05.yml	8055	i1b：100.127.101.5/16 i2b：100.128.102.5/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	答
meta_06.yml	8065	i2b：100.128.102.6/16 i1b：100.127.101.6/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	B
meta_07.yml	8075	i3b：100.127.101.7/16 i4b：100.128.102.7/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	答
meta_08.yml	8085	i4b：100.128.102.8/16 i3b：100.127.101.8/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	B

4. 在`group_vars/`下、使用以下模板为资源组`stor_01`到`stor_08`创建文件、然后参考以下示例填写每个服务的占位值：

```
# stor_0X - BeeGFS HA Storage Resource
Groupbeegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!             owning_controller:
<OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"。

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_01.yml	8013	i1b：100.127.103.1/16 i2b：100.128.104.1/16	0	ictad22a01	beegfs_s1_s2	答
STOR_02.yml	8023	i2b：100.128.104.2/16 i1b：100.127.103.2/16	0	ictad22a01	beegfs_s1_s2	B
STOR_03.yml	8033	i3b：100.127.103.3/16 i4b：100.128.104.3/16	1.	ictad22a02	beegfs_s3_s4.	答
STOR_04.yml	8043	i4b：100.128.104.4/16 i3b：100.127.103.4/16	1.	ictad22a02	beegfs_s3_s4.	B

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_05.yml	8053	i1b：100.127.103.5/16 i2b：100.128.104.5/16	0	ictad22a01	beegfs_s5_s6	答
STOR_06.yml	8063	i2b：100.128.104.6/16 i1b：100.127.103.6/16	0	ictad22a01	beegfs_s5_s6	B
STOR_07.yml	8073.	i3b：100.127.103.7/16 i4b：100.128.104.7/16	1.	ictad22a02	beegfs_s7_s8.	答
STOR_08.yml	8083.	i4b：100.128.104.8/16 i3b：100.127.103.8/16	1.	ictad22a02	beegfs_s7_s8.	B

第3步：为元数据+存储构建块配置清单

以下步骤介绍如何为BeeGFS元数据+存储构建块设置Ansible清单。

步骤

1. 在`inventory.yml`中、在现有配置下填充以下参数：

```
        meta_09:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_09:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_10:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_10:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_11:
```

```
          hosts:
            ictad22h03:
            ictad22h04:
        stor_11:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_12:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_12:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_13:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_13:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_14:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_14:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_15:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_15:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_16:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_16:
          hosts:
            ictad22h04:
```

```
            ictad22h03:
```

2. 在`group_vars/`下、使用以下模板为资源组`mETA_09`到`mETA_16`创建文件、然后参考以下示例填写每个服务的占位值：

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.5 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"。

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_09.yml	8015	i1b：100.127.101.9/16 i2b：100.128.102.9/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	答
meta_10.yml	8025	i2b：100.128.102.10/16 i1b：100.127.101.10/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	B
meta_11.yml	8035	i3b：100.127.2.1.1/16 i4b：100.128.102.11/16	1.	ictad22a04	Beegfs_M11_M12_M15_16	答

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_12.yml	8045	i4b：100.128.102.12或16 i3b：100.127.1.1.12或16	1.	ictad22a04	Beegfs_M11_M12_M15_16	B
meta_13.yml	8055	i1b：100.127.102.13/16 i2b：100.128.102.13/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	答
meta_14.yml	8065	i2b：100.128.102.14/16 i1b：100.127.101.14/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	B
meta_15.yml	8075	i3b：100.127.101.15或16 i4b：100.128.102.15或16	1.	ictad22a04	Beegfs_M11_M12_M15_16	答
meta_16.yml	8085	i4b：100.128.102.16/16 i3b：100.127.101.16/16	1.	ictad22a04	Beegfs_M11_M12_M15_16	B

3. 在`group_vars/下、`使用以下模板为资源组`stor_09`到`stor_16`创建文件、然后参考以下示例填写每个服务的占位值：

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"...

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_09.yml	8013	i1b：100.127.103.9/16 i2b：100.128.104.9/16	0	ictad22a03	beegfs_s9_s10	答
STOR_10.yml	8023	i2b：100.128.104.10/16 i1b：100.127.103.10/16	0	ictad22a03	beegfs_s9_s10	B
STOR_11.yml	8033	i3b：100.127.103.11/16 i4b：100.128.104.11/16	1.	ictad22a04	beegfs_s11_s12	答
STOR_12.yml	8043	i4b：100.128.104.12/16 i3b：100.127.103.12/16	1.	ictad22a04	beegfs_s11_s12	B

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
STOR_13.yml	8053	i1b ： 100.127.103.13/16 i2b ： 100.128.104.13/ 16	0	ictad22a03	beegfs_s13_s14	答
STOR_14.yml	8063	i2b ： 100.128.104.14/16 i1b ： 100.127.103.14/16	0	ictad22a03	beegfs_s13_s14	B
STOR_15.yml	8073.	i3b ： 100.127.103.15/16 i4b ： 100.128.104.15/16	1.	ictad22a04	beegfs_s15_s16	答
STOR_16.yml	8083.	i4b ： 100.128.104.16/16 i3b ： 100.127.103.16/16	1.	ictad22a04	beegfs_s15_s16	B

第**4**步：为纯存储组件配置清单

以下步骤介绍如何为BeeGFS纯存储组件设置Ansible清单。设置元数据+存储与仅存储组件的配置之间的主要区别是、省略了所有元数据资源组、并将每个存储池的`Criteria_drive_count`从10更改为12。

步骤

1. 在`inventory.yml`中、在现有配置下填充以下参数：

```
# ictad22h05/ictad22h06 HA Pair (storage only building block):
  stor_17:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_18:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_19:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_20:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_21:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_22:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_23:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_24:
    hosts:
      ictad22h06:
      ictad22h05:
```

2. 在`group_vars/`下、使用以下模板为资源组`stor_17`到`stor_24`创建文件、然后参考以下示例填写每个服务的占位值：

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 12
        common_volume_configuration:
          segment_size_kb: 512
        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50
            owning_controller: <OWNING CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"。

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
STOR_17.yml	8013	i1b：100.127.103.17/16 i2b：100.128.104.17/16	0	ictad22a05	beegfs_s17_s18	答
STOR_18.yml	8023	i2b：100.128.104.18/16 i1b：100.127.103.18/16	0	ictad22a05	beegfs_s17_s18	B
STOR_19.yml	8033	i3b：100.127.103.19/16 i4b：100.128.104.19/16	1.	ictad22a06	beegfs_s19_s20	答

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_20.yml	8043	i4b：100.128.104.20/16 i3b：100.127.103.20/16	1.	ictad22a06	beegfs_s19_s20	B
STOR_21.yml	8053	i1b：100.127.103.21/16 i2b：100.128.104.21/16	0	ictad22a05	Beegfs_S21_S22	答
STOR_22.yml	8063	i2b：100.128.104.22/16 i1b：100.127.103.22/16	0	ictad22a05	Beegfs_S21_S22	B
STOR_23.yml	8073.	i3b：100.127.103.23/16 i4b：100.128.104.23/16	1.	ictad22a06	beegfs_s23_s24	答
STOR_24.yml	8083.	i4b：100.128.104.24/16 i3b：100.127.103.24/16	1.	ictad22a06	beegfs_s23_s24	B

部署**BeeGFS**

部署和管理配置涉及运行一个或多个攻略手册、其中包含Ansible需要执行的任务、并将整个系统置于所需状态。

虽然所有任务都可以包含在一本攻略手册中、但对于复杂的系统、这很快就会变得难以管理。Ansible可用于创建和分发角色、以便打包可重复使用的攻略手册和相关内容(例如：默认变量、任务和处理程序)。有关详细信息、请参见的Ansible文档 "角色"。

角色通常作为包含相关角色和模块的Ansible集合的一部分进行分布。因此、这些攻略手册主要仅导入分布在各种NetApp E系列Ansible资料集中的多个角色。

> 目前、部署BeeGFS至少需要两个组件(四个文件节点)、除非将单独的仲裁设备配置为Tiebreaker、以缓解在双节点集群中建立仲裁时出现的任何问题。

步骤

1. 创建新的`playbook.yml`文件并包括以下内容：

```
# BeeGFS HA (High Availability) cluster playbook.
- hosts: eseries_storage_systems
```

```
  gather_facts: false
  collections:
    - netapp_eseries.santricity
  tasks:
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version
        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'
        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true
        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
```

```
groups[beegfs_ha_ansible_storage_group]
    - name: Verify any provided tags are supported.
      fail:
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_2
```

> 本攻略手册将运行几个`pre_tasks`、用于验证Python 3是否已安装在文件节点上、并检查所提供的Ansible标记是否受支持。

2. 准备好部署BeeGFS时、请使用`Ansible攻略手册`命令和清单和攻略手册文件。

   此部署将运行所有`pre_tasks`、然后提示用户确认、然后再继续实际的BeeGFS部署。

   运行以下命令、根据需要调整货叉数量(请参见以下注释)：

```
ansible-playbook -i inventory.yml playbook.yml --forks 20
```

> 特别是对于大型部署、建议使用`fork`参数覆盖默认的货叉数(5)、以增加Ansible并行配置的主机数。(有关详细信息、请参见 "Ansible性能调整" 和 "控制攻略手册执行") 最大值设置取决于Ansible控制节点上可用的处理能力。以上示例20是在具有4个CPU (Intel ® Xeon ® Gold 6146 CPU @ 3.20GHz)的虚拟Ansible控制节点上运行的。

根据Ansible控制节点与BeeGFS文件和块节点之间的部署规模和网络性能、部署时间可能会有所不同。

配置**BeeGFS**客户端

您必须在需要访问BeeGFS文件系统的任何主机上安装和配置BeeGFS客户端、例如计算或GPU节点。在此任务中、您可以使用Ansible和BeeGFS收集。

步骤

1. 如果需要、请从Ansible控制节点向要配置为BeeGFS客户端的每个主机设置无密码SSH：

```
ssh-copy-id <user>@<HOSTNAME_OR_IP>
```

2. 在`host_vars/下、为名为<HOSTNAME>.yml`的每个BeeGFS客户端创建一个包含以下内容的文件、并使用适用于您环境的正确信息填写占位符文本：

```
# BeeGFS Client
ansible_host: <MANAGEMENT_IP>
# OPTIONAL: If you want to use the NetApp E-Series Host Collection's
IPoIB role to configure InfiniBand interfaces for clients to connect to
BeeGFS file systems:
eseries_ipoib_interfaces:
  - name: <INTERFACE>
    address: <IP>/<SUBNET_MASK> # Example: 100.127.1. 1/16
  - name: <INTERFACE>0
    address: <IP>/<SUBNET_MASK>
```

> 目前、必须在每个客户端上配置两个InfiniBand接口、两个存储IPoIB子网中每个接口一个。如果对此处列出的每个BeeGFS服务使用示例子网和建议范围、则客户端应配置一个介于`100.127.1范围内的接口。0`到`100.127.99.255`、另一个位于`100.128.1中。0`到`100.128。99.255`。

3. 创建一个新文件`client_inventory.yml`、然后在顶部填充以下参数：

```
# BeeGFS client inventory.
all:
  vars:
    ansible_ssh_user: <USER> # This is the user Ansible should use to
connect to each client.
    ansible_become_password: <PASSWORD> # This is the password Ansible
will use for privilege escalation, and requires the ansible_ssh_user be
root, or have sudo privileges.
The defaults set by the BeeGFS HA role are based on the testing
performed as part of this NetApp Verified Architecture and differ from
the typical BeeGFS client defaults.
```

> 请勿以纯文本格式存储密码。请改用Ansible Vault (请参见的Ansible文档) "使用Ansible Vault加密内容")或在运行攻略手册时使用`-ask-pass-pass`选项。

4. 在`client_inventory.yml`文件中、列出应在`beegfs_clients`组下配置为BeeGFS客户端的所有主机、然后指定构建BeeGFS客户端内核模块所需的任何其他配置。

```
  children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        ictad21h01:
        ictad21h02:
        ictad21h03:
        ictad21h04:
        ictad21h05:
        ictad21h06:
        ictad21h07:
        ictad21h08:
        ictad21h09:
        ictad21h10:
      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        beegfs_client_ofed_enable: True
        beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"
        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        eseries_ib_skip: False # Default value.
        beegfs_client_ofed_enable: False # Default value.
```

使用Mellanox OFED驱动程序时、请确保`beegfs_client_OFED_include_path`指向适用于Linux安装的正确"header include path"。有关详细信息、请参见的BeeGFS文档 "RDMA支持"。

5. 在`client_inventory.yml`文件中、列出要挂载到任何先前定义的`vars`底部的BeeGFS文件系统。

```
        beegfs_client_mounts:
          - sysMgmtdHost: 100.127.101.0 # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default "sync" mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
```

`beegfs_client_config`表示已测试的设置。有关所有选项的全面概述、请参见`netapp_eseries.beegfs`集合的`beegfs_client` role附带的文档。其中包括有关挂载多个BeeGFS文件系统或多次挂载相同BeeGFS文件系统的详细信息。

6. 创建新的`client_playbook.yml`文件、然后填充以下参数：

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

> 如果您已在相应的IPoIB接口上安装了所需的IB/RDMA驱动程序和配置的IP、请省略导入`NetApp_E系列.host`集合和`IPoIB`角色。

7. 要安装和构建客户端并挂载BeeGFS、请运行以下命令：

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

8. 在将BeeGFS文件系统投入生产之前、我们*强烈*建议您登录到任何客户端并运行`beegfs-fsck -checkfs`以确保所有节点均可访问且未报告任何问题。

扩展到五个组件以上

您可以将Pacemaker和Corosync配置为扩展到五个组件(10个文件节点)以上。但是、较大的集群存在一些缺点、最终、Pacemaker和Corosync最多会占用32个节点。

NetApp仅测试了最多10个节点的BeeGFS HA集群；不建议或不支持将单个集群扩展到超过此限制的范围。但是、BeeGFS文件系统仍需要扩展到10个以上的节点、而NetApp已在NetApp解决方案 上的BeeGFS中对此进行了说明。

通过在每个文件系统中部署包含一个子块的多个HA集群、您可以扩展整个BeeGFS文件系统、而不受底层HA集群机制的任何建议限制或硬限制。在此情景中、请执行以下操作：

- 创建一个新的Ansible清单以表示其他HA集群、然后省略配置另一个管理服务。请将每个附加集群`ha_cluster.yml`中的`beegfs_ha_mgmtd_floating_ip`变量指向第一个BeeGFS管理服务的IP。
- 在向同一文件系统添加其他HA集群时、请确保满足以下要求：
  - BeeGFS节点ID是唯一的。
  - 与`group_vars`下的每个服务对应的文件名在所有集群中都是唯一的。
  - BeeGFS客户端和服务器IP地址在所有集群中都是唯一的。
  - 在尝试部署或更新其他集群之前、包含BeeGFS管理服务的第一个HA集群正在运行。
- 在每个HA集群各自的目录树中单独维护清单。

  尝试在一个目录树中混用多个集群的清单文件可能会导致发生原因 对BeeGFS HA角色聚合应用于特定集群的配置的方式产生问题。

> 无需在创建新的HA集群之前将每个HA集群扩展到五个组件。在许多情况下、每个集群使用较少的组件会更易于管理。一种方法是将每个机架中的组件配置为HA集群。

建议的存储池过度配置百分比

如果对第二代构建块采用每个存储池配置的标准四个卷、请参见下表。

下表提供了在`eseries_storage_pool_configuration`中为每个BeeGFS元数据或存储目标用作卷大小的建议百分比：

驱动器大小	**Size**
1.92 TB	18
3.84 TB	21.5.
7.68 TB	22.5
15.3 TB	24

> 上述指南不适用于包含管理服务的存储池、该存储池应将上述大小减少0.25%、以便为管理数据分配1%的存储池。

要了解如何确定这些值、请参见 "TR-4800：附录A：了解SSD持久性和过度配置"。

高容量组件

标准BeeGFS解决方案 部署指南概述了满足高性能工作负载要求的过程和建议。希望满足高容量要求的客户应观察此处所述的不同部署和建议。

[]

控制器

对于高容量组件、EF600控制器应更换为EF300控制器、每个控制器都安装有一个级联HIC、用于SAS扩展。每个块节点将在阵列机箱中为BeeGFS元数据存储填充最少数量的NVMe SSD、并将其连接到为BeeGFS存储卷填充NL-SAS HDD的扩展架。

文件节点到块节点的配置保持不变。

驱动器放置

每个块节点至少需要4个NVMe SSD才能存储BeeGFS元数据。这些驱动器应放置在机箱最外侧的插槽中。

[]

扩展托盘

高容量组件的大小可以为每个存储阵列配备1到7个60个驱动器扩展托盘。

有关为每个扩展托盘布线的说明、 "有关驱动器架的信息、请参见EF300布线"。

使用自定义架构

概述和要求

使用Ansible部署BeeGFS高可用性集群时、请使用任何NetApp E/EF系列存储系统作为BeeGFS块节点、使用x86服务器作为BeeGFS文件节点。

有关本节中使用的术语定义、请参见 "术语和概念" 页面。

简介

同时 "经过NetApp验证的架构" 提供预定义的参考配置和规模估算指导、某些客户和合作伙伴可能更愿意设计更适合特定要求或硬件首选项的自定义架构。在NetApp上选择BeeGFS的主要优势之一是、能够使用Ansible部署BeeGFS共享磁盘HA集群、从而简化集群管理、并通过NetApp编写的HA组件提高可靠性。在NetApp上部署自定义BeeGFS架构仍可使用Ansible来完成、从而在一系列灵活的硬件上保持类似设备的方法。

本节概述了在NetApp硬件上部署BeeGFS文件系统以及使用Ansible配置BeeGFS文件系统所需的常规步骤。有关BeeGFS文件系统设计最佳实践以及优化示例的详细信息、请参见 "经过NetApp验证的架构" 部分。

部署概述

通常、部署BeeGFS文件系统涉及以下步骤：

- 初始设置：
  - 安装硬件/为硬件布线。
  - 设置文件和块节点。
  - 设置Ansible控制节点。
- 将BeeGFS文件系统定义为Ansible清单。
- 对文件和块节点运行Ansible以部署BeeGFS。
  - 也可以设置客户端并挂载BeeGFS。

后续章节将更详细地介绍这些步骤。

Ansible负责处理所有软件配置和配置任务、包括：

- 在块节点上创建/映射卷。
- 格式化/调整文件节点上的卷。
- 在文件节点上安装/配置软件。
- 建立HA集群并配置BeeGFS资源和文件系统服务。

要求

在Ansible中支持BeeGFS的发布时间为 "Ansible Galax河" 作为一组角色和模块、用于自动执行BeeGFS HA集群

的端到端部署和管理。

BeeGFS本身采用<major> <major>。<minor>。<patch> 版本控制方案进行版本控制、该集合会为每个受支持的BeeGFS的BeeGFS <minor> 版本维护角色、例如BeeGFS 7.2或BeeGFS 7.3。发布此集合的更新后、每个角色中的修补程序版本将更新为指向该版本分支的最新可用BeeGFS版本(例如：7.2.8)。此外、该集合的每个版本还会通过特定的Linux分发版和版本进行测试并受支持、当前的Red Hat用于文件节点、RedHat和Ubuntu用于客户端。不支持运行其他分发版、也不建议运行其他版本(尤其是其他主要版本)。

Ansible控制节点

此节点将包含用于管理BeeGFS的清单和攻略手册。它需要：

- Ansible 6.x (Ansible核心2.13)
- Python 3.6 (或更高版本)
- Python (pip)软件包：ipaddr和netaddr

此外、建议您将控制节点中的无密码SSH设置为连接到所有BeeGFS文件节点和客户端。

BeeGFS文件节点

文件节点必须运行RedHat 8.4、并有权访问包含所需软件包(pacemaker、corosync、fence-agents-all、resource-agents.)的HA存储库。例如、可以执行以下命令在RedHat 8上启用相应的存储库：

```
subscription-manager repo-override repo=rhel-8-for-x86_64-
highavailability-rpms --add=enabled:1
```

BeeGFS客户端节点

可以使用BeeGFS客户端Ansible角色安装BeeGFS客户端软件包并管理BeeGFS挂载。此角色已通过RedHat 8.4和Ubuntu 22.04的测试。

如果未使用Ansible设置BeeGFS客户端并挂载BeeGFS、则为any "BeeGFS支持的Linux分发版和内核" 可以使用。

初始设置

安装硬件并为其布线

安装用于在NetApp上运行BeeGFS的硬件并为其布线所需的步骤。

规划安装

每个BeeGFS文件系统将包含一些文件节点、这些文件节点运行BeeGFS服务、并使用由某些数量的块节点提供的后端存储。文件节点配置为一个或多个高可用性集群、以便为BeeGFS服务提供容错功能。每个块节点都已是一个主动/主动HA对。每个HA集群中支持的文件节点的最小数量为3个、每个集群中支持的文件节点的最大数量为10个。BeeGFS文件系统可以通过部署多个独立的HA集群来扩展到10个以上节点、这些集群可以协同工作来提供一个文件系统命名空间。

通常、每个HA集群都部署为一系列"组件"、其中、一些文件节点(x86服务器)直接连接到某些数量的块节点(通常为E系列存储系统)。此配置会创建一个非对称集群、在此集群中、BeeGFS服务只能在某些文件节点上运行、这些文件节点可以访问用于BeeGFS目标的后端块存储。每个组件中的文件到块节点与直接连接所使用的存储协议的平衡取决于特定安装的要求。

另一种HA集群架构使用文件节点和块节点之间的存储网络结构(也称为存储区域网络或SAN)来建立对称集群。这样、BeeGFS服务便可在特定HA集群中的任何文件节点上运行。由于增加了SAN硬件、一般对称集群的成本效益不高、因此本文档假定使用的是部署为一系列一个或多个组件的非对称集群。

> 在继续安装之前、请确保已充分了解特定BeeGFS部署所需的文件系统架构。

机架硬件

在规划安装时、每个组件中的所有设备都必须安装在相邻的机架单元中。最佳实践是、将文件节点直接机架安装在每个构建块中的块节点上方。按照文件和型号对应的文档进行操作 "块" 在机架中安装导轨和硬件时所使用的节点。

单个组件示例：

[构建块示例]

大型BeeGFS安装示例、其中每个HA集群中有多个组件、文件系统中有多个HA集群：

[BeeGFS部署示例]

为文件和块节点布线

通常、您会将E系列块节点的HIC端口直接连接到文件节点的指定主机通道适配器(对于InfiniBand协议)或主机总线适配器(对于光纤通道和其他协议)端口。建立这些连接的确切方式取决于所需的文件系统架构、下面是一个示例 "基于经过NetApp验证的第二代BeeGFS架构"：

[示例BeeGFS文件到块节点布线]

使用缆线将文件节点连接到客户端网络

每个文件节点都将为BeeGFS客户端流量指定一定数量的InfiniBand或以太网端口。根据架构的不同、每个文件节点将有一个或多个连接到高性能客户端/存储网络、也可能连接到多个交换机以实现冗余并增加带宽。以下是使用冗余网络交换机进行客户端布线的示例、其中以深绿色和浅绿色突出显示的端口连接到不同的交换机：

[BeeGFS客户端布线示例]

连接管理网络和电源

建立带内和带外网络所需的任何网络连接。

连接所有电源、确保每个文件和块节点都连接到多个配电单元以实现冗余(如果可用)。

设置文件和块节点

在运行Ansible之前设置文件和块节点所需的手动步骤。

文件节点

配置基板管理控制器(BMC)

基板管理控制器(Baseboard Management Controller、BMC)有时也称为服务处理器、它是内置于各种服务器平台中的带外管理功能的通用名称、即使操作系统未安装或无法访问、也可以提供远程访问。供应商通常会使用自己的品牌推广此功能。例如、在联想SR665上、BMC称为联想XicLityController (XCC)。

按照服务器供应商的文档启用访问此功能所需的任何许可证、并确保BMC已连接到网络并已正确配置用于远程访问。

> 如果需要使用Redfish进行基于BMC的隔离、请确保已启用Redfish、并且可以从文件节点上安装的操作系统访问BMC界面。如果BMC和操作系统共享同一物理网络接口、则可能需要对网络交换机进行特殊配置。

调整系统设置

使用系统设置(BIOS/UEFI)界面、确保设置为最大程度地提高性能。确切设置和最佳值将因所使用的服务器型号而异。提供了相关指导 "已验证文件节点型号"、否则、请根据您的型号参考服务器供应商的文档和最佳实践。

安装操作系统

根据列出的文件节点要求安装受支持的操作系统 "此处"。根据您的Linux版本、请参见以下任何其他步骤。

RedHat

使用RedHat订阅管理器注册并订阅系统、以便可以从官方Red Hat存储库安装所需的软件包、并将更新限制为受支持的Red Hat版本： `subscription-manager release --set=<MAJOR_VERSION>.<MINOR_VERSION>`。有关说明，请参见 "如何注册和订阅RHEL系统" 和 "如何限制更新"。

启用包含高可用性所需软件包的Red Hat存储库：

```
subscription-manager repo-override --repo=rhel-8-for-x86_64
-highavailability-rpms --add=enabled:1
```

配置管理网络

配置所需的任何网络接口、以便对操作系统进行带内管理。具体步骤取决于所使用的特定Linux版本。

> 确保已启用SSH、并且可从Ansible控制节点访问所有管理接口。

更新HCA和HBA固件

确保所有HBA和HCA均运行上列出的受支持固件版本 "NetApp 互操作性表" 并根据需要进行升级。有关NVIDIA ConnectX适配器的其他建议、请参见 "此处"。

块节点

按照步骤执行操作 "启动并运行E系列" 在每个块节点控制器上配置管理端口、并可选择为每个系统设置存储阵列

名称。

> 除了确保可从Ansible控制节点访问所有块节点之外、无需进行其他配置。其余系统配置将使用Ansible进行应用/维护。

设置Ansible控制节点

设置Ansible控制节点以部署和管理文件系统。

概述

Ansible控制节点是用于管理集群的物理或虚拟Linux计算机。它必须满足以下要求：

- 了解 "要求" 用于BeeGFS HA角色、包括已安装的Ansible、Python和任何其他Python软件包版本。
- 与官方人员会面 "Ansible控制节点要求" 包括操作系统版本。
- 对所有文件和块节点具有SSH和HTTPS访问权限。

有关详细的安装步骤、请参见 "此处"。

定义BeeGFS文件系统

Ansible清单概述

Ansible清单是一组配置文件、用于定义所需的BeeGFS HA集群。

概述

建议遵循标准的Ansible实践来组织 "清单"、包括使用 "子目录/文件" 而不是将整个清单存储在一个文件中。

单个BeeGFS HA集群的Ansible清单组织如下：

[Ansible清单概述]

> 由于一个BeeGFS文件系统可以跨越多个HA集群、因此大型安装可能具有多个Ansible清单。通常、不建议尝试将多个HA集群定义为一个Ansible清单以避免出现问题。

步骤

1. 在Ansible控制节点上、创建一个空目录、该目录将包含要部署的BeeGFS集群的Ansible清单。
   a. 如果您的文件系统最终将包含/可能包含多个HA集群、建议先为文件系统创建一个目录、然后为表示每个HA集群的清单创建子目录。例如：

```
beegfs_file_system_1/
    beegfs_cluster_1/
    beegfs_cluster_2/
    beegfs_cluster_N/
```

2. 在包含要部署的HA集群清单的目录中、创建两个目录 `group_vars` 和 `host_vars` 和两个文件 `inventory.yml` 和 `playbook.yml`。

以下各节将逐步介绍如何定义每个文件的内容。

规划文件系统

在构建Ansible清单之前规划文件系统部署。

概述

在部署文件系统之前、您应定义集群中运行的所有文件节点、块节点和BeeGFS服务需要哪些IP地址、端口和其他配置。虽然具体配置会因集群架构而异、但本节定义了一般适用的最佳实践和应遵循的步骤。

步骤

1. 如果您使用基于IP的存储协议(例如iSER、iSCSI、NVMe/IB或NVMe/RoCE)将文件节点连接到块节点、请为每个组件填写以下工作表。一个构建块中的每个直接连接都应具有一个唯一的子网、并且不应与用于客户端-服务器连接的子网重叠。

文件节点	IB端口	IP 地址	块节点	IB端口	物理IP	虚拟IP (仅适用于具有HDR IB的EF600)
<HOSTNAME>	<PORT>	<IP/SUBNET>	<HOSTNAME>	<PORT>	<IP/SUBNET>	<IP/SUBNET>

> 如果每个构建块中的文件和块节点直接连接、您通常可以对多个构建块重复使用相同的IP/方案。

2. 无论您是在存储网络中使用InfiniBand还是基于融合以太网的RDMA (RoCE)、请填写以下工作表以确定将用于HA集群服务、BeeGFS文件服务和客户端进行通信的IP范围：

目的	**InfiniBand**端口	**IP**地址或范围
BeeGFS集群IP	<INTERFACE(s)>	<RANGE>
BeeGFS管理	<INTERFACE(s)>	<IP(s)>
BeeGFS元数据	<INTERFACE(s)>	<RANGE>
BeeGFS存储	<INTERFACE(s)>	<RANGE>
BeeGFS客户端	<INTERFACE(s)>	<RANGE>

   a. 如果您使用的是一个IP子网、则只需要一个工作表、否则还需要填写第二个子网的工作表。

3. 根据上述内容、为集群中的每个组件填写以下工作表、以定义要运行的BeeGFS服务。对于每个服务、请指定首选/二级文件节点、网络端口、浮动IP、NUMA分区分配(如果需要)以及要用于其目标的块节点。填写工作表时、请参见以下准则：

   a. 将BeeGFS服务指定为 `mgmt.yml`， `meta_<ID>.yml``或 `storage_<ID>.yml` 其中ID表示此文件系统中此类型的所有BeeGFS服务的唯一编号。此约定将简化在创建用于配置每个服务的文件时在后续章节中引用此工作表的过程。

   b. 用于BeeGFS服务的端口只需在特定组件中是唯一的。确保具有相同端口号的服务不能在同一文件节点上运行、以避免端口冲突。

   c. 如果需要、服务可以使用多个块节点和/或存储池中的卷(并非所有卷都需要归同一控制器所有)。多个服务也可以共享同一个块节点和/或存储池配置(各个卷将在后面的章节中进行定义)。

BeeGFS服务(文件名)	文件节点	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
<SERVICE TYPE> <ID>	<PREFERRED FILE NODE> <SECONDARY FILE NODE(s)>	<PORT>	<INTERFACE>：<IP/SUBNET> <INTERFACE>：<IP/SUBNET>	<NUMA NODE/ZONE>	<BLOCK NODE>	<STORAGE POOL/VOLUME GROUP>	<A OR B>

有关标准约定、最佳实践以及填写的示例工作表的详细信息、请参见 "最佳实践" 和 "定义BeeGFS组件" BeeGFS中有关NetApp经验证的架构的章节。

定义文件和块节点

配置单个文件节点

使用主机变量(host_vars)指定各个文件节点的配置。

概述

本节将逐步介绍填充 `host_vars/<FILE_NODE_HOSTNAME>.yml` 集群中每个文件节点的文件。这些文件只能包含特定文件节点独有的配置。这通常包括：

- 定义Ansible连接到节点时应使用的IP或主机名。
- 配置用于HA集群服务(Pacemaker和Corosync)的其他接口和集群IP、以便与其他文件节点进行通信。默认情况下、这些服务与管理接口使用相同的网络、但应提供更多接口以实现冗余。通常的做法是、在存储网络上定义其他IP、从而避免需要额外的集群或管理网络。
  - 用于集群通信的任何网络的性能对于文件系统性能并不重要。对于默认集群配置、通常至少1 Gbps的网络将为集群操作提供足够的性能、例如同步节点状态和协调集群资源状态更改。慢速/繁忙网络可能会使发生原因 资源状态发生变化、所需时间比平常要长、在极端情况下、如果节点无法在合理的时间范围内发送检测信号、可能会导致节点被逐出集群。
- 配置用于通过所需协议连接到块节点的接口(例如：iSCSI/iSER、NVMe/IB、NVMe/RoCE、FCP等)

步骤

引用中定义的IP地址方案 "规划文件系统" 部分中、为集群中的每个文件节点创建一个文件 `host_vars/<FILE_NODE_HOSTNAME>/yml` 并按如下所示进行填充：

1. 在顶部、指定Ansible通过SSH连接到节点并对其进行管理时应使用的IP或主机名：

```
ansible_host: "<MANAGEMENT_IP>"
```

2. 配置可用于集群流量的其他IP：
   a. 网络类型为 "InfiniBand (使用IPoIB)"：

```
eseries_ipoib_interfaces:
- name: <INTERFACE>  # Example: ib0 or i1b
  address: <IP/SUBNET> # Example: 100.127.100.1/16
- name: <INTERFACE>  # Additional interfaces as needed.
  address: <IP/SUBNET>
```

   b. 网络类型为 "基于融合以太网的RDMA (RoCE)"：

```
eseries_roce_interfaces:
- name: <INTERFACE>  # Example: eth0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
- name: <INTERFACE>  # Additional interfaces as needed.
  address: <IP/SUBNET>
```

   c. 网络类型为 "以太网(仅限TCP、无RDMA)"：

```
eseries_ip_interfaces:
- name: <INTERFACE>  # Example: eth0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
- name: <INTERFACE>  # Additional interfaces as needed.
  address: <IP/SUBNET>
```

3. 指示哪些IP应用于集群流量、优先IP列在上面：

```
beegfs_ha_cluster_node_ips:
- <MANAGEMENT_IP> # Including the management IP is typically but not
required.
- <IP_ADDRESS>    # Ex: 100.127.100.1
- <IP_ADDRESS>    # Additional IPs as needed.
```

> 步骤2中配置的IP不会用作集群IP、除非它们包含在中 `beegfs_ha_cluster_node_ips` 列表这样、您就可以使用Ansible配置其他IP /接口、如果需要、这些IP /接口可用于其他目的。

4. 如果文件节点需要通过基于IP的协议与块节点进行通信、则需要在相应的接口上配置IP、并安装/配置该协议所需的任何软件包。

   a. 如果使用 "iSCSI"：

```
eseries_iscsi_interfaces:
- name: <INTERFACE>  # Example: eth0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
```

   b. 如果使用 "iSER"：

```
eseries_ib_iser_interfaces:
- name: <INTERFACE>  # Example: ib0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
  configure: true # If the file node is directly connected to the
block node set to true to setup OpenSM.
```

   c. 如果使用 "NVMe/IB"：

```
eseries_nvme_ib_interfaces:
- name: <INTERFACE>  # Example: ib0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
  configure: true # If the file node is directly connected to the
block node set to true to setup OpenSM.
```

   d. 如果使用 "NVMe/RoCE"：

```
eseries_nvme_roce_interfaces:
- name: <INTERFACE>  # Example: eth0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
```

   e. 其他协议：

      i. 如果使用 "NVMe/FC"、不需要配置各个接口。BeeGFS集群部署将根据需要自动检测协议和安装/配置要求。如果您使用网络结构连接文件和块节点、请确保按照NetApp和交换机供应商的最佳实践正确对交换机进行分区。

      ii. 使用FCP或SAS不需要安装或配置其他软件。如果使用FCP、请确保交换机已正确分区、如下所示 "NetApp" 以及交换机供应商的最佳实践。

      iii. 目前不建议使用IB SRP。根据E系列块节点支持的内容、使用NVMe/IB或iSER。

单击 "此处" 有关表示单个文件节点的完整清单文件的示例。

高级：在以太网模式和**InfiniBand**模式之间切换**NVIDIA ConnectX VPI**适配器

NVIDIA ConnectX-Virtual Protocol Interconnect & reg；(VPI)适配器既支持InfiniBand、也支持以太网作为传输层。在模式之间切换不会自动协商、必须使用进行配置 `mstconfig` 中包含的工具 `mstflint`是中的一个开源软件包 "Mellanox Firmare工具(MFT)"。只需更改一次适配器的模式即可。此操作可以手动完成、也可以作为使用配置的任何接口的一部分包含在Ansible清单中 `eseries-[ib|ib_iser|ipoib|nvme_ib|nvme_roce|roce]_interfaces:` 部分、以使其自动检查/应用。

例如、要将InfiniBand模式下的接口电流更改为以太网、以便用于RoCE：

1. 对于要配置的每个接口、请指定 `mstconfig` 作为指定的映射(或词典) `LINK_TYPE_P<N>` 其中： `<N>` 由接口的HCA端口号决定。。 `<N>` 可以通过运行来确定值 `grep PCI_SLOT_NAME /sys/class/net/<INTERFACE_NAME>/device/uevent` 并从PCI插槽名称中将1添加到最后一个数字、然后转换为十进制值。
   a. 例如给定 `PCI_SLOT_NAME=0000:2f:00.2` (2 + 1 → HCA端口3)→ `LINK_TYPE_P3: eth`：

```
eseries_roce_interfaces:
- name: <INTERFACE>
  address: <IP/SUBNET>
  mstconfig:
    LINK_TYPE_P3: eth
```

有关更多详细信息、请参见 "NetApp E系列主机集合的文档" 所使用的接口类型/协议。

配置单个块节点

使用主机变量(host_vars)指定各个块节点的配置。

概述

本节将逐步介绍填充 `host_vars/<BLOCK_NODE_HOSTNAME>.yml` 集群中每个块节点的文件。这些文件只能包含特定块节点特有的配置。这通常包括：

- 系统名称(如System Manager中所示)。
- 其中一个控制器的HTTPS URL (用于使用其REST API管理系统)。
- 用于连接到此块节点的存储协议文件节点。
- 配置主机接口卡(HIC)端口、例如IP地址(如果需要)。

步骤

引用中定义的IP地址方案 "规划文件系统" 部分中、为集群中的每个块节点创建一个文件 `host_vars/<BLOCK_NODE_HOSTNAME>/yml` 并按如下所示进行填充：

1. 在顶部指定其中一个控制器的系统名称和HTTPS URL：

```
eseries_system_name: <SYSTEM_NAME>
eseries_system_api_url:
https://<MANAGEMENT_HOSTNAME_OR_IP>:8443/devmgr/v2/
```

2. 选择 "协议" 文件节点将用于连接到此块节点：
   a. 支持的协议： `auto`， `iscsi`， `fc`， `sas`， `ib_srp`， `ib_iser`， `nvme_ib`， `nvme_fc`， `nvme_roce`。

   ```
   eseries_initiator_protocol: <PROTOCOL>
   ```

3. 根据所使用的协议、HIC端口可能需要额外配置。如果需要、应定义HIC端口配置、以便每个控制器配置中的顶部条目与每个控制器上最左侧的物理端口相对应、底部端口对应最右侧的端口。所有端口都需要有效配置、即使它们当前未在使用中也是如此。

   > 如果要将HDR (200 GB) InfiniBand或200 GB RoCE与EF600块节点结合使用、另请参见以下部分。

   a. 对于iSCSI：

```
eseries_controller_iscsi_port:
  controller_a:         # Ordered list of controller A channel
definition.
    - state:            # Whether the port should be enabled.
Choices: enabled, disabled
      config_method:    # Port configuration method Choices: static,
dhcp
      address:          # Port IPv4 address
      gateway:          # Port IPv4 gateway
      subnet_mask:      # Port IPv4 subnet_mask
      mtu:              # Port IPv4 mtu
    - (...)             # Additional ports as needed.
  controller_b:         # Ordered list of controller B channel
definition.
    - (...)             # Same as controller A but for controller B

# Alternatively the following common port configuration can be
defined for all ports and omitted above:
eseries_controller_iscsi_port_state: enabled         # Generally
specifies whether a controller port definition should be applied
Choices: enabled, disabled
eseries_controller_iscsi_port_config_method: dhcp    # General port
configuration method definition for both controllers. Choices:
static, dhcp
eseries_controller_iscsi_port_gateway:               # General port
IPv4 gateway for both controllers.
eseries_controller_iscsi_port_subnet_mask:           # General port
IPv4 subnet mask for both controllers.
eseries_controller_iscsi_port_mtu: 9000              # General port
maximum transfer units (MTU) for both controllers. Any value greater
than 1500 (bytes).
```

b. 对于iSER：

```
eseries_controller_ib_iser_port:
  controller_a:    # Ordered list of controller A channel address
definition.
    -              # Port IPv4 address for channel 1
    - (...)        # So on and so forth
  controller_b:    # Ordered list of controller B channel address
definition.
```

c. 对于NVMe/IB：

```
eseries_controller_nvme_ib_port:
  controller_a:    # Ordered list of controller A channel address
definition.
    -              # Port IPv4 address for channel 1
    - (...)        # So on and so forth
  controller_b:    # Ordered list of controller B channel address
definition.
```

d. 对于NVMe/RoCE：

```
eseries_controller_nvme_roce_port:
  controller_a:         # Ordered list of controller A channel
definition.
    - state:            # Whether the port should be enabled.
      config_method:    # Port configuration method Choices: static,
dhcp
      address:          # Port IPv4 address
      subnet_mask:      # Port IPv4 subnet_mask
      gateway:          # Port IPv4 gateway
      mtu:              # Port IPv4 mtu
      speed:            # Port IPv4 speed
  controller_b:         # Ordered list of controller B channel
definition.
    - (...)             # Same as controller A but for controller B

# Alternatively the following common port configuration can be
defined for all ports and omitted above:
eseries_controller_nvme_roce_port_state: enabled         # Generally
specifies whether a controller port definition should be applied
Choices: enabled, disabled
eseries_controller_nvme_roce_port_config_method: dhcp    # General
port configuration method definition for both controllers. Choices:
static, dhcp
eseries_controller_nvme_roce_port_gateway:               # General
port IPv4 gateway for both controllers.
eseries_controller_nvme_roce_port_subnet_mask:           # General
port IPv4 subnet mask for both controllers.
eseries_controller_nvme_roce_port_mtu: 4200              # General
port maximum transfer units (MTU). Any value greater than 1500
(bytes).
eseries_controller_nvme_roce_port_speed: auto            # General
interface speed. Value must be a supported speed or auto for
automatically negotiating the speed with the port.
```

e. FC和SAS协议不需要额外配置。不正确建议使用SRP。

有关配置HIC端口和主机协议的其他选项、包括配置iSCSI CHAP的功能、请参见 "文档。" 包含在SANtricity 集合中。注意在部署BeeGFS时、存储池、卷配置以及配置存储的其他方面将在其他位置进行配置、不应在此文件中定义。

单击 "此处" 表示单个块节点的完整清单文件示例。

将HDR (200 GB) InfiniBand或200 GB RoCE与NetApp EF600块节点结合使用：

要将HDR (200 GB) InfiniBand与EF600结合使用、必须为每个物理端口配置第二个"虚拟" IP。以下是配置配有双端口InfiniBand HDR HIC的EF600的正确方法示例：

```
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101 # Port 2a (physical)
    - 192.168.2.101 # Port 2a (virtual)
    - 192.168.1.100 # Port 2b (physical)
    - 192.168.2.100 # Port 2b (virtual)
  controller_b:
    - 192.168.3.101 # Port 2a (physical)
    - 192.168.4.101 # Port 2a (virtual)
    - 192.168.3.100 # Port 2b (physical)
    - 192.168.4.100 # Port 2b (virtual)
```

指定通用文件节点配置

使用组变量(group_vars)指定通用文件节点配置。

概述

应连接到所有文件节点的配置在中进行定义 `group_vars/ha_cluster.yml`。通常包括：

- 有关如何连接和登录到每个文件节点的详细信息。
- 通用网络配置。
- 是否允许自动重新启动。
- 应如何配置防火墙和SELinux状态。
- 集群配置、包括警报和隔离。
- 性能调整。
- 常见BeeGFS服务配置。

> 此外、还可以在各个文件节点上定义此文件中设置的选项、例如、如果使用的是混合硬件型号、或者每个节点的密码不同。单个文件节点上的配置将优先于此文件中的配置。

步骤

创建文件 `group_vars/ha_cluster.yml` 并按如下所示进行填充：

1. 指示Ansible Control节点应如何向远程主机进行身份验证：

```
ansible_ssh_user: root
ansible_become_password: <PASSWORD>
```

> 尤其是在生产环境中、不要以纯文本格式存储密码。请改用Ansible Vault (请参见 "使用Ansible Vault加密内容")或 `--ask-become-pass` 运行攻略手册时的选项。如果 `ansible_ssh_user` 已为root用户、则可以选择省略 `ansible_become_password`。

2. 如果要在以太网或InfiniBand接口上配置静态IP (例如集群IP)、并且多个接口位于同一IP子网中(例如、ib0使用192.168.1.10/24、ib1使用192.168.1.11/24)、要使多主机支持正常工作、必须设置其他IP路由表和规则。只需按如下所示启用提供的网络接口配置挂钩：

```
eseries_ip_default_hook_templates:
  - 99-multihoming.j2
```

3. 部署集群时、根据存储协议的不同、可能需要重新启动节点、以便于发现远程块设备(E系列卷)或应用配置的其他方面。默认情况下、节点将在重新启动之前进行提示、但您可以通过指定以下内容来允许节点自动重新启动：

```
eseries_common_allow_host_reboot: true
```

   a. 默认情况下、重新启动后、为确保块设备和其他服务准备就绪、Ansible将等待系统完成 `default.target` 将在继续部署之前访问。在某些使用NVMe/IB的情况下、此过程可能不够长、无法初始化、发现和连接到远程设备。这可能会导致自动部署过早继续并失败。为避免在使用NVMe/IB时出现这种情况、还应定义以下内容：

```
eseries_common_reboot_test_command: "! systemctl status
eseries_nvme_ib.service || systemctl --state=exited | grep
eseries_nvme_ib.service"
```

4. 要使BeeGFS和HA集群服务进行通信、需要使用多个防火墙端口。除非您要手动配置固件(不建议)、否则请指定以下内容以创建所需的防火墙区域并自动打开端口：

```
beegfs_ha_firewall_configure: True
```

5. 目前不支持SELinux、建议将状态设置为disabled以避免冲突(尤其是在使用RDMA时)。设置以下内容以确保SELinux已禁用：

```
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
```

6. 配置身份验证以使文件节点能够进行通信、并根据组织策略根据需要调整默认值：

```
beegfs_ha_cluster_name: hacluster                  # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster              # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword             # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
```

7. 基于 "规划文件系统" 部分指定此文件系统的BeeGFS管理IP：

```
beegfs_ha_mgmtd_floating_ip: <IP ADDRESS>
```

> 虽然`冗余、但当您将BeeGFS文件系统扩展到单个HA集群之外时、`beegfs_ha_mgmtd_floating_ip非常重要。部署后续HA集群时无需额外的BeeGFS管理服务、并指向第一个集群提供的管理服务。

8. 根据需要启用电子邮件警报：

```
beegfs_ha_enable_alerts: True
# E-mail recipient list for notifications when BeeGFS HA resources
change or fail.
beegfs_ha_alert_email_list: ["<EMAIL>"]
# This dictionary is used to configure postfix service
(/etc/postfix/main.cf) which is required to set email alerts.
beegfs_ha_alert_conf_ha_group_options:
      # This parameter specifies the local internet domain name. This is
optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com)
      mydomain: <MY_DOMAIN>
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

9. 强烈建议启用隔离、否则、在主节点发生故障时、可能会阻止服务在二级节点上启动。

a. 通过指定以下内容全局启用隔离：

```
beegfs_ha_cluster_crm_config_options:
  stonith-enabled: True
```

i. 记下任何受支持的 "集群属性" 如果需要、也可以在此处指定。通常不需要调整这些值、因为BeeGFS HA角色具有许多经过良好测试的功能 "默认值"。

b. 接下来、选择并配置隔离代理：

i. 选项1：要使用APC配电单元(PDU)启用隔离、请执行以下操作：

```
beegfs_ha_fencing_agents:
  fence_apc:
    - ipaddr: <PDU_IP_ADDRESS>
      login: <PDU_USERNAME>
      passwd: <PDU_PASSWORD>
      pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>
"
```

ii. 选项2：要使用联想XCC (及其他BMC)提供的Redffish API启用隔离：

```
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify “1”.

beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
```

iii. 有关配置其他隔离代理的详细信息、请参见 "RedHat文档"。

10. BeeGFS HA角色可以应用多种不同的调整参数来帮助进一步优化性能。其中包括优化内核内存利用率和块设备I/O等参数。角色附带一组合理的 "默认值" 根据使用NetApp E系列块节点进行的测试、但默认情况下、除非您指定以下内容、否则不会应用这些块节点：

```
beegfs_ha_enable_performance_tuning: True
```

a. 如果需要、还可以在此处指定对默认性能调整所做的任何更改。请参见完整内容 "性能调整参数" 有关其他详细信息、请参见相关文档。

11. 为了确保用于BeeGFS服务的浮动IP地址(有时称为逻辑接口)可以在文件节点之间进行故障转移、所有网络接口的名称必须一致。默认情况下、网络接口名称由内核生成、即使在安装了相同PCIe插槽中的网络适配器的相同服务器型号上、也不能保证生成一致的名称。在部署设备之前创建清单并知道生成的接口名称时、这一点也很有用。以确保设备名称一致、具体取决于服务器或的方框图 `lshw -class network -businfo` 输出中、按如下所示指定所需的PCIe地址到逻辑接口映射：

a. 对于InfiniBand (IPoIB)网络接口：

```
eseries_ipoib_udev_rules:
  "<PCIe ADDRESS>": <NAME> # Ex: 0000:41:00.0: i1a
```

b. 对于以太网网络接口：

```
eseries_ip_udev_rules:
  "<PCIe ADDRESS>": <NAME> # Ex: 0000:41:00.0: e1a
```

> 为避免重命名接口时发生冲突(防止重命名接口)、不应使用任何可能的默认名称、例如eth0、ens9f0、ib0或ibs4f0。一种常见的命名约定是、对以太网或InfiniBand使用"e"或"i"、后跟PCIe插槽编号和一个字母以指示端口。例如、插槽3中安装的InfiniBand适配器的第二个端口为：i3b。

> 如果您使用的是经验证的文件节点型号、请单击 "此处" PCIe地址到逻辑端口映射示例。

12. (可选)指定应应用于集群中所有BeeGFS服务的配置。可以找到默认配置值 "此处"、并在其他位置指定每服务配置：

a. BeeGFS管理服务：

```
beegfs_ha_beegfs_mgmtd_conf_ha_group_options:
  <OPTION>: <VALUE>
```

b. BeeGFS元数据服务：

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  <OPTION>: <VALUE>
```

c. BeeGFS存储服务：

```
beegfs_ha_beegfs_storage_conf_ha_group_options:
  <OPTION>: <VALUE>
```

13. 截至BeeGFS 7.2.7和7.3.1 "连接身份验证" 必须配置或显式禁用。使用基于Ansible的部署可以通过以下几种方式进行配置：
    a. 默认情况下、部署将自动配置连接身份验证并生成 `connauthfile` 该文件将分发到所有文件节点、并与BeeGFS服务结合使用。此文件还将放置/维护在位于的Ansible控制节点上 `<INVENTORY>/files/beegfs/<sysMgmtdHost>_connAuthFile` 应(安全)维护的位置、以便在需要访问此文件系统的客户端中重复使用。
        i. 要生成新密钥、请指定 `-e "beegfs_ha_conn_auth_force_new=True` 运行Ansible攻略手册时。请注意、如果出现、则会忽略此问题 `beegfs_ha_conn_auth_secret` 已定义。
        ii. 有关高级选项、请参阅随附的默认值完整列表 "BeeGFS HA角色"。
    b. 可以通过在中定义以下内容来使用自定义密钥 `ha_cluster.yml`：

```
beegfs_ha_conn_auth_secret: <SECRET>
```

    c. 可以完全禁用连接身份验证(不建议使用)：

```
beegfs_ha_conn_auth_enabled: false
```

单击 "此处" 有关表示通用文件节点配置的完整清单文件的示例。

将**HDR (200 GB) InfiniBand**与**NetApp EF600**块节点结合使用：

要将HDR (200 GB) InfiniBand与EF600结合使用、子网管理器必须支持虚拟化。如果使用交换机连接文件和块节点、则需要在子网管理器管理器上为整个网络结构启用此功能。

如果块和文件节点使用InfiniBand直接连接、则为的一个实例 `opensm` 必须在每个文件节点上为直接连接到块节点的每个接口配置。可通过指定来完成此操作 `configure: true` 时间 "配置文件节点存储接口"。

当前的收件箱版本 `opensm` 随受支持的Linux分发版一起提供、不支持虚拟化。而是需要您安装和配置的版本 `opensm` 来自Mellanox OpenFabrics Enterprise Distribution (OFED)。尽管仍支持使用Ansible进行部署、但还需要执行一些额外步骤：

1. 使用curl或所需工具下载中列出的OpenSM版本的软件包 "技术要求" 从Mellanox网站到的部分 `<INVENTORY>/packages/` 目录。例如：

```
curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm

curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
```

2. 下 `group_vars/ha_cluster.yml` 定义以下配置：

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm

eseries_ib_opensm_options:
  virt_enabled: "2"
```

使用组变量(group_vars)指定通用块节点配置。

概述

应通过定义适用于所有块节点的配置 `group_vars/eseries_storage_systems.yml`。通常包括：

- 有关Ansible控制节点应如何连接到用作块节点的E系列存储系统的详细信息。
- 节点应运行的固件、NVSRAM和驱动器固件版本。
- 全局配置、包括缓存设置、主机配置以及应如何配置卷的设置。

> 此外、还可以在各个块节点上定义此文件中设置的选项、例如、如果使用的是混合硬件型号、或者每个节点的密码不同。单个块节点上的配置将优先于此文件中的配置。

步骤

创建文件 `group_vars/eseries_storage_systems.yml` 并按如下所示进行填充：

1. Ansible不使用SSH连接到块节点、而是使用REST API。为此、我们必须设置：

```
ansible_connection: local
```

2. 指定用于管理每个节点的用户名和密码。用户名可以选择省略(并默认为admin)、否则您可以指定具有管理员权限的任何帐户。此外、还应指定是验证SSL证书还是忽略SSL证书：

```
eseries_system_username: admin
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> 不建议以纯文本格式列出任何密码。使用Ansible存储或提供 `eseries_system_password` 使用-extra vars运行Ansible时。

3. 也可以指定应在节点上安装的控制器固件、NVSRAM和驱动器固件。需要将这些文件下载到 `packages/` 目录、然后运行Ansible。可以下载E系列控制器固件和NVSRAM "此处" 和驱动器固件 "此处"：

```
eseries_firmware_firmware: "packages/<FILENAME>.dlp" # Ex.
"packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/<FILENAME>.dlp" # Ex.
"packages/N6000-872834-D06.dlp"
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
  # Additional firmware versions as needed.
eseries_drive_firmware_upgrade_drives_online: true # Recommended unless
BeeGFS hasn't been deployed yet, as it will disrupt host access if set
to "false".
```

> 如果指定了此配置、Ansible将自动更新所有固件、包括重新启动控制器(如有必要)、而不会出现其他提示。这对BeeGFS/主机I/O预期不会造成中断、但发生原因 性能可能会暂时下降。

4. 调整全局系统配置默认值。此处列出的选项和值通常建议用于NetApp上的BeeGFS、但可以根据需要进行调整：

```
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required by default.
```

5. 配置全局卷配置默认值。此处列出的选项和值通常建议用于NetApp上的BeeGFS、但可以根据需要进行调整：

```
eseries_volume_size_unit: pct # Required by default. This allows volume
capacities to be specified as a percentage, simplifying putting together
the inventory.
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
```

6. 如果需要、请根据以下最佳实践调整Ansible为存储池和卷组选择驱动器的顺序：
   a. 列出应首先用于管理和/或元数据卷、最后用于存储卷的任何(可能较小)驱动器。
   b. 确保根据磁盘架/驱动器机箱型号在可用驱动器通道之间平衡驱动器选择顺序。例如、如果使用EF600且不进行扩展、则驱动器0-11位于驱动器通道1上、驱动器12-23位于驱动器通道上。因此、要选择一种平衡驱动器选择的策略 `disk shelf:drive` 99：0、99：23、99：1、99：22等如果存在多个机箱、则第一个数字表示驱动器架ID。

```
# Optimal/recommended order for the EF600 (no expansion):
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99
:6,99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

单击 "此处" 有关表示通用块节点配置的完整清单文件的示例。

定义BeeGFS服务

定义BeeGFS管理服务

BeeGFS服务使用组变量(group_vars)进行配置。

概述

本节将介绍如何定义BeeGFS管理服务。对于特定文件系统、HA集群中只应存在一个此类型的服务。配置此服务包括定义：

- 服务类型(管理)。
- 定义应仅适用于此BeeGFS服务的任何配置。
- 配置可访问此服务的一个或多个浮动IP (逻辑接口)。
- 指定卷存储此服务数据的位置/方式(BeeGFS管理目标)。

步骤

创建新文件 `group_vars/mgmt.yml` 并引用 "规划文件系统" 部分将其填充如下：

1. 指示此文件表示BeeGFS管理服务的配置：

```
beegfs_service: management
```

2. 定义应仅应用于此BeeGFS服务的任何配置。管理服务通常不需要执行此操作、除非您需要启用配额、无论您的任何受支持的配置参数如何 `beegfs-mgmtd.conf` 可以包括在内。请注意、以下参数是自动/在其他位置配置的、不应在此处指定： `storeMgmtdDirectory`， `connAuthFile`， `connDisableAuthentication`， `connInterfacesFile`，和 `connNetFilterFile`。

```
beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
```

3. 配置一个或多个浮动IP、供其他服务和客户端连接到此服务时使用(此操作将自动设置BeeGFS `connInterfacesFile` 选项)：

```
floating_ips:
  - <INTERFACE>:<IP/SUBNET> # Primary interface. Ex.
i1b:100.127.101.0/16
  - <INTERFACE>:<IP/SUBNET> # Secondary interface(s) as needed.
```

4. 或者、指定一个或多个允许用于传出通信的IP子网(此操作将自动设置BeeGFS `connNetFilterFile` 选项)：

```
filter_ip_ranges:
  - <SUBNET>/<MASK> # Ex. 192.168.10.0/24
```

5. 根据以下准则指定此服务将在其中存储数据的BeeGFS管理目标：
   a. 同一存储池或卷组名称可用于多个BeeGFS服务/目标、只需确保使用相同的即可 `name`， `raid_level`， `criteria_*`，和 `common_*` 每个服务的配置(为每个服务列出的卷应不同)。
   b. 卷大小应指定为存储池/卷组的百分比、并且使用特定存储池/卷组的所有服务/卷的总数不应超过100。注意：在使用SSD时、建议在卷组中保留一些可用空间、以最大限度地提高SSD性能和使用寿命(单击 "此处" 了解更多详细信息）。
   c. 单击 "此处" 以获取可用于的配置选项的完整列表 `eseries_storage_pool_configuration`。请注意一些选项、例如 `state`， `host`， `host_type`， `workload_name`，和 `workload_metadata` 和卷名称将自动生成、不应在此指定。

```
beegfs_targets:
  <BLOCK_NODE>: # The name of the block node as found in the Ansible
inventory. Ex: ictad22a01
    eseries_storage_pool_configuration:
      - name: <NAME> # Ex: beegfs_m1_m2_m5_m6
        raid_level: <LEVEL> # One of: raid1, raid5, raid6, raidDiskPool
        criteria_drive_count: <DRIVE COUNT> # Ex. 4
        common_volume_configuration:
          segment_size_kb: <SEGMENT SIZE> # Ex. 128
        volumes:
          - size: <PERCENT> # Percent of the pool or volume group to
allocate to this volume. Ex. 1
            owning_controller: <CONTROLLER> # One of: A, B
```

单击 "此处" 有关表示BeeGFS管理服务的完整清单文件的示例。

定义**BeeGFS**元数据服务

BeeGFS服务使用组变量(group_vars)进行配置。

概述

本节将介绍如何定义BeeGFS元数据服务。对于特定文件系统、HA集群中应至少存在一个此类型的服务。配置此服务包括定义：

- 服务类型(元数据)。
- 定义应仅适用于此BeeGFS服务的任何配置。
- 配置可访问此服务的一个或多个浮动IP (逻辑接口)。
- 指定卷存储此服务数据的位置/方式(BeeGFS元数据目标)。

步骤

参考 "规划文件系统" 部分中、在创建文件 `group_vars/meta_<ID>.yml` 对于集群中的每个元数据服务、请按如下所示进行填充：

1. 指示此文件表示BeeGFS元数据服务的配置：

```
beegfs_service: metadata
```

2. 定义应仅应用于此BeeGFS服务的任何配置。您必须至少指定所需的TCP和UDP端口、无论这些端口来自哪个受支持的配置参数 `beegfs-meta.conf` 也可以包括在内。请注意、以下参数是自动/在其他位置配置的、不应在此处指定： `sysMgmtdHost`， `storeMetaDirectory`， `connAuthFile`， `connDisableAuthentication`， `connInterfacesFile`，和 `connNetFilterFile`。

```
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <TCP PORT>
  connMetaPortUDP: <UDP PORT>
  tuneBindToNumaZone: <NUMA ZONE> # Recommended if using file nodes with
multiple CPU sockets.
```

3. 配置一个或多个浮动IP、供其他服务和客户端连接到此服务时使用(此操作将自动设置BeeGFS `connInterfacesFile` 选项)：

```
floating_ips:
  - <INTERFACE>:<IP/SUBNET> # Primary interface. Ex.
i1b:100.127.101.1/16
  - <INTERFACE>:<IP/SUBNET> # Secondary interface(s) as needed.
```

4. 或者、指定一个或多个允许用于传出通信的IP子网(此操作将自动设置BeeGFS `connNetFilterFile` 选项)：

```
filter_ip_ranges:
  - <SUBNET>/<MASK> # Ex. 192.168.10.0/24
```

5. 根据以下准则指定此服务将在其中存储数据的BeeGFS元数据目标(此操作也会自动配置) `storeMetaDirectory` 选项：

   a. 同一存储池或卷组名称可用于多个BeeGFS服务/目标、只需确保使用相同的即可 `name`， `raid_level`， `criteria_*`，和 `common_*` 每个服务的配置(为每个服务列出的卷应不同)。

   b. 卷大小应指定为存储池/卷组的百分比、并且使用特定存储池/卷组的所有服务/卷的总数不应超过100。注意：在使用SSD时、建议在卷组中保留一些可用空间、以最大限度地提高SSD性能和使用寿命(单击 "此处" 了解更多详细信息）。

   c. 单击 "此处" 以获取可用于的配置选项的完整列表 `eseries_storage_pool_configuration`。请注意一些选项、例如 `state`， `host`， `host_type`， `workload_name`，和 `workload_metadata` 和卷名称将自动生成、不应在此指定。

```
beegfs_targets:
  <BLOCK_NODE>: # The name of the block node as found in the Ansible
inventory. Ex: ictad22a01
    eseries_storage_pool_configuration:
      - name: <NAME> # Ex: beegfs_m1_m2_m5_m6
        raid_level: <LEVEL> # One of: raid1, raid5, raid6, raidDiskPool
        criteria_drive_count: <DRIVE COUNT> # Ex. 4
        common_volume_configuration:
          segment_size_kb: <SEGMENT SIZE> # Ex. 128
        volumes:
          - size: <PERCENT> # Percent of the pool or volume group to
allocate to this volume. Ex. 1
            owning_controller: <CONTROLLER> # One of: A, B
```

单击 "此处" 有关表示BeeGFS元数据服务的完整清单文件的示例。

定义**BeeGFS**存储服务

BeeGFS服务使用组变量(group_vars)进行配置。

概述

本节将介绍如何定义BeeGFS存储服务。对于特定文件系统、HA集群中应至少存在一个此类型的服务。配置此服务包括定义：

- 服务类型(存储)。
- 定义应仅适用于此BeeGFS服务的任何配置。
- 配置可访问此服务的一个或多个浮动IP (逻辑接口)。
- 指定卷存储此服务数据的位置/方式(BeeGFS存储目标)。

步骤

参考 "规划文件系统" 部分中、在创建文件 `group_vars/stor_<ID>.yml` 对于集群中的每个存储服务、请按如下所示进行填充：

1. 指示此文件表示BeeGFS存储服务的配置：

```
beegfs_service: storage
```

2. 定义应仅应用于此BeeGFS服务的任何配置。您必须至少指定所需的TCP和UDP端口、无论这些端口来自哪个受支持的配置参数 `beegfs-storage.conf` 也可以包括在内。请注意、以下参数是自动/在其他位置配置的、不应在此处指定： `sysMgmtdHost`， `storeStorageDirectory`， `connAuthFile`， `connDisableAuthentication`， `connInterfacesFile`，和 `connNetFilterFile`。

```
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <TCP PORT>
  connStoragePortUDP: <UDP PORT>
  tuneBindToNumaZone: <NUMA ZONE> # Recommended if using file nodes with
multiple CPU sockets.
```

3. 配置一个或多个浮动IP、供其他服务和客户端连接到此服务时使用(此操作将自动设置BeeGFS `connInterfacesFile` 选项)：

```
floating_ips:
  - <INTERFACE>:<IP/SUBNET> # Primary interface. Ex.
i1b:100.127.101.1/16
  - <INTERFACE>:<IP/SUBNET> # Secondary interface(s) as needed.
```

4. 或者、指定一个或多个允许用于传出通信的IP子网(此操作将自动设置BeeGFS `connNetFilterFile` 选项)：

```
filter_ip_ranges:
  - <SUBNET>/<MASK> # Ex. 192.168.10.0/24
```

5. 根据以下准则指定此服务要存储数据的BeeGFS存储目标(此操作也会自动配置) `storeStorageDirectory` 选项)：

   a. 同一存储池或卷组名称可用于多个BeeGFS服务/目标、只需确保使用相同的即可 `name`， `raid_level`， `criteria_*`，和 `common_*` 每个服务的配置(为每个服务列出的卷应不同)。

   b. 卷大小应指定为存储池/卷组的百分比、并且使用特定存储池/卷组的所有服务/卷的总数不应超过100。注意：在使用SSD时、建议在卷组中保留一些可用空间、以最大限度地提高SSD性能和使用寿命(单击 "此处" 了解更多详细信息）。

   c. 单击 "此处" 以获取可用于的配置选项的完整列表 `eseries_storage_pool_configuration`。请注意一些选项、例如 `state`， `host`， `host_type`， `workload_name`，和 `workload_metadata` 和卷名称将自动生成、不应在此指定。

```
beegfs_targets:
  <BLOCK_NODE>: # The name of the block node as found in the Ansible
inventory. Ex: ictad22a01
    eseries_storage_pool_configuration:
      - name: <NAME> # Ex: beegfs_s1_s2
        raid_level: <LEVEL> # One of: raid1, raid5, raid6,
raidDiskPool
        criteria_drive_count: <DRIVE COUNT> # Ex. 4
        common_volume_configuration:
          segment_size_kb: <SEGMENT SIZE> # Ex. 128
        volumes:
          - size: <PERCENT> # Percent of the pool or volume group to
allocate to this volume. Ex. 1
            owning_controller: <CONTROLLER> # One of: A, B
        # Multiple storage targets are supported / typical:
          - size: <PERCENT> # Percent of the pool or volume group to
allocate to this volume. Ex. 1
            owning_controller: <CONTROLLER> # One of: A, B
```

单击 "此处" 有关表示BeeGFS存储服务的完整清单文件的示例。

将BeeGFS服务映射到文件节点

使用指定可以运行每个BeeGFS服务的文件节点 `inventory.yml` 文件

概述

本节将介绍如何创建 `inventory.yml` 文件其中包括列出所有块节点并指定可以运行每个BeeGFS服务的文件节点。

步骤

创建文件 `inventory.yml` 并按如下所示进行填充：

1. 从文件顶部、创建标准Ansible清单结构：

```
# BeeGFS HA (High_Availability) cluster inventory.
all:
  children:
```

2. 创建一个包含参与此HA集群的所有块节点的组：

```
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        <BLOCK NODE HOSTNAME>:
        <BLOCK NODE HOSTNAME>:
        # Additional block nodes as needed.
```

3. 创建一个组、其中包含集群中的所有BeeGFS服务以及要运行这些服务的文件节点：

```
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

4. 对于集群中的每个BeeGFS服务、定义应运行该服务的首选文件节点和任何二级文件节点：

```
        <SERVICE>: # Ex. "mgmt", "meta_01", or "stor_01".
          hosts:
            <FILE NODE HOSTNAME>:
            <FILE NODE HOSTNAME>:
            # Additional file nodes as needed.
```

单击 "此处" 有关完整清单文件的示例。

部署BeeGFS文件系统

Ansible攻略手册概述

使用Ansible部署和管理BeeGFS HA集群。

概述

前面几节介绍了构建表示BeeGFS HA集群的Ansible清单所需的步骤。本节介绍由NetApp开发的用于部署和管理集群的Ansible自动化功能。

Ansible：关键概念

在继续操作之前、熟悉几个关键的Ansible概念会很有帮助：

- 根据Ansible清单执行的任务在称为*攻略手册*的内容中进行了定义。
  - Ansible中的大多数任务都设计为*幂等*、这意味着可以多次运行这些任务、以验证所需的配置/状态是否仍然适用、而不会造成中断或进行不必要的更新。
- Ansible中最小的执行单位是*模块*。

    - 典型的攻略手册使用多个模块。
        - 示例：下载软件包、更新配置文件、启动/启用服务。
    - NetApp分发模块以自动执行NetApp E系列系统。
- 更好地将复杂的自动化作为一个角色进行打包。
    - 基本上是分发可重复使用的攻略手册的标准格式。
    - NetApp为Linux主机和BeeGFS文件系统分发角色。

BeeGFS HA Role for Ansible：关键概念

在NetApp上部署和管理每个版本的BeeGFS所需的所有自动化功能均作为Ansible角色打包、并作为的一部分进行分发 "适用于BeeGFS的NetApp E系列Ansible资料集"：

- 可以将此角色视为BeeGFS的*安装程序*和现代*部署/管理*引擎之间的某个位置。
    - 将现代基础架构应用为代码实践和理念、以简化任何规模的存储基础架构管理。
    - 与类似 "Kubespray" Project允许用户部署/维护整个Kubernetes分发版、以横向扩展计算基础架构。
- 此角色是NetApp用于打包、分发和维护基于NetApp的BeeGFS解决方案的*软件定义*格式。
    - 无需分发整个Linux分发版或大型映像、即可努力打造"类似设备的"体验。
    - 包括NetApp编写的符合Open Cluster Framework (OCF)的集群资源代理、用于自定义BeeGFS目标、IP地址和监控、从而实现智能Pacemaker/BeeGFS集成。
- 此角色不仅仅是部署"自动化"、还用于管理整个文件系统生命周期、包括：
    - 应用按服务或集群范围的配置更改和更新。
    - 解决硬件问题后自动执行集群修复和恢复。
    - 通过对BeeGFS和NetApp卷进行广泛测试来设置默认值、简化性能调整。
    - 验证并更正配置偏差。

NetApp还为提供了Ansible角色 "BeeGFS客户端"、可选择用于安装BeeGFS并将文件系统挂载到计算/GPU/登录节点。

部署BeeGFS HA集群

使用攻略手册指定部署BeeGFS HA集群时应运行的任务。

概述

本节介绍如何组合用于在NetApp上部署/管理BeeGFS的标准攻略手册。

步骤

创建Ansible攻略手册

创建文件 `playbook.yml` 并按如下所示进行填充：

1. 首先定义一组任务(通常称为 "播放")、并且只能在NetApp E系列块节点上运行。我们会使用暂停任务在运行

安装之前进行提示(以避免意外运行攻略手册)、然后导入 `nar_santricity_management` 角色。此角色负责应用中定义的任何常规系统配置 `group_vars/eseries_storage_systems.yml` 或个人 `host_vars/<BLOCK NODE>.yml` 文件。

```
- hosts: eseries_storage_systems
  gather_facts: false
  collections:
    - netapp_eseries.santricity
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
```

2. 定义对所有文件和块节点运行的播放：

```
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
```

3. 在此角色中、我们可以选择定义一组"预任务"、这些任务应在部署HA集群之前运行。这对于验证/安装Python等任何前提条件非常有用。我们还可以注入任何飞行前检查、例如验证是否支持提供的Ansible标记：

```
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version

        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
```

```
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'

        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true

        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
groups[beegfs_ha_ansible_storage_group]

    - name: Verify any provided tags are supported.
      fail:
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
```

4. 最后、此Play会为您要部署的BeeGFS版本导入BeeGFS HA角色：

```
  tasks:
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_3 # Alternatively specify: beegfs_ha_7_2.
```

> 每个受支持的BeeGFS主要版本都会保留一个BeeGFS HA角色。这样、用户可以选择何时升级主要/次要版本。目前为BeeGFS 7.3.x (`beegfs_7_3`)或BeeGFS 7.2.x (`beegfs_7_2`)。默认情况下、这两个角色将在发布时部署最新的BeeGFS修补程序版本、但用户可以选择覆盖此修补程序、并根据需要部署最新的修补程序。请参见最新信息 "升级指南" 有关详细信息：

5. 可选：如果要定义其他任务、请记住这些任务是否应定向到 `all` 主机(包括E系列存储系统)或仅文件节点。如果需要、可使用定义一个专门针对文件节点的新Play - `hosts: ha_cluster`。

单击 "此处" 有关完整攻略手册文件的示例。

安装NetApp Ansible Collections

Ansible和所有依赖关系的BeeGFS收集将在上进行维护 "Ansible Galax河"。在Ansible控制节点上、运行以下命令以安装最新版本：

```
ansible-galaxy collection install netapp_eseries.beegfs
```

虽然通常不建议这样做、但也可以安装此集合的特定版本：

```
ansible-galaxy collection install netapp_eseries.beegfs:
==<MAJOR>.<MINOR>.<PATCH>
```

运行攻略手册

位于包含的Ansible控制节点上的目录中 `inventory.yml` 和 `playbook.yml` 文件、请按如下所示运行攻略手册：

```
ansible-playbook -i inventory.yml playbook.yml
```

根据集群的大小、初始部署可能需要20分钟以上的时间。如果部署因任何原因失败、只需更正任何问题(例如布线不当、节点未启动等)、然后重新启动Ansible攻略手册即可。

指定时 "通用文件节点配置"、如果您选择默认选项让Ansible自动管理基于连接的身份验证、则为 `connAuthFile` 现在、您可以在中找到用作共享密钥的 `<playbook_dir>/files/beegfs/<sysMgmtdHost>_connAuthFile` (默认情况下)。任何需要访问文件系统的客户端都需要使用此共享密钥。如果使用配置客户端、则会自动处理此问题 "BeeGFS客户端角色"。

部署BeeGFS客户端

也可以使用Ansible配置BeeGFS客户端并挂载文件系统。

概述

要访问BeeGFS文件系统、需要在需要挂载文件系统的每个节点上安装和配置BeeGFS客户端。本节介绍如何使用可用执行这些任务 "Ansible角色"。

步骤

创建客户端清单文件

1. 如果需要、请从Ansible控制节点向要配置为BeeGFS客户端的每个主机设置无密码SSH：

```
ssh-copy-id <user>@<HOSTNAME_OR_IP>
```

2. 下 `host_vars/`下、为名为的每个BeeGFS客户端创建一个文件 `<HOSTNAME>.yml` 使用以下内容、在占位符文本中填写适用于您环境的正确信息：

```
# BeeGFS Client
ansible_host: <MANAGEMENT_IP>
```

3. 如果要使用NetApp E系列主机集合的角色配置InfiniBand或以太网接口、以便客户端连接到BeeGFS文件节点、也可以包括以下选项之一：

   a. 网络类型为 "InfiniBand (使用IPoIB)"：

```
eseries_ipoib_interfaces:
- name: <INTERFACE>  # Example: ib0 or i1b
  address: <IP/SUBNET> # Example: 100.127.100.1/16
- name: <INTERFACE>  # Additional interfaces as needed.
  address: <IP/SUBNET>
```

   b. 网络类型为 "基于融合以太网的RDMA (RoCE)"：

```
eseries_roce_interfaces:
- name: <INTERFACE>  # Example: eth0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
- name: <INTERFACE>  # Additional interfaces as needed.
  address: <IP/SUBNET>
```

   c. 网络类型为 "以太网(仅限TCP、无RDMA)"：

```
eseries_ip_interfaces:
- name: <INTERFACE>  # Example: eth0.
  address: <IP/SUBNET> # Example: 100.127.100.1/16
- name: <INTERFACE>  # Additional interfaces as needed.
  address: <IP/SUBNET>
```

4. 创建新文件 `client_inventory.yml` 并指定Ansible应用于连接到每个客户端的用户、Ansible应用于权限升级的密码(这需要 `ansible_ssh_user` 为root用户或具有sudo权限)：

```
# BeeGFS client inventory.
all:
  vars:
    ansible_ssh_user: <USER>
    ansible_become_password: <PASSWORD>
```

> 请勿以纯文本格式存储密码。请改用Ansible Vault (请参见) "Ansible文档" 使用Ansible Vault对内容进行加密)或使用 `--ask-become-pass` 运行攻略手册时的选项。

5. 在中 `client_inventory.yml` 文件中、列出应在下配置为BeeGFS客户端的所有主机 `beegfs_clients` 组、然后参考实时注释、取消注释在系统上构建BeeGFS客户端内核模块所需的任何其他配置：

```
children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        <CLIENT HOSTNAME>:
        # Additional clients as needed.

      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        #eseries_ib_skip: True # Skip installing inbox drivers when
using the IPoIB role.
        #beegfs_client_ofed_enable: True
        #beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"

        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        #eseries_ib_skip: True # Skip installing inbox drivers when
using the IPoIB role.

        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        #eseries_ib_skip: False # Default value.
        #beegfs_client_ofed_enable: False # Default value.
```

> 使用Mellanox OFED驱动程序时、请确保beegfs_client_OFED_include_path指向适用于Linux安装的正确"header include path"。有关详细信息、请参见的BeeGFS文档 "RDMA支持"。

6. 在中 `client_inventory.yml` 文件中、列出要在先前定义的任何下挂载的BeeGFS文件系统 `vars`：

```
        beegfs_client_mounts:
          - sysMgmtdHost: <IP ADDRESS>  # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default "sync" mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
        # Specify additional file system mounts for this or other file
systems.
```

7. 截至BeeGFS 7.2.7和7.3.1 "连接身份验证" 必须配置或显式禁用。根据指定时选择的配置基于连接的身份验证的方式 "通用文件节点配置"、您可能需要调整客户端配置：

   a. 默认情况下、HA集群部署将自动配置连接身份验证并生成 `connauthfile` 该位置将放置/维护在位于的Ansible控制节点上 `<INVENTORY>/files/beegfs/<sysMgmtdHost>_connAuthFile`。默认情况下、BeeGFS客户端角色设置为读取此文件或将其分发到中定义的客户端 `client_inventory.yml`、不需要执行其他操作。

      i. 有关高级选项、请参阅随附的默认值完整列表 "BeeGFS客户端角色"。

   b. 如果选择使用指定自定义密钥 `beegfs_ha_conn_auth_secret` 在中指定 `client_inventory.yml` 文件：

```
beegfs_ha_conn_auth_secret: <SECRET>
```

   c. 如果选择完全禁用基于连接的身份验证 `beegfs_ha_conn_auth_enabled`、在中指定 `client_inventory.yml` 文件：

```
beegfs_ha_conn_auth_enabled: false
```

有关支持的参数的完整列表和其他详细信息、请参见 "完整的BeeGFS客户端文档"。有关客户端清单的完整示例、请单击 "此处"。

创建BeeGFS客户端攻略手册文件

1. 创建新文件 `client_playbook.yml`

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
```

2. 可选：如果要使用NetApp E系列主机集合的角色配置客户端连接到BeeGFS文件系统的接口、请导入与要配置的接口类型对应的角色：

   a. 如果您使用的是使用InfiniBand (IPoIB)：

```
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
```

   b. 如果您使用的是基于融合以太网的RDMA (RoCE)：

```
    - name: Ensure IPoIB is configured
      import_role:
        name: roce
```

   c. 如果您使用的是以太网(仅限TCP、无RDMA)：

```
    - name: Ensure IPoIB is configured
      import_role:
        name: ip
```

3. 最后、导入BeeGFS客户端角色以安装客户端软件并设置文件系统挂载：

```
    # REQUIRED: Install the BeeGFS client and mount the BeeGFS file
system.
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

有关客户端攻略手册的完整示例、请单击 "此处"。

运行BeeGFS客户端攻略手册

要安装/构建客户端并挂载BeeGFS、请运行以下命令：

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

验证BeeGFS部署

在将系统投入生产之前、请验证文件系统部署。

概述

在将BeeGFS文件系统投入生产之前、请执行一些验证检查。

步骤

1. 登录到任何客户端并运行以下命令、以确保所有预期节点均存在/可访问、并且未报告任何不一致或其他问题：

```
beegfs-fsck --checkfs
```

2. 关闭整个集群、然后重新启动它。从任何文件节点运行以下命令：

```
pcs cluster stop --all # Stop the cluster on all file nodes.
pcs cluster start --all  # Start the cluster on all file nodes.
pcs status # Verify all nodes and services are started and no failures
are reported (the command may need to be reran a few times to allow time
for all services to start).
```

3. 将每个节点置于备用状态、并验证BeeGFS服务是否能够故障转移到二级节点。要登录到任何文件节点并运行以下命令：

```
pcs status # Verify the cluster is healthy at the start.
pcs node standby <FILE NODE HOSTNAME> # Place the node under test in
standby.
pcs status # Verify services are started on a secondary node and no
failures are reported.
pcs node unstandby <FILE NODE HOSTNAME> # Take the node under test out
of standby.
pcs status # Verify the file node is back online and no failures are
reported.
pcs resource relocate run # Move all services back to their preferred
nodes.
pcs status # Verify services have moved back to the preferred node.
```

4. 使用IOR和MDTest等性能基准测试工具验证文件系统性能是否满足预期。有关BeeGFS使用的常见测试和参数的示例、请参见 "设计验证" BeeGFS中有关NetApp经验证的架构的章节。

应根据为特定站点/安装定义的验收标准执行其他测试。

管理BeeGFS集群

概述、关键概念和术语

了解如何在部署BeeGFS HA集群后对其进行管理。

概述

本节面向需要在部署BeeGFS HA集群后对其进行管理的集群管理员。即使是熟悉Linux HA集群的用户也应阅读本指南、因为在管理集群方面存在许多差异、尤其是在使用Ansible进行重新配置方面。

关键概念

虽然其中一些概念是在主上介绍的 "术语和概念" 页面上、在BeeGFS HA集群的上下文中重新介绍这些配置非常有用：

"*集群节点：*运行Pacemaker和Corosync服务并加入HA集群的服务器。

*文件节点：*用于运行一个或多个BeeGFS管理、元数据或存储服务的集群节点。

"*块节点：*一种为文件节点提供块存储的NetApp E系列存储系统。这些节点不会加入BeeGFS HA集群、因为它们可以提供自己的独立HA功能。每个节点都包含两个存储控制器、这些控制器可在块层提供高可用性。

"BeeGFS服务：* BeeGFS管理、元数据或存储服务。每个文件节点将运行一个或多个服务、这些服务将使用块节点上的卷来存储其数据。

构建块： BeeGFS文件节点、E系列块节点以及在其上运行的BeeGFS服务的标准化部署、可简化按照NetApp经验证的架构横向扩展BeeGFS HA集群/文件系统的过程。自定义HA集群也受支持、但通常会采用类似的组件方法来简化扩展。

"BeeGFS HA Cluster"：*用于运行由块节点提供支持的BeeGFS服务以高可用性方式存储BeeGFS数据的文件节点数量可扩展。基于行业认可的开源组件Pacemaker和Corosync构建、并使用Ansible进行打包和部署。

"*集群服务："*是指在加入集群的每个节点上运行的Pacemaker和Corosync服务。请注意、如果只需要两个文件节点、则节点可能不会运行任何BeeGFS服务、而是仅以"Tiebreaker "节点的形式加入集群。

"*集群资源："*对于集群中运行的每个BeeGFS服务、您将看到一个BeeGFS监控资源和一个资源组、其中包含BeeGFS目标、IP地址(浮动IP)和BeeGFS服务本身的资源。

"*可访问：*一种用于软件配置、配置管理和应用程序部署的工具、可将基础架构作为代码来启用。如何打包BeeGFS HA集群、以简化在NetApp上部署、重新配置和更新BeeGFS的过程。

"* pcs："* a command line interface available from any of the file nodes in the cluster used to query and control the state of nodes and resources in the cluster.

通用术语

**故障转移：*每个BeeGFS服务都有一个首选文件节点、除非该节点发生故障。当BeeGFS服务在非首选/二级文件节点上运行时、该服务称为处于故障转移状态。

"*。*故障恢复：*将BeeGFS服务从非首选文件节点移回其首选节点的操作。

"HA对："*可以访问同一组块节点的两个文件节点有时也称为HA对。这是整个NetApp通用的一个术语、指可以相互"接管"的两个存储控制器或节点。

"维护模式："会禁用所有资源监控、并阻止Pacemaker移动或以其他方式管理集群中的资源(另请参见上的一节"维护模式"）。

- HA集群：*一个或多个运行BeeGFS服务的文件节点、这些文件节点可以在集群中的多个节点之间进行故障转移、以创建高可用性的BeeGFS文件系统。通常、文件节点会配置为HA对、这些HA对能够在集群中运行一部分BeeGFS服务。

何时使用Ansible与pcs工具

何时应使用Ansible与pcs命令行工具来管理HA集群?

所有集群部署和重新配置任务都应使用外部Ansible控制节点上的Ansible完成。集群状态的临时更改(例如、将节点置于待机状态和非待机状态)通常通过登录到集群的一个节点(最好是未降级或即将进行维护的节点)并使用pcs命令行工具来执行。

修改任何集群配置(包括资源、约束、属性和BeeGFS服务本身)都应始终使用Ansible完成。维护集群是为了维护Ansible清单和攻略手册的最新副本(理想情况下是为了控制源以跟踪更改)。如果需要更改配置、请更新清单并重新运行导入BeeGFS HA角色的Ansible攻略手册。

HA角色将处理将集群置于维护模式的问题、然后在重新启动BeeGFS或集群服务以应用新配置之前进行任何必要的更改。由于在初始部署之外通常不需要完全重新启动节点、因此重新运行Ansible通常被视为"安全"操作步骤、但始终建议在维护时段或非工作时间重新启动任何BeeGFS服务。这些重新启动通常不应发生发生原因 应用程序错误、但可能会影响性能(某些应用程序可能比其他应用程序处理得更好)。

如果您希望将整个集群恢复为完全最佳状态、也可以选择重新运行Ansible、并且在某些情况下、与使用pcs相比、恢复集群状态可能更容易。尤其是在集群因某种原因关闭的紧急情况下、所有节点都备份后、重新运行Ansible可能会比尝试使用pcs更快、更可靠地恢复集群。

检查集群的状态

使用pcs查看集群的状态。

概述

正在运行 `pcs status` 从任何集群节点上、都是查看集群整体状态和每个资源(例如BeeGFS服务及其依赖关系)状态的最简单方法。本节将介绍的输出内容 `pcs status` 命令：

了解输出 `pcs status`

运行 `pcs status` 在启动了集群服务(Pacemaker和Corosync)的任何集群节点上。输出顶部将显示集群的摘要：

```
[root@ictad22h01 ~]# pcs status
Cluster name: hacluster
Cluster Summary:
  * Stack: corosync
  * Current DC: ictad22h01 (version 2.0.5-9.el8_4.3-ba59be7122) -
partition with quorum
  * Last updated: Fri Jul  1 13:37:18 2022
  * Last change:  Fri Jul  1 13:23:34 2022 by root via cibadmin on
ictad22h01
  * 6 nodes configured
  * 235 resource instances configured
```

以下部分列出了集群中的节点：

```
Node List:
  * Node ictad22h06: standby
  * Online: [ ictad22h01 ictad22h02 ictad22h04 ictad22h05 ]
  * OFFLINE: [ ictad22h03 ]
```

这主要表示处于备用或脱机状态的任何节点。处于备用状态的节点仍在参与集群、但被标记为不符合运行资源的条件。处于脱机状态的节点表示该节点上未运行集群服务、原因可能是已手动停止或节点已重新启动/关闭。

> 节点首次启动时、集群服务将停止、需要手动启动、以避免意外将资源故障恢复到运行状况不正常的节点。

如果节点由于非管理原因(例如故障)处于备用或脱机状态、则会在节点状态旁边用圆括号显示其他文本。例如、如果禁用了隔离、并且某个资源遇到了您将看到的故障 `Node <HOSTNAME>: standby (on-fail)`。另一种可能的状态是 `Node <HOSTNAME>: UNCLEAN (offline)`、该节点会被简要视为已隔离节点、但如果隔离失败、则此隔离将持续存在、指示集群无法确认节点的状态(这可能会阻止其他节点上启动资源)。

下一节显示了集群中所有资源及其状态的列表：

```
Full List of Resources:
  * mgmt-monitor    (ocf::eseries:beegfs-monitor):   Started ictad22h01
  * Resource Group: mgmt-group:
    * mgmt-FS1  (ocf::eseries:beegfs-target):    Started ictad22h01
    * mgmt-IP1  (ocf::eseries:beegfs-ipaddr2):   Started ictad22h01
    * mgmt-IP2  (ocf::eseries:beegfs-ipaddr2):   Started ictad22h01
    * mgmt-service  (systemd:beegfs-mgmtd):  Started ictad22h01
[...]
```

与节点类似、如果资源存在任何问题、则会在资源状态旁边用圆括号显示其他文本。例如、如果Pacemaker请求停止资源、但此操作无法在分配的时间内完成、则Pacemaker将尝试隔离此节点。如果禁用了隔离或隔离操作失败、则资源状态将为 `FAILED <HOSTNAME> (blocked)` Pacemaker将无法在其他节点上启动它。

值得注意的是、BeeGFS HA集群使用了许多经过BeeGFS优化的自定义OCF资源代理。特别是、BeeGFS监控器负责在特定节点上的BeeGFS资源不可用时触发故障转移。

重新配置和更新

重新配置HA集群和BeeGFS

使用Ansible重新配置集群。

概述

通常、应通过更新Ansible清单并重新运行来重新配置BeeGFS HA集群的任何方面 `ansible-playbook` 命令：其中包括更新警报、更改永久隔离配置或调整BeeGFS服务配置。这些设置可使用进行调整 `group_vars/ha_cluster.yml` 可在中找到文件和完整的选项列表 "指定通用文件节点配置" 部分。

有关管理员在执行集群维护或维护时应注意的部分配置选项的其他详细信息、请参见下文。

如何禁用和启用隔离

设置集群时、默认情况下会启用/需要隔离。在某些情况下、可能需要暂时禁用隔离、以确保在执行某些维护操作(例如升级操作系统)时节点不会意外关闭。虽然可以手动禁用此功能、但管理员应注意一些利弊权衡。

选项1：使用Ansible禁用隔离(建议)。

使用Ansible禁用隔离后、BeeGFS监控器的"发生故障"操作将从"隔离"更改为"备用"。这意味着、如果BeeGFS监控器检测到故障、它将尝试将节点置于备用状态并对所有BeeGFS服务进行故障转移。在外部主动故障排除/测试中、这通常比选项2更理想。缺点是、如果某个资源无法在原始节点上停止、则会被阻止从其他位置启动(这就是生产集群通常需要隔离的原因)。

1. 位于Ansible清单中 `groups_vars/ha_cluster.yml` 添加以下配置：

```
beegfs_ha_cluster_crm_config_options:
    stonith-enabled: False
```

2. 重新运行Ansible攻略手册、将所做的更改应用于集群。

选项2：手动禁用隔离。

在某些情况下、您可能希望在不重新运行Ansible的情况下临时禁用隔离、以便于对集群进行故障排除或测试。

> 在此配置中、如果BeeGFS监控器检测到故障、集群将尝试停止相应的资源组。它不会触发完全故障转移、也不会尝试重新启动受影响的资源组或将其移动到其他主机。要进行恢复、请解决所有问题、然后运行 `pcs resource cleanup` 或者手动将节点置于备用状态。

步骤

1. 要确定是否全局启用或禁用了隔离(stonith")、请运行： `pcs property show stonith-enabled`
2. 要禁用隔离、请运行： `pcs property set stonith-enabled=false`

3. 要启用隔离、请运行： `pcs property set stonith-enabled=true`

注意：下次运行Ansible攻略手册时、此设置将被覆盖。

更新HA集群和BeeGFS

使用Ansible更新BeeGFS和HA集群。

概述

BeeGFS的版本已按照A进行版本控制 `major.minor.patch` 为每个受支持的BeeGFS提供了版本控制方案和BeeGFS HA Ansible角色 `major.minor` 版本(例如 `beegfs_ha_7_2` 和 `beegfs_ha_7_3`）。在发布Ansible集合时、每个HA角色都固定到最新的BeeGFS修补程序版本。

所有BeeGFS升级都应使用Ansible、包括在BeeGFS的主要、次要和修补版本之间移动。要更新BeeGFS、您首先需要更新BeeGFS Ansible集合、该集合还将提供有关部署/管理自动化和底层HA集群的最新修复和增强功能。即使更新到最新版本的集合、BeeGFS也不会升级、直到 `ansible-playbook` 与一起运行 `-e "beegfs_ha_force_upgrade=true"` 设置。

> 有关BeeGFS版本的详细信息、请参见 "BeeGFS升级文档"。

已测试升级路径

每个版本的BeeGFS集合都会使用特定版本的BeeGFS进行测试、以确保所有组件之间的互操作性。此外、还会执行测试、以确保可以从收集的最后一个版本支持的BeeGFS版本升级到最新版本支持的版本。

原始版本	升级版本	多导轨	详细信息
7.2.6	7.3.2	是的。	将beegfs收集从v3.0.1升级到v3.1.0、添加了多轨
7.2.6	7.2.8	否	将beegfs收集从v3.0.1升级到v3.1.0
7.2.8	7.3.1	是的。	使用beegfs集合v3.1.0进行升级、并添加了多轨
7.3.1	7.3.2	是的。	使用beegfs Collection v3.1.0进行升级

BeeGFS升级步骤

以下各节介绍了更新BeeGFS Ansible集合和BeeGFS本身的步骤。请特别注意更新BeeGFS主要或次要版本的任何额外步骤。

第1步：升级BeeGFS集合

用于升级可访问的集合 "Ansible Galax河"下，运行以下命令：

```
ansible-galaxy collection install netapp_eseries.beegfs --upgrade
```

要进行脱机收集升级、请从下载此收集 "Ansible Galax河" 单击所需的 `Install Version` 然后 `Download tarball`。将tarball传输到Ansible控制节点并运行以下命令。

```
ansible-galaxy collection install netapp_eseries-beegfs-<VERSION>.tar.gz
--upgrade
```

请参见 "正在安装收集" 有关详细信息 ...

第2步：更新可逆清单

对集群的Ansible清单文件进行任何必要或所需的更新。请参见 "版本升级说明" 有关特定升级要求的详细信息、请参见下面的部分。请参见 "使用自定义架构" 第节介绍了有关配置BeeGFS HA清单的一般信息。

第3步：更新Ansible攻略手册(仅在更新主要或次要版本时)

如果要在主要或次要版本之间移动、请在中 `playbook.yml` 用于部署和维护集群的文件、请更新的名称 `beegfs_ha_<VERSION>` 用于反映所需版本的角色。例如、如果您要部署BeeGFS 7.3、则此值为 `beegfs_ha_7_3`：

```
- hosts: all
  gather_facts: false
  any_errors_fatal: true
  collections:
    - netapp_eseries.beegfs
  tasks:
    - name: Ensure BeeGFS HA cluster is setup.
      ansible.builtin.import_role:  # import_role is required for tag
availability.
        name: beegfs_ha_7_3
```

有关此攻略手册文件内容的更多详细信息、请参见 "部署BeeGFS HA集群" 部分。

第4步：运行BeeGFS升级

要应用BeeGFS更新、请执行以下操作：

```
ansible-playbook -i inventory.yml beegfs_ha_playbook.yml -e
"beegfs_ha_force_upgrade=true" --tags beegfs_ha
```

BeeGFS HA角色将在后台处理：

- 确保集群处于最佳状态、每个BeeGFS服务都位于其首选节点上。
- 将集群置于维护模式。
- 更新HA集群组件(如果需要)。
- 按照以下步骤逐个升级每个文件节点：
  - 将其置于备用状态并将其服务故障转移到二级节点。

- 升级BeeGFS软件包。
- 回退服务。

- 将集群移出维护模式。

版本升级说明

从BeeGFS 7.2.6或7.3.0版升级

对基于连接的身份验证进行的更改

7.3.1之后发布的BeeGFS版本将不再允许在未指定的情况下启动服务 `connAuthFile` 或设置 `connDisableAuthentication=true` 在服务的配置文件中。强烈建议启用基于连接的身份验证安全性。请参见 "基于BeeGFS连接的身份验证" 有关详细信息 ...

默认情况下、 `beegfs_ha*` 角色将生成并分发此文件、同时将其添加到的Ansible控制节点 `<playbook_directory>/files/beegfs/<beegfs_mgmt_ip_address>_connAuthFile`。。 `beegfs_client` 角色还会检查此文件是否存在、并将其提供给客户端(如果有)。

> 如果 `beegfs_client` 角色未用于配置客户端、需要手动将此文件分发到每个客户端和 `connAuthFile` 中的配置 `beegfs-client.conf` 要使用的文件集。从先前版本的BeeGFS升级时、如果未启用基于连接的身份验证、则客户端将丢失访问权限、除非在升级过程中通过设置禁用基于连接的身份验证 `beegfs_ha_conn_auth_enabled: false` 在中 `group_vars/ha_cluster.yml` (不建议)。

有关其他详细信息和备用配置选项、请参见中配置连接身份验证的步骤 "指定通用文件节点配置" 部分。

维护和维护

故障转移和故障恢复服务

在集群节点之间移动BeeGFS服务。

概述

BeeGFS服务可以在集群中的节点之间进行故障转移、以确保客户端能够在节点发生故障或您需要执行计划内维护时继续访问文件系统。本节介绍管理员在从故障中恢复后修复集群或在节点之间手动移动服务的各种方法。

步骤

故障转移和故障恢复

故障转移(计划内)

通常、当您需要使单个文件节点脱机进行维护时、您需要从该节点移动(或消耗)所有BeeGFS服务。要实现此目的、可以先将节点置于备用状态：

```
pcs node standby <HOSTNAME>
```

验证后使用 `pcs status` 已在备用文件节点上重新启动所有资源、您可以根据需要关闭此节点或对其进行其他

更改。

故障恢复(计划内故障转移之后)

准备好先将BeeGFS服务还原到首选节点时、请先运行 `pcs status` 并在"节点列表"中验证状态是否为备用。如果节点已重新启动、它将显示脱机、直到您使集群服务联机为止：

```
pcs cluster start <HOSTNAME>
```

节点联机后、使用以下命令将其置于备用状态：

```
pcs cluster node unstandby <HOSTNAME>
```

最后、使用以下命令将所有BeeGFS服务重新定位回其首选节点：

```
pcs resource relocate run
```

故障恢复(计划外故障转移之后)

如果节点发生硬件或其他故障、HA集群应自动做出响应并将其服务移动到运行正常的节点、以便管理员有时间采取更正操作。在继续操作之前、请参考 "故障排除" 第节确定故障转移的发生原因 并解决任何未解决的问题。在节点重新启动并运行正常后、您可以继续执行故障恢复。

如果节点在计划外(或计划内)重新启动后启动、则集群服务不会设置为自动启动、因此您首先需要使用以下命令使节点联机：

```
pcs cluster start <HOSTNAME>
```

接下来、清理任何资源故障并重置节点的隔离历史记录：

```
pcs resource cleanup node=<HOSTNAME>
pcs stonith history cleanup <HOSTNAME>
```

在中验证 `pcs status` 节点已联机且运行状况良好。默认情况下、BeeGFS服务不会自动进行故障恢复、以避免意外将资源移回运行状况不正常的节点。准备好后、使用以下命令将集群中的所有资源返回到其首选节点：

```
pcs resource relocate run
```

将单个**BeeGFS**服务移动到备用文件节点

将BeeGFS服务永久移动到新文件节点

如果要永久更改单个BeeGFS服务的首选文件节点、请调整Ansible清单、以便首先列出首选节点、然后重新运行Ansible攻略手册。

例如、在此示例中 `inventory.yml` file、ictad22h01是运行BeeGFS管理服务的首选文件节点：

```
        mgmt:
          hosts:
            ictad22h01:
            ictad22h02:
```

反转顺序将使ictad22h02上的管理服务成为首选发生原因：

```
        mgmt:
          hosts:
            ictad22h02:
            ictad22h01:
```

临时将BeeGFS服务移动到备用文件节点

通常、如果某个节点正在进行维护、您需要使用[故障转移和故障恢复步骤](#故障转移和故障恢复)将所有服务从该节点移出。

如果出于某种原因需要将单个服务移动到其他文件节点、请运行：

```
pcs resource move <SERVICE>-monitor <HOSTNAME>
```

> 请勿指定单个资源或资源组。请始终为要重新定位的BeeGFS服务指定监控器的名称。例如、要将BeeGFS管理服务移动到ictad22h02、请运行： `pcs resource move mgmt-monitor ictad22h02`。可以重复执行此过程、将一个或多个服务从其首选节点移出。使用进行验证 `pcs status` 服务已在新节点上重新定位/启动。

要将BeeGFS服务移回其首选节点、请先清除临时资源限制(根据需要对多个服务重复此步骤)：

```
pcs resource clear <SERVICE>-monitor
```

然后、在准备好将服务实际移回其首选节点时、运行：

```
pcs resource relocate run
```

注意：此命令将重新定位不再具有临时资源限制的任何服务、而这些服务不位于其首选节点上。

将集群置于维护模式

防止HA集群意外对环境中的预期更改做出响应。

概述

将集群置于维护模式将禁用所有资源监控、并阻止Pacemaker移动或以其他方式管理集群中的资源。所有资源都将在其原始节点上保持运行状态、无论是否出现临时故障情况、从而使其无法访问。建议/有用的情形包括：

- 网络维护、可能会暂时中断文件节点与BeeGFS服务之间的连接。
- 块节点升级。
- 文件节点操作系统、内核或其他软件包更新。

通常、手动将集群置于维护模式的唯一原因是、防止集群对环境中的外部更改做出响应。如果集群中的单个节点需要物理修复、请勿使用维护模式、只需按照上述操作步骤 将该节点置于备用状态即可。请注意、重新运行Ansible会自动将集群置于维护模式、以便于进行大多数软件维护、包括升级和配置更改。

步骤

要检查集群是否处于维护模式、请运行：

```
pcs property show maintenance-mode
```

当集群正常运行时、此操作将返回false。要启用维护模式、请运行：

```
pcs property set maintenance-mode=true
```

您可以通过运行pcs状态并确保所有资源显示"(非受管)"来进行验证。要使集群退出维护模式、请运行：

```
pcs property set maintenance-mode=false
```

停止并启动集群

正常停止和启动HA集群。

概述

本节介绍如何正常关闭和重新启动BeeGFS集群。可能需要执行此操作的示例场景包括电气维护或在数据中心或机架之间迁移。

步骤

如果出于任何原因需要停止整个BeeGFS集群并关闭、则会运行所有服务：

```
pcs cluster stop --all
```

也可以在单个节点上停止集群(这将自动将服务故障转移到另一个节点)、但建议先将该节点置于备用状态(请参见 "故障转移" 部分)：

```
pcs cluster stop <HOSTNAME>
```

要在所有节点上启动集群服务和资源、请运行：

```
pcs cluster start --all
```

或使用以下命令在特定节点上启动服务：

```
pcs cluster start <HOSTNAME>
```

此时、请运行 `pcs status` 并验证集群和BeeGFS服务是否已在所有节点上启动、以及服务是否正在预期节点上运行。

> 根据集群的大小、整个集群可能需要一段时间(秒到分钟)才能停止、或者显示已在中启动 `pcs status`。条件 `pcs cluster <COMMAND>` 挂起超过五分钟、然后运行Ctrl+C以取消命令、登录到集群中的每个节点并使用 `pcs status` 查看集群服务(CorosynC/Pacemaker)是否仍在该节点上运行。您可以从集群仍处于活动状态的任何节点上检查阻止集群的资源。手动输入问题描述 地址、然后命令应完成或可以重新运行以停止任何剩余服务。

替换文件节点

如果原始服务器出现故障、请更换文件节点。

概述

本节概述了更换集群中的文件节点所需的步骤。以下步骤假定文件节点因硬件问题描述 而失败、并替换为新的相同文件节点。

步骤

1. 物理更换文件节点、并还原到块节点和存储网络的所有布线。
2. 在文件节点上重新安装操作系统、包括添加Red Hat订阅。
3. 在文件节点上配置管理和BMC网络。
4. 如果主机名、IP、PCIe到逻辑接口映射或有关新文件节点的任何其他更改、请更新Ansible清单。通常、如果节点已更换为相同的服务器硬件、而您使用的是原始网络配置、则不需要执行此操作。
    a. 例如、如果主机名发生更改、请创建(或重命名)节点的清单文件 (`host_vars/<NEW_NODE>.yml`)、然后在Ansible清单文件中 (`inventory.yml`)、将旧节点的名称替换为新节点名称：

```
all:
    ...
    children:
    ha_cluster:
        children:
        mgmt:
            hosts:
            node_h1_new:    # Replaced "node_h1" with "node_h1_new"
            node_h2:
```

5. 从集群中的其他一个节点中、删除旧节点：`pcs cluster node remove <HOSTNAME>`。

> 请勿在运行此步骤之前继续操作。

6. 在Ansible控制节点上：
   a. 使用删除旧SSH密钥：

```
`ssh-keygen -R <HOSTNAME_OR_IP>`
```

   b. 使用以下命令为替换节点配置无密码SSH：

```
ssh-copy-id <USER>@<HOSTNAME_OR_IP>
```

7. 重新运行Ansible攻略手册以配置节点并将其添加到集群：

```
ansible-playbook -i <inventory>.yml <playbook>.yml
```

8. 此时、请运行 `pcs status` 并验证已更换的节点现在是否已列出并正在运行服务。

扩展或缩减集群

在集群中添加或删除组件。

概述

本节介绍了调整BeeGFS HA集群大小的各种注意事项和选项。通常、可以通过添加或删除组件来调整集群大小、这些组件通常是将两个文件节点设置为HA对的组件。如果需要、还可以添加或删除单个文件节点(或其他类型的集群节点)。

向集群添加组件

注意事项

通过添加更多组件来扩展集群是一个简单的过程。开始之前、请牢记每个HA集群中集群节点的最小和最大数量限制、并确定是否应向现有HA集群添加节点或创建新的HA集群。通常、每个组件都包含两个文件节点、但每个集群的最小节点数为三个节点(用于建立仲裁)、建议的最大值为十个(已测试)。对于高级情形、可以在部署双节点集群时添加一个不运行任何BeeGFS服务的"Tiebreaker "节点。如果您考虑进行此类部署、请联系NetApp支持部门。

在决定如何扩展集群时、请记住这些限制以及任何预期的未来集群增长。例如、如果您有一个六节点集群、并且需要再添加四个节点、则建议您只启动一个新的HA集群。

> 请记住、一个BeeGFS文件系统可以由多个独立的HA集群组成。这样、文件系统就可以继续扩展、远远超出底层HA集群组件的建议/硬限制。

步骤

向集群添加组件时、您需要创建 `host_vars` 每个新文件节点和块节点(E系列阵列)的文件。需要将这些主机的名称以及要创建的新资源添加到清单中。对应的 `group_vars` 需要为每个新资源创建文件。请参见 "使用自定义架构" 部分以了解详细信息。

创建正确的文件后、只需使用命令重新运行自动化即可：

```
ansible-playbook -i <inventory>.yml <playbook>.yml
```

从集群中删除组件

在需要停用组件时、需要牢记一些注意事项、例如：

- 此组件正在运行哪些BeeGFS服务？
- 是否只是文件节点停用、块节点应附加到新的文件节点？
- 如果停用整个组件、数据应移至新的组件、分散到集群中的现有节点中、还是移动到新的BeeGFS文件系统或其他存储系统？
- 是否可以在中断期间发生这种情况、或者是否应在无中断的情况下进行？
- 该组件是否正在使用中、或者它是否主要包含不再处于活动状态的数据？

由于可能的起始点和所需的最终状态各不相同、因此请联系NetApp支持部门、以便我们能够根据您的环境和要求确定并帮助实施最佳策略。

故障排除

对BeeGFS HA集群进行故障排除。

概述

本节将介绍如何调查各种故障以及在运行BeeGFS HA集群时可能出现的其他情形并对其进行故障排除。

故障排除指南

正在调查意外故障转移

当节点意外隔离并将其服务移至另一个节点时、第一步应是查看集群是否在底部指示任何资源故障 `pcs status`。如果成功完成隔离且资源已在另一节点上重新启动、则通常不会显示任何内容。

通常、下一步是使用搜索整个systemd日志 `journalctl` 在其余任何一个文件节点上(所有节点上的Pacemaker日志均会同步)。如果您知道发生故障的时间、则可以在发生故障之前开始搜索(通常建议至少提前十分钟)：

```
journalctl --since "<YYYY-MM-DD HH:MM:SS>"
```

以下各节显示了您可以在日志中查看的常见文本、以进一步缩小调查范围。

调查/解决步骤

第1步：检查BeeGFS监控器是否检测到故障：

如果故障转移是由BeeGFS监控器触发的、则应看到错误(如果不是、请继续执行下一步)。

```
journalctl --since "<YYYY-MM-DD HH:MM:SS>" | grep -i unexpected
[...]
Jul 01 15:51:03 ictad22h01 pacemaker-schedulerd[9246]:  warning:
Unexpected result (error: BeeGFS service is not active!) was recorded for
monitor of meta_08-monitor on ictad22h02 at Jul  1 15:51:03 2022
```

在本示例中、BeeGFS服务meta_08因某种原因停止。要继续进行故障排除、我们应启动ictad22h02并查看服务的日志、网址为 `/var/log/beegfs-meta-meta_08_tgt_0801.log`。例如、BeeGFS服务可能由于内部问题描述 或节点问题而遇到应用程序错误。

> 与Pacemaker中的日志不同、BeeGFS服务的日志不会分发到集群中的所有节点。要调查这些类型的故障、需要原始节点中发生故障的日志。

监控器可能报告的问题包括：

- 无法访问目标！
  - 问题描述 ： 表示无法访问块卷。
  - 故障排除：
    - 如果此服务也无法在备用文件节点上启动、请确认块节点运行状况良好。
    - 检查是否存在任何可能阻止从此文件节点访问块节点的物理问题、例如、InfiniBand适配器或缆线出现故障。
- 无法访问网络！
  - 问题描述 ： 客户端用于连接到此BeeGFS服务的适配器均未联机。
  - 故障排除：

    - 如果多个/所有文件节点受到影响、请检查用于连接BeeGFS客户端和文件系统的网络是否存在故障。
    - 检查是否存在任何可能阻止从此文件节点访问客户端的物理问题、例如InfiniBand适配器或缆线出现故障。
- BeeGFS服务未处于活动状态！
  - 问题描述：BeeGFS服务意外停止。
  - 故障排除：
    - 在报告错误的文件节点上、检查受影响的BeeGFS服务的日志以查看它是否报告崩溃。如果发生这种情况、请向NetApp支持部门创建一个案例、以便可以调查崩溃情况。
    - 如果BeeGFS日志中未报告任何错误、请检查日志日志以查看systemd是否记录了服务停止的原因。在某些情况下、BeeGFS服务可能没有机会在进程终止之前记录任何消息(例如、有人运行时) `kill -9 <PID>`）。

第**2**步：检查节点是否意外离开集群

如果节点发生了一些灾难性硬件故障(例如、系统主板已损坏)或发生内核崩溃或类似的软件问题描述 、BeeGFS监控器将不会报告错误。而是查找主机名、此时您应看到Pacemaker发出的消息、指出节点意外丢失：

```
journalctl --since "<YYYY-MM-DD HH:MM:SS>" | grep -i <HOSTNAME>
[...]
Jul 01 16:18:01 ictad22h01 pacemaker-attrd[9245]:  notice: Node ictad22h02
state is now lost
Jul 01 16:18:01 ictad22h01 pacemaker-controld[9247]:  warning:
Stonith/shutdown of node ictad22h02 was not expected
```

第**3**步：验证**Pacemaker**是否能够隔离节点

在所有情况下、您都应看到Pacemaker尝试隔离节点、以验证其是否实际脱机(确切消息可能因隔离的发生原因而异)：

```
Jul 01 16:18:02 ictad22h01 pacemaker-schedulerd[9246]:  warning: Cluster
node ictad22h02 will be fenced: peer is no longer part of the cluster
Jul 01 16:18:02 ictad22h01 pacemaker-schedulerd[9246]:  warning: Node
ictad22h02 is unclean
Jul 01 16:18:02 ictad22h01 pacemaker-schedulerd[9246]:  warning:
Scheduling Node ictad22h02 for STONITH
```

如果隔离操作成功完成、您将看到如下消息：

```
Jul 01 16:18:14 ictad22h01 pacemaker-fenced[9243]:  notice: Operation
'off' [2214070] (call 27 from pacemaker-controld.9247) for host
'ictad22h02' with device 'fence_redfish_2' returned: 0 (OK)
Jul 01 16:18:14 ictad22h01 pacemaker-fenced[9243]:  notice: Operation
'off' targeting ictad22h02 on ictad22h01 for pacemaker-
controld.9247@ictad22h01.786df3a1: OK
Jul 01 16:18:14 ictad22h01 pacemaker-controld[9247]:  notice: Peer
ictad22h02 was terminated (off) by ictad22h01 on behalf of pacemaker-
controld.9247: OK
```

如果隔离操作因某种原因失败、则BeeGFS服务将无法在另一个节点上重新启动、以避免出现数据损坏的风险。如果隔离设备(PDU或BMC)不可访问或配置不当、则该问题描述 将单独进行调查。

解决失败的资源操作(位于**pcs**状态的底部)

如果运行BeeGFS服务所需的资源发生故障、BeeGFS监控器将触发故障转移。如果发生这种情况、则底部可能不会列出任何"失败的资源操作" `pcs status` 您应参考有关如何操作的步骤 "计划外故障转移后的故障恢复"。

否则、通常只能在两种情况下看到"资源操作失败"。

调查/解决步骤

场景**1**：检测到使用隔离代理的临时或永久问题描述 、并且已重新启动或将其移动到其他节点。

某些隔离代理比其他隔离代理更可靠、每个隔离代理都将实施自己的监控方法、以确保隔离设备已准备就绪。特别是、我们发现Redffish隔离代理报告的资源操作失败、即使它仍显示为已启动：

```
  * fence_redfish_2_monitor_60000 on ictad22h01 'not running' (7):
call=2248, status='complete', exitreason='', last-rc-change='2022-07-26
08:12:59 -05:00', queued=0ms, exec=0ms
```

如果隔离代理报告某个特定节点上的资源操作失败、则不会触发该节点上运行的BeeGFS服务的故障转移。只需在同一节点或不同节点上自动重新启动即可。

解决步骤：

1. 如果隔离代理始终拒绝在全部或部分节点上运行、请检查这些节点是否能够连接到隔离代理、并验证是否已在Ansible清单中正确配置隔离代理。
   a. 例如、如果Redsfish (BMC)隔离代理与它负责隔离的节点运行在同一个节点上、而操作系统管理和BMC IP位于同一个物理接口上、则某些网络交换机配置将不允许这两个接口之间进行通信(以防止网络环路)。默认情况下、HA集群会尝试避免在其负责隔离的节点上放置隔离代理、但在某些情形/配置中可能会发生这种情况。
2. 解决所有问题后(或者如果问题描述 似乎是临时的)、请运行 `pcs resource cleanup` 重置失败的资源操作。

场景**2**：**BeeGFS**监控器检测到问题描述 并触发故障转移、但出于某种原因、资源无法在二级节点上启动。

如果已启用隔离且未阻止资源在原始节点上停止(请参见"备用(故障)"故障排除部分)、则最可能的原因包括在二级节点上启动资源时出现问题、因为：

- 二级节点已脱机。
- 物理或逻辑配置问题描述 会阻止二级系统访问用作BeeGFS目标的块卷。

解决步骤：

1. 对于失败的资源操作中的每个条目：
   a. 确认失败的资源操作为启动操作。
   b. 根据出现故障的资源操作中指示的资源和指定的节点：
      i. 查找并更正可能会阻止节点启动指定资源的任何外部问题。例如、如果BeeGFS IP地址(浮动IP)无法启动、请确认至少有一个所需接口已连接/联机并已连接到正确的网络交换机。如果BeeGFS目标(块设备/E系列卷)发生故障、请验证与后端块节点的物理连接是否按预期连接、并验证块节点是否运行正常。
   c. 如果没有明显的外部问题、并且您希望在此事件中使用根发生原因 、建议您先与NetApp支持部门一起创建案例进行调查、然后再继续操作、因为以下步骤可能会使根发生原因 分析(Root Analysis、RCA)变得极具挑战性/不可能。
2. 解决任何外部问题后：
   a. 从Ansible inventory.yml文件中注释掉所有无法正常工作的节点、然后重新运行完整的Ansible攻略手册、以确保在二级节点上正确设置所有逻辑配置。
      i. 注意：在节点运行状况良好且您准备好进行故障恢复后、请勿忘记取消对这些节点的注释并重新运行攻略手册。
   b. 或者、您也可以尝试手动恢复集群：
      i. 使用以下命令将所有脱机节点重新联机： `pcs cluster start <HOSTNAME>`
      ii. 使用以下命令清除所有失败的资源操作： `pcs resource cleanup`
      iii. 运行pcs状态并验证所有服务是否按预期启动。
      iv. 如果需要、请运行 `pcs resource relocate run` 将资源移回其首选节点(如果有)。

常见问题

BeeGFS服务在请求时不会进行故障转移或故障恢复

可能的问题描述： `pcs resource relocate` 运行命令已执行、但从未成功完成。

*如何检查：*运行 `pcs constraint --full` 并检查ID为的任何位置约束 `pcs-relocate-<RESOURCE>`。

*如何解决：*运行 `pcs resource relocate clear` 然后重新运行 `pcs constraint --full` 以验证是否已删除额外的约束。

禁用隔离后、处于**pcs**状态的一个节点将显示**"standby (on-fail)"**

可能的问题描述： Pacemaker无法成功确认故障节点上的所有资源均已停止。

如何解决：

1. 运行 `pcs status` 并检查输出底部是否存在未"启动"或显示错误的任何资源、并解决任何问题。
2. 要使节点恢复联机、请运行 `pcs resource cleanup --node=<HOSTNAME>`。

在发生意外故障转移后、如果启用了隔离、则资源将以**pcs**状态显示**"started (on-fail)"**

*可能的问题描述：*发生了一个问题、触发了故障转移、但Pacemaker无法验证节点是否已隔离。发生这种情况的原因可能是隔离配置不当或存在具有隔离代理的问题描述 (例如：PDU已与网络断开连接)。

如何解决：

1. 验证节点是否已实际关闭。

> 如果您指定的节点实际上未关闭、但正在运行集群服务或资源、则会发生数据损坏/集群故障。

2. 手动确认隔离：`pcs stonith confirm <NODE>`

此时、服务应完成故障转移并在另一个运行正常的节点上重新启动。

常见故障排除任务

重新启动单个**BeeGFS**服务

通常、如果需要重新启动BeeGFS服务(例如、为了便于更改配置)、应通过更新Ansible清单并重新运行攻略手册来完成此操作。在某些情况下、可能需要重新启动单个服务以加快故障排除速度、例如更改日志记录级别而无需等待整个攻略手册运行。

> 除非同时将任何手动更改添加到Ansible清单中、否则将在下次运行Ansible攻略手册时还原这些更改。

选项**1**：由系统**d**控制的重新启动

如果存在BeeGFS服务无法使用新配置正确重新启动的风险、请先将集群置于维护模式、以防止BeeGFS监控器检测到服务已停止并触发不需要的故障转移：

```
pcs property set maintenance-mode=true
```

如果需要、可通过对服务配置进行任何更改 `/mnt/<SERVICE_ID>/`**`_config/beegfs-`**`.conf` (例如：`/mnt/meta_01_tgt_0101/metadata_config/beegfs-meta.conf`)、然后使用systemd重新启动它：

```
systemctl restart beegfs-*@<SERVICE_ID>.service
```

示例 `systemctl restart beegfs-meta@meta_01_tgt_0101.service`

选项2：起搏器控制的重新启动

如果您不关心新配置是否可能发生原因 会使服务意外停止(例如、更改日志记录级别)、或者您处于维护窗口而不担心停机、则只需为要重新启动的服务重新启动BeeGFS监控器即可：

```
pcs resource restart <SERVICE>-monitor
```

例如、要重新启动BeeGFS管理服务、请执行以下操作： `pcs resource restart mgmt-monitor`

法律声明

法律声明提供对版权声明、商标、专利等的访问。

版权

"https://www.netapp.com/company/legal/copyright/"

商标

NetApp 、 NetApp 徽标和 NetApp 商标页面上列出的标记是 NetApp 、 Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。

"https://www.netapp.com/company/legal/trademarks/"

专利

有关 NetApp 拥有的专利的最新列表，请访问：

https://www.netapp.com/pdf.html?item=/media/11887-patentspage.pdf

隐私政策

"https://www.netapp.com/company/legal/privacy-policy/"

开放源代码

通知文件提供有关 NetApp 软件中使用的第三方版权和许可证的信息。

"E 系列 /EF 系列 SANtricity 操作系统注意事项"

版权信息

版权所有 © 2024 NetApp, Inc.。保留所有权利。中国印刷。未经版权所有者事先书面许可，本文档中受版权保护的任何部分不得以任何形式或通过任何手段（图片、电子或机械方式，包括影印、录音、录像或存储在电子检索系统中）进行复制。

从受版权保护的 NetApp 资料派生的软件受以下许可和免责声明的约束：

本软件由 NetApp 按“原样”提供，不含任何明示或暗示担保，包括但不限于适销性以及针对特定用途的适用性的隐含担保，特此声明不承担任何责任。在任何情况下，对于因使用本软件而以任何方式造成的任何直接性、间接性、偶然性、特殊性、惩罚性或后果性损失（包括但不限于购买替代商品或服务；使用、数据或利润方面的损失；或者业务中断），无论原因如何以及基于何种责任理论，无论出于合同、严格责任或侵权行为（包括疏忽或其他行为），NetApp 均不承担责任，即使已被告知存在上述损失的可能性。

NetApp 保留在不另行通知的情况下随时对本文档所述的任何产品进行更改的权利。除非 NetApp 以书面形式明确同意，否则 NetApp 不承担因使用本文档所述产品而产生的任何责任或义务。使用或购买本产品不表示获得 NetApp 的任何专利权、商标权或任何其他知识产权许可。

本手册中描述的产品可能受一项或多项美国专利、外国专利或正在申请的专利的保护。

有限权利说明：政府使用、复制或公开本文档受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中“技术数据权利 — 非商用”条款第 (b)(3) 条规定的限制条件的约束。

本文档中所含数据与商业产品和/或商业服务（定义见 FAR 2.101）相关，属于 NetApp, Inc. 的专有信息。根据本协议提供的所有 NetApp 技术数据和计算机软件具有商业性质，并完全由私人出资开发。 美国政府对这些数据的使用权具有非排他性、全球性、受限且不可撤销的许可，该许可既不可转让，也不可再许可，但仅限在与交付数据所依据的美国政府合同有关且受合同支持的情况下使用。除本文档规定的情形外，未经 NetApp, Inc. 事先书面批准，不得使用、披露、复制、修改、操作或显示这些数据。美国政府对国防部的授权仅限于 DFARS 的第 252.227-7015(b)（2014 年 2 月）条款中明确的权利。

商标信息

NetApp、NetApp 标识和 http://www.netapp.com/TM 上所列的商标是 NetApp, Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。